

BDL Design and Formalization Monograph

A Behavior Design Language for Industrial Designers:
Motivation, Mechanically Derived Kernel, Production Toolchain,
Correspondence, Rejected Alternatives and Open Agenda

ZHU ZHEHAO

Living technical record — revision of 2026-09-18

Formal development KCN-judu/BDL_FV at 3b4f11b · production KCN-judu/BDL at f1ce82c

This document is the authoritative narrative record of BDL. It is not a paper and is not written to a page limit; it records what the language is, why each construct exists or was rejected, which theorem or executed example supports each claim, how production implements it, where production deviates from the formal model, and what remains open. Every claim carries a strength label (formally proved, executable example, informed by FV, production-tested, design recommendation). No usability claim in it has been tested with users.

Contents

About this document	4
Claim strength	4
How the document is organized	4
Revision log	5
Part I — Motivation: Behavior as Design Material	6
Introduction	6
The Representation Problem	7
Part II — BDL from the Designer’s Side	9
BDL from the Designer’s Side	9
Designer Interaction Model	11
Part III — The Formal Kernel: Declarations, Identity and Dimensions	16
Method	16
Scale and trust base	16
From Surface to Kernel: Architecture	16
Declarations, Interfaces, and Refinement	17
Zero-input relationships: the unit domain (Phase 12)	19
Semantic Identity and Representation	20
Part IV — The Data and Equation Language	23
The question and the hypothesis	23
What entered the kernel, and why each item could not be derived	23
Polymorphism: rank-1, by families, no kernel type variable	24
Capabilities: the audit that reverted a decision	24
The definitional library	25
Enumerations, records, sets: what was encoded and what was deferred	26
The expressiveness ceiling	26
The natural expression surface (Phase 11)	26
Part V — Reactive and Temporal Semantics	28
A Minimal Reactive Semantics	28
Clock Domains	29
Cross-domain occurrences and lossless buffering (Phase 9a)	30
Part VI — Behavior Systems, Groups and Reuse	33
Behavior components (Phase 8a)	33
Behavior groups (Phase 8b)	34
Hardware interaction	36
Part VII — Quantities, Units, Charts and the Formula Composer’s Formal Basis	37
Three notions kept apart	37
Linear units: no kernel construct	37
The numeric domain, in the open	37
Presentation, and the realistic formula	38
Unit ownership in authoring	38
Affine units: the failure of scale-only units, and the revision	38
Charts: coordinate erasure and conversion functoriality (Phase 10b)	38
The production chart model	39
The Formula Composer’s formal basis	40
Part VIII — Physical Outputs, Hardware Validation and Deployment Capacity	41
Physical Outputs Without Arbitration	41
Target-Specific Hardware Validation	44
Deployment capacity	45
Two remarks across Parts III–VIII	45
Part IX — Production Compiler and Runtime Architecture	47

Four trust layers	47
Crates and semantic ownership	47
The compiler as a pipeline of explicit passes	48
Executable IR and lowering	49
The generated core	49
Embedded execution model	49
Differential testing	50
Project persistence and the daemon	51
The elaboration passes the kernel implies	51
Part X — Studio: The User Interface as a Semantic Projection	53
Three information levels	53
The canvas	53
Groups and components on one canvas	54
Design, Code and Split: views of one project	54
The definition editor and the Formula Composer	54
Partial expressions: ?	55
Simulate, Deploy, Library, Monitor	56
What Studio does not decide	56
Part XI — Formal-to-Production Correspondence and Deviations	57
Correspondence by area	58
Deviations	60
Implementation status snapshot (production 876005c, 2026-09-20)	62
Part XII — Rejected Alternatives and the Minimality Ledger	63
Kernel constructs retained	63
Rejected constructs	64
Decisions that changed	66
Part XIII — Known Limitations and the Open Research Agenda	67
Formal limits, stated once	67
Open research problems	67
Empirical questions, explicitly	68
Risks of the design	70
Related Work	71
Industrial design tools and physical prototyping	71
Visual programming and end-user programming	71
Synchronous languages and functional reactive programming	71
Model-based design, statecharts and systems engineering	71
Typed holes, live and structured editing, projectional editing	72
Dimensional typing and units	72
Effects	72
Resource allocation	72
Formal verification, verified compilation and differential testing	72
Embedded DSLs and toolchains	72
Part XIV — Evidence Ledger, Decision Index and Extraction Notes	73
Evidence ledger	73
Design decision index	75
Possible paper slices (internal)	75
Closing	77
References	78

About this document

This is the *BDL Design and Formalization Monograph*: the authoritative narrative record of the Behavior Design Language — its motivation, its designer-facing vocabulary, its formal kernel and the mechanized design-space exploration that derived it, the derived language constructs, the production compiler and runtime, the Studio authoring environment, the correspondence between the formal model and production, the alternatives that were rejected and why, and the questions that remain open. It is not written to a page limit. It is written so that, months or years from now, a reader can answer why BDL has a construct, why it lacks another, which theorem or counterexample supports each decision, how the idea is implemented, where production differs from the model, what evidence already exists, and which parts could become a standalone paper.

Two repositories are described. The formal development is KCN-judu/BDL_FV, a Lean 4 project (Lean 4.33.1, no external libraries); its state as of this revision is the Phase 13 commit that follows dd44a84 (Phase 12), with 3b4f11b (Phase 11) and b087b66 (Phase 10b) also referenced by their commit ids. The production implementation is KCN-judu/BDL, a Rust toolchain with a Flutter authoring environment; its state as of this revision is commit 876005c (PRP-0001, the Source-provision proposal, 2026-09-20), which follows 3c6c8be (the Unit-domain normalization, ADR-0029, protocol 0.14) and f1ce82c; where a statement is only known to hold at an earlier snapshot the text says so. Statements about “production” are statements about 876005c, dated 2026-09-20, and the document says so where the state is likely to move.

Claim strength

Every substantive claim in this document carries one of five labels, used consistently in the text, in the ledgers of Part XIV, and in production’s own records (docs/project/formal-correspondence.md, the fv field of each ADR):

label	means
formally proved	a named Lean theorem proves the stated property of the formal model; it never proves the Rust or Dart code
executable example	a concrete design run through the proved-sound interpreter inside the proof checker (decide), or through production’s differential test harness
informed by FV	a formal result or counterexample bounded an engineering choice
production-tested	executable tests (differential, golden, property, end-to-end) support a production claim
design hypothesis / recommendation	no formal or empirical backing is claimed; a position taken for stated reasons

“Minimal” is never claimed globally. Where the formal development says minimal it means *minimal among the tested candidates*, or *the smallest design found that supports the required cases*, and the text says which.

How the document is organized

Part I states the problem and the design position. Part II is the language as a designer meets it — vocabulary and interaction. Parts III–VIII are the formal development: the kernel of declarations, identity and dimensions; the data and equation language; reactive and temporal semantics including cross-domain buffering; behavior systems and groups; quantities, units, charts and the Formula Composer’s formal basis; outputs, hardware validation and deployment capacity. Parts IX–X describe production: the compiler and runtime, and Studio. Part XI is the formal-to-production correspondence, including an explicit deviations table. Part XII is the rejected-alternatives and minimality ledger. Part XIII is limitations and the open research agenda, including the empirical questions no study has yet answered. Part XIV is the evidence ledger, the design-decision index and notes on which slices could become papers. Related work follows.

Recurring examples are used throughout: the lamp (Tilt → Brightness), a fast sensor read by a slow consumer, a grouped behavior extracted into a component, an Arduino Nano allocation, and Celsius/Fahrenheit with sensor calibration. Where a section introduces a different example it is because none of these exercises the point.

Revision log

milestone	what the record gained
2026-09, conference manuscript (Phases 0–7)	motivation, designer vocabulary, interaction model, the kernel through hardware validation, claim-strength discipline; the manuscript is archived beside this document
Phase 8a/8b — behavior systems and groups	components, fresh instantiation, bindings, flattening, substitutability; groups as authoring metadata and their extraction
Phase 9a — list data and lossless buffering	<code>list</code> τ , the object-language window, capacity as validation
Phase 9b/9c — data and equation language	products, the list recursor, rank-1 definitional polymorphism, the capability audit that reverted structural ordering
Phase 10/10b — units, charts, Formula Composer basis	unit coordinates, exact symbolic scales, local dimension inference, affine charts and the AffSort revision
production Formula Composer (P10a/P10b)	the projection, slots, compose operations, unit ownership
Phase 11 — natural expression surface	binders and ranges as conservative desugaring
the monograph rewrite (69caa3a)	the monograph structure, production architecture, correspondence and deviations, ledgers, open agenda
production P11 and ADR-0029 (3c6c8be)	the natural forms implemented; one canonical type per relationship, the empty product <code>()</code> as the unit domain, protocol 0.13/0.14
Phase 13 — Source provision (PRP-0001 audit)	device channels and profiles, shared-raw provision as an <code>EnvRefines</code> step, transparency, trace abstraction, joint-section exactness, the corrected claims
Phase 12 — unit-domain normalization	<code>()</code> $\rightarrow$ B as an interface normalization whose value is the kernel type B; the source role as a realization state; A $\rightarrow$ <code>()</code> shown unable to name a consumer

Part I — Motivation: Behavior as Design Material

Introduction

Industrial design is increasingly concerned with products whose behavior is determined not only by geometry, materials, and mechanisms, but also by sensing, logic, timing, software, and networked control. A cup may infer that it has been lifted, a lamp may adapt to ambient conditions, a medical device may gate an action on multiple safety conditions, and a consumer robot may continuously map sensor estimates to actuator behavior. In such products, behavior is no longer a late implementation detail. It is part of the product concept.

Yet the dominant design media remain asymmetrical. Industrial designers have mature media for shape, layout, appearance, and physical assembly, while product behavior is usually externalized through prose, storyboards, flowcharts, state diagrams, interactive mock-ups, or ad hoc embedded code. The first four are easy to sketch but weak as executable specifications; the last is executable but forces the designer to adopt implementation-oriented concepts such as mutable variables, callbacks, polling loops, and device APIs. Existing physical prototyping systems such as Phidgets and d.tools substantially lowered the cost of building interactive prototypes [1], [2], but their behavioral representations still inherit important assumptions from programming and state-machine formalisms.

A concrete instance of the resulting cost appeared in a two-day introductory hardware workshop that the author designed and taught for ten participants with no prior embedded experience. Two of the available sensors behaved in opposite ways: the ambient-light sensor reports larger values as illumination increases, while the distance sensor reports smaller values as the target recedes. Participants lost track of which was which, repeatedly and across the whole group. This pattern suggests a representational problem rather than merely a syntactic one. Whether a reading rises or falls with the quantity it measures is a stable fact about a device, and in the code they were writing there was nowhere to record it. It survived only as a sign buried inside an expression and had to be reconstructed each time it was needed. The obstacle was not syntax. A piece of semantic information had no place to live.

BDL draws a different boundary. The goal is not to make engineering representations merely easier for designers to use. The goal is to define a *native representation of product behavior for design itself*, while retaining enough formal structure for static checking, simulation, and eventual implementation.

The system is **BDL**, a Behavior Design Language. BDL is organized around a working hypothesis: when a behavioral relationship is first specified, *what kind of relationship should exist* is frequently settled before its exact implementation is. A designer may know that **Tilt influences Brightness** before deciding the transfer function; that **CupPickedUp** should activate a behavior before deciding which sensor and threshold detect pickup; or that a safety condition should suppress an actuator before choosing the device driver. Therefore the primary design artifact should be the *typed relationship*, not the procedure that computes it.

The central example is a Mapping Block. Instead of decomposing a design into procedural steps such as “read tilt,” “calculate brightness,” and “set the LED,” BDL represents one mapping:

$$? f : \text{Tilt} \rightarrow \text{Brightness} .$$

The mapping may exist before its body. A formula, curve, examples, or a fitted function can later be attached as a definition of the same block. Once a formula is supplied, for example

$$f(\theta) = \text{clamp} \left(0.2 + 0.8 \frac{\theta}{60^\circ}, 0, 1 \right),$$

it inhabits the previously declared signature. The claim attached to this **signature-first** model is deliberately narrow. It is not that designers universally think in signatures before bodies, nor that they should be trained to. It is that an unresolved typed relationship is a legal, statically meaningful state of the design, so that stopping between declaring a relationship and realizing it costs nothing. Whether designers make use of that position, and at what level of task complexity, is an empirical question that remains open; Part XIII states it as such and lists the studies that would answer it.

Signature first; formula attached later

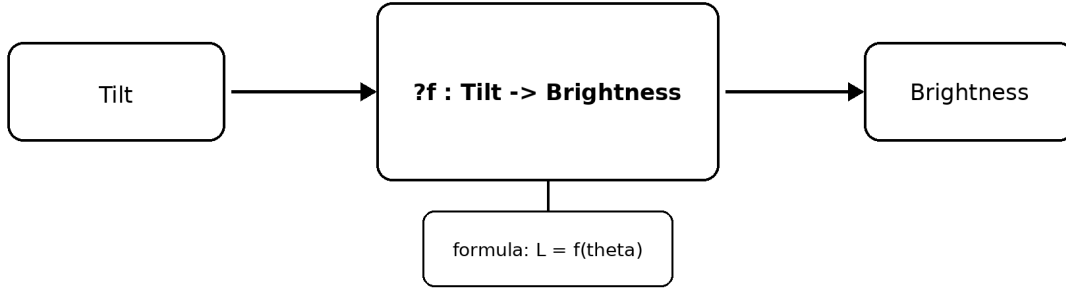


Figure 1: A signature-first Mapping Block. The flow graph contains one semantic relationship, `Tilt -> Brightness`; the formula is attached to the block rather than represented as an additional execution step.

BDL separates a designer-facing surface language from a small formal kernel. The kernel presented here was not designed on paper and then implemented. It was derived by a mechanized design-space exploration in the Lean 4 proof assistant [3]: each candidate construct from an earlier draft of the language was formalized, attacked with counterexamples, reduced to other constructs where possible, and kept only when the tested alternatives failed for a stated reason. The result is smaller than the draft it replaced. Clock-indexed signal types, a separate event type, effect rows, action requests, and runtime actuator arbitration were all removed, each for a reason recorded in the accompanying development. What remains is an environment of named declarations with frozen expected types, monotone public commitments, and write-once realizations; nominal semantic types whose values can be constructed only inside a declaration whose own signature announces the concept; physical dimensions carried in the types of primitive operators; one temporal primitive that reads a clock domain at its previous activation; nominal clock domains with a separate domain judgment; and nominal physical outputs with a single explicit driver each. A validation layer outside the kernel decides whether a design can be placed on a declared target board. Later phases added, each for a reason recorded in Parts IV–VII, list and product data with one recursor, behavior components and groups, an exact model of units and charts, and an interface-level account of the canonical type $() \rightarrow B$ that leaves the kernel without a unit; nothing that was removed has returned.

The conference manuscript from which this document grew presented three views of the same language — the design problem and the designer’s vocabulary, the formal architecture, and the interaction model that connects them — and ended where the formal development then ended, with an elaboration architecture that had not been implemented. The situation has changed. As of production commit 3c6c8be (2026-09-18) an elaborator, a compiler to executable IR and to a `no_std` Rust core, a daemon, and the Studio authoring environment with its Formula Composer exist and are described in Parts IX–X; what has *not* changed is that no user study has been run, and every statement in this document about what designers find natural remains a hypothesis (Part XIII).

The Representation Problem

The target user is not a programmer with fewer syntax skills

Many low-code and visual programming systems reduce textual syntax while preserving the underlying computational ontology: variables, assignments, loops, callbacks, functions, transitions, and scheduling. For software developers this can be convenient. For industrial designers, however, the dominant difficulty is often not syntax but *semantic translation*. The designer begins with a product statement such as “while the cup is held, brightness follows tilt” and must translate it into implementation machinery.

BDL therefore follows a stronger criterion: a surface primitive should be exposed only when it corresponds to a concept that is independently meaningful in the design task. A mutable accumulator used to count samples is generally not such a concept; “three pickup events within ten minutes” is. A polling loop is generally not; “while the product is held” is. A callback is not; “when the button is pressed” is.

The language should directly expose semantic properties, time-varying quantities, discrete occurrences, typed mappings, conditions, behavioral contexts, temporal relations, physical outputs, and safety constraints. By default

it should hide program counters, threads, callbacks, continuations, clock variables, and bus transactions. These may remain inspectable in an expert or debugging view, but they are not the primary design medium.

A flow graph is a dependency view, not a program counter

BDL uses a flow-like canvas because causal and functional paths are useful visual structures. The arrows do **not** mean “execute the left node and then the right node.” They mean that the target relationship depends on the source relationship or value.

Consider the procedural decomposition:

```
[Cup picked up]
|
[Read tilt]
|
[Compute L = f(theta)]
|
[Set LED brightness]
```

This decomposition is rejected as a primary design representation because the “read,” “compute,” and “set” nodes are artifacts of an execution model. The corresponding BDL view is a `held` context containing a mapping from `Tilt` to `Brightness`. The formula is a property of that mapping, and the realization layer later binds the mapping’s value to a physical light output.

The distinction matters because it changes what the designer edits. In a procedure-centric editor, changing an implementation may require rewriting steps and intermediate state. In BDL, changing a transfer function edits the definition of one relationship while preserving the surrounding product logic. This is not only a matter of presentation. Realizing or refining one declaration leaves every other declaration’s typing untouched, and leaves every commitment that was already discharged in place, provided the evidence for those commitments does not depend on what was still unknown.

Cognitive budget

The surface language must remain intentionally small. Each additional primitive has a cost in learnability, visibility, consistency, and error-proneness, all familiar concerns in the Cognitive Dimensions tradition [4]. BDL therefore uses a *cognitive budget*: a new visible construct is justified only when it captures a distinct design concept that cannot be expressed cleanly as a property of an existing construct.

The same budget was applied to the kernel, under a stricter test. A kernel construct earns its place only if removing it makes some design either unrepresentable or ambiguous, and the argument for each is a theorem or a counterexample rather than a preference. Several constructs that the surface exposes as distinct concepts turned out, under that test, to be derived forms. Every temporal modifier reduces to one delay primitive. Every cross-domain policy reduces to one transport primitive plus ordinary data. Every output-selection policy reduces to ordinary computation upstream of a single drive edge.

Progressive disclosure is a semantic property

A design tool can be visually simple and still force premature decisions. BDL instead treats progressive disclosure as part of the language semantics. A design may contain a declaration whose realization is intentionally absent. Later steps may refine the same declaration with mathematical properties, a body, a clock domain, an output binding, and a target board. Each step is either a *refinement*, which preserves everything previously established, or an *edit*, which is permitted but reopens the validation of dependents. That distinction is made precise in the section on declarations, and it is the organizing principle of the interaction model.

Part II — BDL from the Designer’s Side

This part is the language as a designer meets it, unchanged in substance from the conference manuscript except where production has since fixed a detail; where Studio’s realization of an interaction differs from the model described here, Part X says so. The vocabulary is introduced first and the interaction model second.

BDL from the Designer’s Side

This section presents the language as a designer encounters it. Every construct here is a surface form; the architecture section states which forms are kernel primitives and which are derived.

Semantic properties and signatures

The first-class visual object in BDL is a *semantic property*, not a raw scalar. Examples include `Tilt`, `Brightness`, `Temperature`, `CupContact`, and `MotorAngle`. Each semantic property has a representation, and may carry units, ranges, and documentation, but its identity is what the language tracks. `Tilt` and `MotorAngle` may share an angular dimension without being interchangeable design concepts, and the kernel keeps them apart by identity rather than by representation.

A Mapping Block is created from a signature

$$m : (A_1, \dots, A_n) \rightarrow B.$$

At creation time the implementation may be absent:

$$? m : (A_1, \dots, A_n) \rightarrow B.$$

This is a valid partial artifact. The editor can already reject wires with incompatible types, propagate semantic types downstream, show documentation for the intended relationship, and record additional properties such as monotonicity.

This design deliberately exploits the information density of a function type. `Real -> Real` reveals little. `Tilt -> Brightness` already communicates much of the design intent. More refined signatures can add dimensions and representation constraints without forcing those details into the main canvas.

Mapping Blocks

A Mapping Block has a stable identity, a display name, a signature, an optional definition, and a set of declared properties. A definition may be a mathematical expression, a piecewise curve, a set of input-output examples to be fitted, or a reference to an external component. All definition forms elaborate to the same kernel realization, so different authoring styles do not fragment the semantic model.

Three consequences follow from separating identity from definition.

Identity belongs to the signature. The internal identity is stable across every change to the definition; the display name is mutable, so renaming is a refactoring that updates all reference sites rather than a textual edit. Because a mapping can be named before it can be computed, the rest of the model may refer to it, compose with it, and state properties about it while it is still undefined. Declared properties such as monotonicity attach to the identity, not to any particular definition, and other declarations may rely on them.

The unresolved state is a state of the same object. Behind a Mapping Block is a declaration whose realization is optional. An unresolved block is a declaration with no realization; it is not a separate kind of thing. The word *hole* is retained in the editor as a designer-facing metaphor for this state, because it communicates intentional openness better than “declaration without realization.” It is not a kernel concept.

Attaching a definition is a refinement; replacing or detaching one is an edit. A block may be given a definition once as a refinement step. Replacing the definition with a different one, or removing it, is permitted, but it is an edit: other declarations may have discharged commitments through the old definition, and they must be rechecked. The editor may keep several candidate definitions with one active as a surface convenience; whether this reduces to the write-once kernel realization is an open item.

Nothing in this record imposes an authoring order. Declaring a signature and attaching a formula may be a single action, exactly as a type signature and an equation are written together in a functional language. What the record guarantees is that stopping between the two costs nothing.

Time-varying values and occurrences

The designer sees two kinds of temporal thing. A quantity such as tilt or temperature has a value whenever its domain is active. An occurrence such as “button pressed” or “pickup detected” may or may not be present at an activation, optionally with a payload.

The surface may display these differently, because they invite different operations. A quantity is mapped, compared, and held; an occurrence is counted, latched, and used to enter a context. Underneath, the distinction is not one of type. Every declaration is a stream under the tick semantics, and an occurrence is a declaration of optional type. Within a single clock domain that is the whole story. Across domains, where several occurrences may fall between two observations, a buffered transport is required, and it is derived from the same two primitives as everything else temporal.

Time as a modifier, not a waiting instruction

The surface language avoids `wait(300 ms)` as a primary construct because `wait` suggests a suspended sequential thread. Instead, BDL uses temporal modifiers that describe relationships:

- `p for 300 ms;`
- `after e by 2 s;`
- `while p;`
- `until e;`
- `since e;`
- `once e;`
- `every 1 s;`
- `rise p, previous x, count e, hold x e.`

The designer expresses a temporal property; the elaborator emits the declaration shape that carries the required state. Each of these is a derived form over one kernel primitive. `previous`, `hold`, `count`, `since`, `once`, `every`, and `rise` were each elaborated and run on a concrete input trace; the duration-qualified forms `after`, `for`, `while`, and `until` are compositions of these with comparisons and activation and were not separately executed.

Behavioral contexts

A `StateHandler` is not a program-counter location. Its surface meaning is:

Within this product context, these behavioral relationships are active.

A context may be activated by a condition, or entered by one occurrence and left by another. It contains a local flow graph and nested contexts. For example, a `Held` context may be active while the cup is not in contact with the table. Inside it, a `Drinking` sub-context may activate when tilt exceeds a semantic threshold.

Nested contexts form a tree, avoiding the need to flatten every orthogonal concern into a Cartesian-product state machine. BDL still borrows the established value of hierarchical state models [5], but the surface metaphor is a *context containing behavior*, not a transition diagram that the user must manually maintain.

`StateHandler` remains useful at the surface because it gives a name and a boundary to a product context. In the cases examined here, however, that boundary introduces no new execution mechanism. Activation is a Boolean declaration, entry is its rising edge, local temporal state is a delayed cell gated by activation and reset on entry, an inactive context contributes a default, and choosing between the outputs of two contexts is an ordinary conditional in the one declaration that drives the output. The cases examined are condition-scoped activation, entry, reset-on-entry state, inactive default, event-latched activation with exit-wins, state-local output selection, and nested selection with an output. Contexts that carry their own clock domain, and nesting across independently clocked contexts, were not examined, and nothing here says how they would elaborate.

Physical outputs

A design computes values; it does not move hardware. Physical effect happens only through an explicit binding of one declaration to one named physical output, such as a light channel or a motor. Where several behaviors would influence the same output — a safety override and an interaction context, say — they are combined by an ordinary declaration that becomes the single driver of that output. The combination rule, whether priority, blend, maximum, or clamp, is written in the design, where it can be read, rather than resolved by a runtime policy.

This replaces the request-and-policy model of an earlier draft of BDL. The change is discussed in the section on physical outputs, where the tested alternatives are shown either to duplicate the single-driver rule or to move the conflict into a collector that must itself be a policy.

Supplied computation blocks

Some product behavior is genuinely easier to state as code than as a diagram. Recursive filters, Kalman estimators, spectral transforms, and self-tuning controllers are not clarified by being decomposed into wires and formulas. BDL therefore admits computation blocks written in the host language and linked into the generated implementation.

This is the interface across which an engineer supplies a designer with a capability. Because authoring is signature-first, the request can precede the implementation: the designer places `?smooth : Distance -> Distance` and the signature is the specification the engineer works against. The same interface is used by the standard library that ships with the system, so that first-party components cannot rely on facilities denied to third-party ones.

A supplied block is a pure function or a stateful transducer. It may not drive an output. What the kernel can derive for a native mapping must instead be declared for a supplied one: determinism, totality on well-typed inputs, output range, properties relied upon downstream such as monotonicity, worst-case state size, and the update rate it was designed for. These declarations are validation obligations, not typing, and the mechanism by which each is discharged is recorded with it. There is no primitive for reusable stateful components; a component used in several places is instantiated into fresh declarations by the elaborator, because temporal state belongs to declarations rather than to functions.

Designer Interaction Model

The previous section described what a designer sees. This one describes what a designer does. It is written as the interaction architecture the language implies, not as a report on an implemented tool: no editor exists, and every statement about what a designer would find natural is a hypothesis for the evaluation plan to test. The intention is nonetheless concrete. A reader should be able to picture the workspace, what it contains at each stage of a design, and what it says when something is wrong.

The environment is meant to be closer to a behavioral CAD system than to a visual programming IDE. The main canvas holds named product concepts, the relationships between them, the contexts in which those relationships apply, and the physical outputs the product finally drives. Formulas, units, documentation, declared properties, clock information, deployment bindings, and validation status live in a local inspector attached to whatever is selected. Kernel machinery — concept identities, grants, dependency ranks, domain judgments, solver predicates — does not appear as ordinary vocabulary at all. It can be reached through an explanation view, described at the end of this section, but the designer is not expected to go there.

A running scenario

Consider a table lamp that responds to being handled. When the lamp is picked up and tilted, its brightness follows the tilt, so that a user can dim it by tipping it. When it is set down, it holds the brightness it last had. The lamp has a small heater in its base to keep a drink warm, and a temperature sensor beside it; above a warning temperature the lamp should pulse, and above a critical temperature the heater must switch off regardless of anything else the lamp is doing. The product is to run on an Arduino Nano, which will be chosen last.

This is small, but it exercises everything the interaction model has to offer: relationships that are known before their formulas, a behavior that depends on a context, history without explicit state, a safety condition that competes with an interaction for the same output, two quantities that update at very different rates, and a deployment step at which the design meets a board.

Starting from intent

The designer begins by naming the concepts the product is about: `Tilt`, `Brightness`, `Temperature`, `Held`. None of these is a sensor or a number. `Tilt` is the product's orientation as the designer means it, not an accelerometer channel; `Brightness` is how bright the lamp is, not a PWM duty. Creating a concept places it on the canvas and opens an inspector where a description, an expected range, and a unit may be written down — or not.

The first relationship is drawn as an arrow from `Tilt` to `Brightness`. The tool asks for nothing else. The arrow becomes a Mapping Block with the signature `Tilt -> Brightness`, a default name that the designer changes to `dimByTilt`, and an empty body. It is drawn distinctly — the editor may render it as a hole — but not as an error. The design now says that brightness depends on tilt, and says nothing about how.

That statement already does work. A second relationship can be drawn from `dimByTilt` onward; a property such as “increasing in tilt” can be recorded in the inspector; and a wire from `Tilt` directly to a motor-angle concept, had there been one, would be refused. What the designer can do next is unconstrained: attach a formula now, or leave it and continue with the rest of the product. The inspector's status line reads *declared*, and explains that the relationship is named and typed, that other parts of the design may depend on it, and that a definition is still to come.

This is the first answer to why one would use BDL rather than a node editor or a statechart. In a node editor the arrow cannot exist without something to compute; in a statechart the relationship is not a first-class object at all. Here it is the primary object, and its incompleteness is a state the tool understands.

Refining a relationship locally

When the designer opens `dimByTilt`, the canvas does not change. A local editor appears, offering a formula field, a curve to drag, a table of example pairs to fill in and fit, or a reference to a supplied component. Whichever is chosen, the result is a definition attached to the same block; the surrounding canvas continues to show one arrow from `Tilt` to `Brightness`, not the arithmetic inside it.

Suppose the designer sketches a curve: dim at rest, full brightness at about sixty degrees, clamped. The inspector shows the fitted formula alongside the curve, and shows the units it inferred: the input is an angle, the output is a dimensionless level in $[0, 1]$. Because the block's signature already promises `Brightness`, the designer writes a scalar expression and the tool supplies the semantic wrapping; there is no constructor to type. Had the designer instead written an expression that divides tilt by a time, the inspector would object in the block's own terms: *this expression produces an angular rate, but this block promises brightness*. The status line moves from *declared* to *defined*, and, once the definition checks, to *type-valid*.

The formula is local detail. It matters that it is not another box on the canvas, because the canvas is meant to remain a diagram of the product's behavior, legible to someone who does not want to read equations. A reviewer sees that brightness follows tilt; a designer who wants the curve opens the block.

Time and history

The lamp should hold its brightness when set down. The designer selects the wire from `dimByTilt` to the light and adds the qualifier `hold while not Held`. Nothing else is needed. There is no previous-value variable to declare, no timer to reset, and no flag to clear; the phrase is the whole of the temporal specification.

The same vocabulary covers the rest of the product's history. “Pulse when warm” becomes a `Warm` condition and an every 1 s phrase on the pulse. “The lamp has been picked up at least once since power-on” is `once (rise Held)`. “Picked up more than three times in ten minutes” is `count (rise Held) within 10 min > 3`. Each of these appears as an annotation on a relationship or a condition, not as a node with state inside it.

Two things do surface. Every held or delayed quantity has a value at the first tick, and the tool asks for it — what brightness does the lamp show before it has ever been tilted? — because that value is a product decision, not an implementation default. And the status line, once temporal phrases are present, reports *temporally valid* when every such initial value is given and no relationship depends instantaneously on itself. A design in which the pulse rate depended on the pulse, without any delay between them, would be reported at the loop, in the names of the two relationships, with the suggestion to insert a `previous`.

That these phrases are all shapes over one delay primitive, and that the primitive reads a clock domain at its previous activation, is the subject of the reactive section. The designer does not need to know it, though the explanation view will show it.

Contexts as places

The tilt behavior should only apply while the lamp is held. The designer draws a context named `Held`, with its activation condition attached — the lamp is off the table — and drags `dimByTilt` inside it. Visually the block is now within a region; semantically, the relationship applies while the region is active. A `Drinking` sub-context can be nested inside `Held`, entered when tilt exceeds a threshold and left when it falls back, and given its own behavior: a slower dimming curve, say. There is no state-transition table to maintain and no product of states to enumerate.

What the tool shows about a context is which relationships it contains, what enters and leaves it, and what it contributes to each output while active. Entry and exit conditions are ordinary occurrences, so `rise Held` and the like are available. Contexts with local history reset that history on entry unless the designer marks it persistent; the inspector states which.

The reduction of all this to plain declarations — an activation, an entry edge, gated state, a conditional in the output — is not something the designer manipulates. It is what the explanation view shows and what the reactive section justifies for the cases that were examined. Contexts with their own clocks are outside those cases, and the tool should say so rather than elaborate them silently.

One output, one final target

The lamp's light is a physical output. The designer connects `dimByTilt` (inside `Held`) to it. Then the warning behavior is built: a `Warm` condition on `Temperature`, and a pulsing brightness while it holds. The natural next action is to connect that pulse to the light as well.

The tool declines. The message is not *single-driver violation*. It reads, on the light output, along the lines of *this output already has a final driver, dimByTilt; combine the two brightness values before connecting the output*. The reason is a product reason: two behaviors that both set the same light have no defined result, and any rule the runtime picked — last wins, highest wins, the one drawn first — would be a decision made silently on the designer's behalf.

The repaired design has one more block:

```
dimByTilt (in Held) ----\
                        -> lampTarget -> light
warmPulse (while Warm) -/
```

`lampTarget` : Brightness is an ordinary Mapping Block whose definition states the rule. It might say that the pulse takes priority while `Warm`, or that the two are multiplied, or that the maximum is shown. The critical cutoff is handled the same way: `heaterTarget` takes the heater's demand from wherever it comes and forces it to zero above the critical temperature. The override is visible as a block on the canvas, upstream of the output, rather than as a policy attached to a context. When the designer later asks why the heater is off, the answer is on the canvas.

The status line on an output reads *output-complete* when exactly one final target drives it, and reports each undriven output the product requires until then.

Values that move at different speeds

The temperature sensor updates once a second. Tilt updates fifty times a second. The designer records this not as two rates but as two *domains*: `Temperature` moves with the environment, `Tilt` and `Held` move with the interaction. Domains are declared by name on the concept, and at this stage no rate is attached to either.

The first time a relationship reads across the boundary — `heaterTarget`, in the interaction domain, reads the critical-temperature condition from the ambient domain — the tool stops at that wire. Again the message is not a judgment name. It says that the two values update in different domains, and asks how the reader should see the source: as the last value the source produced, with a stated value to use before the source has ever reported, or by moving the reader into the source's domain and accepting a slower response. Both are legitimate designs. Choosing the first adds a visible transport on the wire, and the initial value it asks for — is the heater interlock engaged or released

before the first temperature reading? — is exactly the kind of decision that ought to be written down for a safety condition. The status line reports *clock-consistent* when every crossing has been settled this way.

Rates are attached at deployment. Changing a rate later changes what the product does, and the tool will re-run any timing obligations, but it changes nothing about which relationships are valid. Moving a concept from one domain to another does, and the tool treats that as an edit: it reopens every relationship that read the concept.

Choosing a board

Until this point the design has said nothing about hardware. The physical outputs are named — `light`, `heater` — and each has a kind: the light is a PWM channel, the heater is a switched load. The sensors are declared by kind too: the tilt estimate comes from an inertial sensor on the I2C bus, the temperature from an analog input.

Selecting an Arduino Nano in the deployment panel derives, from those kinds, what the design needs from a board — one PWM line, one digital output, two bus lines on the same I2C unit, one analog input — and attempts to place them. For the lamp this succeeds immediately, and the panel proposes an allocation: the light on a PWM pin, the heater on a digital pin, the sensor on the two I2C pins. The designer may accept it, or pin a requirement by hand — the light on D3, because that is where the board’s connector is — and let the rest be placed around it. A manual pin is a constraint on deployment, not a change to the design: nothing on the canvas moves.

The shape of a conflict is easier to see on a larger design, and the case examined in the development is the one to picture. A product with four motor channels, each an H-bridge needing a PWM line and a direction line, and an inertial sensor on I2C, fits the Nano; the allocation the solver produced is M1 -> D3/D0, M2 -> D5/D1, M3 -> D6/D2, M4 -> D9/D4, IMU -> A4/A5. A product needing seven independently dimmed channels does not fit, because the Nano has six PWM pins. The behavioral design is untouched by this; nothing on the canvas turns red. The deployment panel reports, on the seventh channel, that seven PWM lines are required and six are available, and names for each PWM pin the channel occupying it. A designer who then selects a larger board sees the same design placed without change. A designer who pins a PWM channel to D3 and later adds a rotary encoder that needs D3 as an interrupt line is told, on the encoder, that the only remaining interrupt pin is held by a manual assignment.

The distinction the panel is built to keep visible is that a design can be entirely valid — typed, temporally sound, every output driven — and still not fit the chosen board. The two are reported in different places and worded differently, so that neither is mistaken for the other.

What the workspace says at each stage

Figure 2 lists the states a design passes through. They are not a row of compiler badges. Each is meant to answer three questions about the selected object: what remains unresolved, what is already settled, and what can be done next.

State	What is settled	Where
declared	the relationship is named and typed; others may depend on it; no definition yet	<i>canvas</i>
defined	a formula, curve, examples, or component is attached	<i>inspector</i>
type-valid	the definition produces what the signature promises; declared properties hold	<i>inspector</i>
temporally valid	every held or delayed value has a first-tick value; no instantaneous loop	<i>canvas</i>
clock-consistent	every read across domains has been given a transport and an initial value	<i>canvas</i>
output-complete	every physical output the product requires has exactly one final target	<i>outputs</i>
hardware-feasible	the chosen board can carry every derived requirement	<i>deployment</i>

Figure 2: States a design passes through, as the workspace reports them. All but the last are properties of the design alone; the last is a property of the design together with a board, and is re-established whenever either changes.

A relationship that is declared but not defined is not flagged as wrong; it is shown as open, and the design around it is checked as far as it can be. A relationship that is defined but not type-valid is flagged at the definition, in the vocabulary of the block. A loop or a missing initial value is flagged at the wire. A missing final target is flagged at the output. A conflict on the board is flagged in the deployment panel, on the requirement that could not be placed.

Two states deserve their own visibility. *Hardware-feasible* is separate from everything above it, because it is the only state that depends on something other than the design, and it is re-established from scratch when the board changes or the design grows; six channels fit, a seventh may not, and nothing about the first six changes. And a property

discharged because a supplied component's author asserted it is not shown the way a property the tool computed is shown. A latency bound that rests on a declared worst-case execution time is weaker than one derived from the design, and the two must not look alike.

Looking underneath

Most of the time the compact surface is all a designer sees. When behavior is surprising — the lamp pulses once more than expected, or the heater takes a second longer than expected to cut off — an explanation view opens the selected object to show what it became. For `hold while not Held` it shows the `Held` value as a delayed cell and the condition that gates it. For a context it shows the activation, the entry edge, and the reset. For a cross-domain wire it shows the transport and the tick at which the source was last read, which is where the extra second lives. For an output it shows the final target and, once a board is chosen, the requirements derived from the output's kind and the pins they were given.

This view is the answer to the hidden-elaboration risk noted in the discussion. The surface stays simple because elaboration does real work; the explanation view exists so that the work is inspectable rather than mysterious. It is also where the kernel vocabulary is allowed to appear, for an engineer who receives the design and wants to know exactly what was generated.

The workflow as a whole

Read end to end, the intended sequence is this. The designer names the concepts the product is about and draws the relationships between them, leaving each undefined until there is something to say. Relationships are refined locally, by formula, curve, examples, or a supplied component, without the canvas changing shape. Time and history are stated as qualifiers on relationships. Contexts give a name and a boundary to the situations in which behaviors apply, and are nested rather than multiplied. Where two behaviors reach for one output, the tool asks for one final target, and the rule that combines them becomes a visible block. Where two quantities move at different speeds, the tool asks how one should see the other, and the answer becomes a visible transport with a stated initial value. Only then is a board chosen; the design's needs are derived from the kinds of its inputs and outputs, an allocation is proposed or a conflict is reported, manual pins are honoured, and switching boards re-solves the same needs without touching the design. Throughout, the workspace reports what is settled and what is open, and keeps a design that is valid distinct from a design that is deployable.

Whether designers work this way when given the chance, and whether it helps them, are the questions the evaluation plan is written to answer.

Part III — The Formal Kernel: Declarations, Identity and Dimensions

Method

The kernel was obtained by a method, and the method is part of the contribution. Each phase of the development took a family of candidate constructs from the earlier draft, formalized the smallest plausible version and its alternatives in Lean 4 without external libraries, and attacked each with the same questions. What does it reject that the others accept? What does it accept that it should not? Is it a special case of another? Which operations on a design are refinements under it, and which are edits? Constructs were promoted from the experiment modules to the kernel only after surviving, and the theorems about them were re-proved in the promoted form.

The formal development labels each conclusion with one of six strengths — formally proved, formally rejected by counterexample, reduction by proof, tested formulation redundant, engineering preference, not ruled out — and this document maps them onto the five labels of the front matter: the first three are **formally proved** (a counterexample and a reduction are theorems), the fourth and fifth are **informed by FV** or **design recommendation** according to whether a mechanized example bounded the choice, and the last is stated as open. Nothing in the development is a proven impossibility, and “minimal” always means minimal among the tested designs.

Scale and trust base

The development builds with Lean 4.33.1 with no sorry. The axioms used by every theorem are propositional extensionality and quotient soundness, the latter only through function extensionality and the choice-free rational quotient of Part VII; classical choice is absent, and each phase re-audited the whole development for it. As of Phase 13 the sources are 55 modules: 11 in Core, 12 in Behavior, 12 in Surface, 2 in Validation, and 18 experiment modules holding alternatives, counterexamples and executed examples. Every trace, assignment, unsatisfiability result and executed example reported here was obtained by running a proved-sound interpreter or solver inside the proof checker. Several theorems are recorded as trivial by definition — the typing half of client stability is a one-liner, and the well-formedness of a refinement target is unused because the invariant was moved into the definition — and they are reported as such rather than presented as content.

From Surface to Kernel: Architecture

The system has three layers, and the boundary between them is the main architectural result of the formal development.

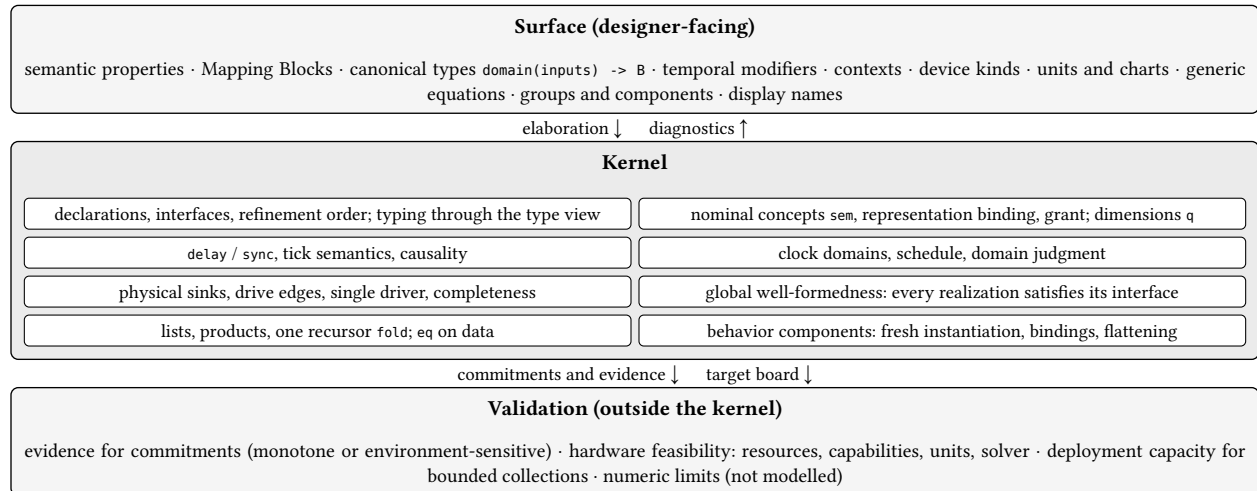


Figure 3: The three layers. Typing consults only the type view of declarations and the representation view of concepts; validation may consult commitments, evidence, and the target; the surface is derived forms over the kernel.

The **surface** is what the designer authors: semantic properties, Mapping Blocks, temporal modifiers, contexts, device bindings, units, generic equations, groups and components, and display names. Everything in it elaborates to kernel objects, and the elaboration is one-directional: the kernel never needs to recover surface structure.

The **kernel** is the formal object of Parts III–VIII. It consists of an environment of declarations, a typing judgment, a tick-indexed evaluation relation over one or several clock domains, a domain judgment, and a small number of global well-formedness conditions: every realization satisfies its interface, the instantaneous dependency graph is acyclic, every reference respects domains, and every physical output has at most one driver. The typing judgment reads only the *type view* of declarations — their expected types — and the *representation view* of concepts. It does not read realizations, commitments, evidence, clocks, or bindings.

The **validation layer** is everything that may depend on more than types. It discharges the commitments a declaration makes, and it decides whether a design fits a target board. Two kinds of evidence live here and are kept apart. Evidence that is meant to survive refinement — a monotonicity commitment discharged compositionally through the commitments of other declarations — must be stable under monotone extension of the environment, and the kernel imposes that condition. Evidence that is not meant to survive refinement — the existence of a pin assignment on a particular board — is re-established after every change and is never merged with the first kind.

Figure 3 shows the layers as they stand after Phase 12; the kernel band now also holds list and product data with one recursor (Part IV) and the behavior-component constructs (Part VI), and the validation band holds deployment capacity (Part V). What is notable about the arrangement is how much of the earlier draft of BDL is absent from the kernel band. Reactive types, event types, effect rows, action requests, policy transformations, and a five-phase tick with a resolve step were all part of the draft kernel. Each was removed because it either added no rejection the smaller kernel lacked, or made a design decision on the designer’s behalf that should have been visible in the design. The states of Figure 2 are the designer-facing face of the same structure: *declared* and *type-valid* are the typing judgment and satisfaction; *temporally valid* is causality; *clock-consistent* is the domain judgment; *output-complete* is the single-driver and completeness conditions; *hardware-feasible* is the validation layer’s solver. Each is decidable for finite designs.

Declarations, Interfaces, and Refinement

The foundational kernel object is a **design declaration**,

$$\text{DesignDecl} = \langle id : \text{DeclId}, \\ interface : \text{DeclInterface}, \\ realization : \text{Option Expr} \rangle,$$

where an interface is an expected type together with a set of public commitments:

$$\text{DeclInterface} = \langle expectedType : \text{Ty}, \\ commitments : \text{PropertyId}^* \rangle.$$

A design is an environment $\Delta : \text{DeclId} \rightarrow \text{Option DesignDecl}$. An unresolved declaration is one whose realization is none; nothing else distinguishes it. Display names are not part of the kernel.

Typing through the type view

Terms refer to declarations by identity, $\text{declRef } d$. The typing judgment $\Theta; \Delta; G; \Gamma \vdash e : \tau$ takes a concept environment Θ and a grant G , both introduced in the next section, and a declaration environment Δ which it consults through exactly one projection, the type view $\Delta^{\text{ty}}(d)$, the expected type of d if declared:

$$\frac{\Delta^{\text{ty}}(d) = \text{some } \tau}{\Theta; \Delta; G; \Gamma \vdash \text{declRef } d : \tau}.$$

This is the only rule that reads Δ . A client of d is therefore typed against d ’s interface and never against its body, whether or not a body exists. Inference is syntax-directed and decidable, and an inference function is proved sound, complete, and unique against the judgment.

Satisfaction, well-formedness, and refinement

A realization e **satisfies** an interface S when it has the expected type under the grant of that type and discharges every commitment:

$$\begin{aligned}
& \text{Satisfies } ev \ \Theta \ \Delta \ \Gamma \ e \ S := \\
& \quad \Theta; \Delta; \text{Grant.of}(S.ty); \Gamma \vdash e : S.ty \\
& \quad \wedge \forall p \in S.commitments. \ ev \ \Delta \ e \ p,
\end{aligned}$$

where $S.ty$ abbreviates the expected type. Here $ev : \text{DeclEnv} \rightarrow \text{Expr} \rightarrow \text{PropertyId} \rightarrow \text{Prop}$ is an abstract **evidence** relation supplied by the validation layer. It takes the environment as an argument because compositional discharge needs it: “ A is monotone because B is committed to be monotone” consults B ’s interface. A design is **globally well formed** when every stored declaration sits under its own identity and its realization, if any, satisfies its interface in that design.

Interfaces are ordered by monotone refinement: $S \sqsubseteq S'$ when the expected type is unchanged and the commitments of S are contained in those of S' . A declaration takes a refinement step in one of three ways. An unresolved declaration may have its interface refined; an unresolved declaration may be realized with a satisfying body; and a realized declaration may have its interface strengthened, provided the body is re-verified against the new interface. The reflexive-transitive closure of these steps is exactly the structural order together with well-formedness of the target, where the structural order DeclLeq requires the same identity, interface refinement, and a write-once realization, and $\text{EnvRefines } \Delta_1 \ \Delta_2$ lifts it pointwise while permitting new declarations.

Client stability

Can a declaration be refined or realized without editing its clients, and without invalidating what was previously established about them? The answer has two halves with deliberately different hypotheses.

The typing half needs only the structural order. If B is declared in Δ and $\text{DeclLeq } B \ B'$, then every typing judgment in Δ holds in $\Delta[B']$. This is a one-line consequence of typing references through the type view, and it should be read as such: its content is that the decision to let clients see interfaces and never bodies is *sufficient* for client stability. It is also necessary. Change B ’s expected type while keeping its identity, and every client breaks.

The commitment half needs more. If Δ is globally well formed, B takes a refinement step whose side conditions are checked in Δ , and the evidence relation is **monotone** — stable under EnvRefines — then $\Delta[B']$ is globally well formed, and the same holds for a whole lifecycle of B checked against the original environment. The monotonicity hypothesis was not part of the original design. It appeared when the theorem was attacked. An evidence relation that consults the *absence* of information — one that discharges a property because a dependency is still unresolved, say — is destroyed by a perfectly valid realization step, and the non-monotonicity of that evidence can be derived from the failure of preservation. Any discharge mechanism intended to survive refinement must therefore be positive in the environment.

Refinement versus edit

The preservation theorems cover refinement only. The table lists the operations examined and their classification; each row is witnessed by an example on a two-declaration design in which $A : \text{nat} \rightarrow \text{bool}$ is realized through an unresolved $B : \text{nat} \rightarrow \text{nat}$.

Operation on B	Kind	Effect on A
add a public commitment	refinement	typing and commitments preserved
realize	refinement	preserved; A now unfolds to a closed program
strengthen a realized interface	refinement, with re-verification	preserved
change the expected type, keep identity	edit	typing broken
drop a commitment	edit	typing silent; A ’s commitment broken
replace B by a new identity	edit	dangling reference
add a new B' beside B	edit	A still depends on the stale B ; never executable
detach or replace the realization	edit	evidence that consulted the body is void
assign or change a clock domain	edit	domain judgment on clients broken
retarget an output binding	edit	completeness or single-driver may break

Two rows deserve comment. Dropping a commitment changes no type, so the type checker is silent, yet A ’s own commitment was discharged through B ’s and is now unsupported. Commitments are therefore part of the interface

in the same load-bearing sense as the expected type. That is a stronger position than the earlier draft of the language took when it described properties as merely attaching to a name. Detaching a realization, which that draft permitted as an ordinary operation, is an edit for the same reason: clients’ typing is unaffected, but any evidence that consulted the body is void. The kernel does not forbid edits. It declines to promise anything about them, and the tool must reopen the validation of transitive dependents.

What persistent identity is

The earlier draft left open whether a persistent, referable identity for an unresolved relationship is a novel abstraction. It is not. Every theorem that mentions identity uses it only to make an update land on the slot a reference resolves to, which is what a name does in any environment-based semantics. The non-trivial content lies in the environment order, and in the two facts that clients depend on it only through the type view for typing and only through monotone evidence for commitments. This is the familiar interface/implementation separation of module signatures, of a parameter later given a definition, or of a metavariable context with write-once assignment and fixed types, together with a Kripke-style stability condition on evidence. What is slightly non-standard is that the interface carries a growable commitment set whose growth is a first-class operation on a declared-but-undefined name, and that the kernel imposes a stability condition on the validation layer. Neither is a new type-theoretic mechanism, and no such claim is made.

Zero-input relationships: the unit domain (Phase 12)

A relationship declared with no inputs — mapping `TempSensor : RoomTemp`, mapping `boost : Brightness` — is typed by the kernel at its output: `expectedType = B`. Production (ADR-0029, 3c6c8be) gives the same relationship a *canonical type* $() \rightarrow B$, with $()$ the empty product, so that every relationship has one type shape $\text{domain}(\text{inputs}) \rightarrow B$: $\text{domain}([]) = ()$, $\text{domain}([A]) = A$, $\text{domain}([A, B, \dots]) = A \times (B \times \dots)$. The surface spells mapping $f : B$ as shorthand for mapping $f : () \rightarrow B$; f , $f()$ and $f(())$ are one reference; $()$ is refused as a value anywhere else, as an output, and as a value form; hover and Explain show the canonical type and the sentence “its canonical domain is $()$, the empty product; the kernel encodes $() \rightarrow B$ as B ”. Production’s `bd1_ir::Ty` has a `Unit` constructor for this — a real IR type with the ordinary predicates — that never types a Core term, a representation or a runtime value. ISS-0014 asked whether the kernel and the canonical type agree by theorem.

Phase 12 (`Surface/UnitDomain.lean`) answers without adding a unit to the kernel. The canonical types live in an *interface layer* above it — `CTy` is the kernel’s `Ty` plus unit, interface arrows and domain products — and the kernel interface type is the value of a normalization function on them (**formally proved**):

$$\text{elim}(\text{canonical}(s)) = \text{encode}(s), \quad \text{encode}(\langle [], B \rangle) = B, \quad \text{encode}(\langle A_1, \dots, A_n; B \rangle) = A_1 \rightarrow \dots \rightarrow A_n \rightarrow B,$$

where `elim` performs unit elimination ($() \rightarrow B \mapsto B$) and currying ($(A \times B) \rightarrow C \mapsto A \rightarrow (B \rightarrow C)$), is total by a weight both steps decrease, and has no value on the bare unit — `elim () = none`: the unit is never the type of anything. The encodings are inverse over every signature whose output is not an arrow, hence over every concept signature (`decode_encode`, `canonicalOfKernel_encode`, `encode_injective`, `canonical_injective`), which is production’s inverse test as a theorem for all signatures. The categorical slogan $\text{Hom}(1, B) \cong B$ is thus, in this model, definitional at the kernel (the realization obligation of $() \rightarrow B$ is $\vdash e : B$ in the empty context, `zero_input_obligation`, by `Iff.rfl`), a theorem at the interface (the inverse pair), and a bijection denotationally (`homUnit : (1 → β) ≅ β`).

Why the unit is eliminated *before* the kernel is a theorem, not a preference. The literal alternative — realize $() \rightarrow B$ as a lambda over the unique argument and read it by application — is refused by typing: no term $\lambda x. \text{delay } i \text{ } e$ has any type, for any binder domain, because `delay` and `sync` are typed only in the empty context (`delay_not_under_binder`, `sync_not_under_binder`), while `delay init e : B` is a legal realization of $() \rightarrow B$ under the kernel encoding (`zero_input_memory`). A kernel unit binder would forbid memory in every zero-input declaration; a counter `boost := delay 0 (boost + 1)` would become untypable. `lams_typed` states the general case: a formula body checked in the context of its inputs is a realization of the encoded type under n binders, and for no inputs there is no binder.

The three spellings are one term (`refForms_agree`), so they share typing, evaluation and the clock judgment by reflexivity (`HasType.refForms`, `Clocked.refForms`). Two theorems say what the unique argument could have carried, and that it carries nothing: a reference’s value at a tick is independent of the local environment — a realized declaration evaluates in the empty environment, an unresolved one is read from the input stream

(Ev.declRef_env_irrelevant, MEv.declRef_env_irrelevant) — and two readings in one tick agree under any two environments (same_tick_same_value). There is no per-reference call to repeat; a zero-input declaration is evaluated as a declaration, once per activation, and the reader observes that value. No unit clock, no extra activation, no evaluation step.

The source role is a realization state, not a type shape. $() \rightarrow A$ describes the shape of a signature; a *source* is a declaration with no realization, whose value at every tick is provided by the environment: $\text{Source } \Delta d := \text{realizationOf } d = \text{none}$, and source_value gives that value as $I(d, t)$ in every domain and under every environment — indexed by the declaration and the tick alone, which is the formal sense in which the unit argument carries no temporal or environmental information. A resolved $() \rightarrow A$ never consults the input stream ($\text{resolved_not_source}$); it is not a sensor, and every mathematical $() \rightarrow A$ is not one either. Production’s *simulation input* — Studio offers a declaration as an input when it is unresolved and unit-domain — is $\text{SimulationInput} := \text{Source } \wedge \text{UnitDomain}$, a narrowing the kernel does not need (its input stream provides a value for every unresolved declaration, arrow-typed ones included) and that is recorded as surface policy over the same semantics: on a unit-domain source the two agree ($\text{SimulationInput.value}$). No source kind exists in either model; production’s $\text{Signature}::\text{is_unit_domain}$ is the one predicate, and the formal $\text{UnitDomain } \Delta d$ is the same predicate on the kernel type.

The two production consequences of the unit domain are corollaries of existing rules (**formally proved**). Only a value can be remembered or transported: a well-typed *sync* or *delay* of a reference forces the referenced type to be data, hence not an arrow, hence unit-domain ($\text{transport_needs_unit_domain}$, $\text{delay_needs_unit_domain}$) — production’s diagnostic *f has inputs, so its value cannot be carried across timing domains* is this theorem’s message. And a driver of a sink is a value of the accepted type, so when the sink accepts a concept the driver is unit-domain ($\text{driver_is_unit_domain}$, from DriveWF). Part VIII takes up the dual form $A \rightarrow ()$.

Executed (**executable example**, `Experiments/UnitDomainExamples.lean`): the lamp’s signatures round-trip through both encodings and `elim` computes them; $\text{boost} := \text{delay } 0 \ (\text{boost} + 1)$ read as boost , $\text{boost}()$ and $\text{boost}()$ gives 0, 1, 3 at ticks 0, 1, 3, and the same under a non-empty local environment; the unresolved TempSensor reads the input while boost ignores it entirely; infer accepts $\text{delay } \dots : 00$ in the empty context and refuses a binder around it; sync of boost and of TempSensor is typed and sync of the one-input dimByTilt is refused.

Verdicts (**informed by FV**): unit in the canonical interface notation — keep, above the kernel; a kernel unit type, a unit runtime value, a unit term — remove; a source semantic kind — remove, derive from the realization state; the source surface role — keep in the surface; $A \rightarrow ()$ as a physical sink — remove (Part VIII); the output/drive boundary — keep. Production’s correspondence row for ADR-0029 can move from *engineering choice* to *formally proved* at the interface and *transcribed* at the kernel encoding, and ISS-0014 can close.

Semantic Identity and Representation

Nominal concepts

Suppose concepts were represented only by their representation types, so that Tilt and MotorAngle are both numbers. Then the wire $\text{motorTarget} := \text{tiltSensor}$ is well typed and the design is globally well formed, because nothing in the model records the distinction. This baseline was built, the wire was accepted, and three ways of recording the distinction were then tried against it.

The one that survived is a single nominal type constructor over an internal identity:

$$\text{SemanticId}, \quad \text{Ty} \ni \text{sem } s.$$

Two distinct identities are distinct types regardless of representation, so the invalid wire is rejected by the ordinary rules of the simply typed calculus with no additional judgment. An explicit relationship between concepts, $\text{tiltToMotor} : \text{Tilt} \rightarrow \text{MotorAngle}$, is an ordinary declaration of arrow type — a design relationship that is itself signature-first and may remain unresolved — and it appears in the term wherever a crossing occurs. It is not a cast, coercion, or conversion; the kernel has no such mechanism.

Semantic identity is independent of declaration identity, of display name, of dimension, and of hardware, and each independence was tested rather than assumed. A rename preserves identity, whereas a model in which the name *is* the identity makes renaming destructive. Treating a concept as an ordinary declaration admits two category errors:

the concept becomes usable as a value, and it can be realized by a number. Keeping identity out of the type as interface metadata, checked by a direct-wire rule, is evaded by η -expansion, since $(\lambda x. x) \text{ tilt}$ has the same flow with no direct wire. A compositional role judgment strong enough to close that gap has the rule shapes of typing over types-with-sem, and the one such formulation examined duplicated nominal typing without benefit. Flow-sensitive or relational semantic analyses were not formalized and are not ruled out; nominal sem is the smallest mechanism among the designs that were tried, not the only one possible.

Erasing all semantic identities is sound — a well-typed semantic term is well typed at its representation — and the baseline is exactly what erasure leaves. Generated code is thus ordinary code; the semantic layer has no runtime residue.

Representation binding and the grant

Nominal identity alone leaves semantic values opaque. Without a way to observe a representation and construct a value, no mapping can be realized by a formula; under nominal typing alone a semantic value can only originate from a declaration of semantic type. That is the right state before representation is added. The question is how to add it without destroying what identity just bought.

The obvious form — global $\text{rep}_s : \text{sem } s \rightarrow R$ and $\text{mk}_s : R \rightarrow \text{sem } s$ available to every term — destroys it immediately. $\lambda x. \text{mk}_{\text{Motor}}(\text{rep}_{\text{Tilt}} x)$ is a well-typed $\text{Tilt} \rightarrow \text{MotorAngle}$ in the empty environment, with no declaration and no mapping; a motor angle can be manufactured from a literal; and the crossing can hide inside a body whose signature mentions no motor. Observation alone, with no construction, is safe but cannot realize a mapping. The surviving model separates the two:

- a **concept environment** $\Theta : \text{SemanticId} \rightarrow \text{Option Ty}$ binds each concept, write-once, to a representation that mentions no semantic type and contains no function type;
- $\text{rep } e$ is typed at R whenever $e : \text{sem } s$ and $\Theta s = \text{some } R$, everywhere;
- $\text{mk } s e$ is typed at $\text{sem } s$ whenever $e : R$, $\Theta s = \text{some } R$, and the grant permits s .

$$\frac{\Theta s = \text{some } R \quad \Theta; \Delta; G; \Gamma \vdash e : \text{sem } s}{\Theta; \Delta; G; \Gamma \vdash \text{rep } e : R}$$

$$\frac{G s \quad \Theta s = \text{some } R \quad \Theta; \Delta; G; \Gamma \vdash e : R}{\Theta; \Delta; G; \Gamma \vdash \text{mk } s e : \text{sem } s}$$

The grant G is a predicate on concepts. Client code is typed under the empty grant. The realization of a declaration is typed under $\text{Grant.of}(\tau)$, the concepts in result position of its own signature τ . A value of MotorAngle can therefore be constructed only inside a declaration that announces MotorAngle in its signature, which is exactly where a reader of the design would look for it. A well-typed term constructs s only where granted s ; the hidden crossing above is rejected under the grant of an unrelated declaration and becomes legal, and visible, once tiltToMotor is declared; binding an unbound concept is monotone for typing, satisfaction, and global well-formedness; and rebinding a concept to a different representation is an edit that breaks existing realizations.

Two constraints on the representation were not anticipated. Representation types must be free of semantic types, because if Tilt may be represented by MotorAngle then rep itself is a hidden mapping under every policy, including observation-only. And they must be data types, a requirement that arrived later from the reactive semantics: a semantic value may be delayed, and a function-typed representation would carry a closure across ticks.

The grant is a known shape — a capability attached to a definition site, or equivalently the private constructor of an abstract type exported only to the module that declares it. Its contribution here is where the capability comes from: the signature the designer already wrote, so no annotation is added. It is not presented as a capability calculus. One consequence should be stated plainly. After all bodies are inlined into one executable program, that program is checked under the universal grant, because each construction was authorized at its own declaration. Semantic isolation is a property of the design graph and survives inlining as provenance, not as a type property of the executable.

Physical dimensions

Physical quantities have type $q\ d$ for a dimension d , an exponent vector over a small set of base dimensions. There is no dimension-specific typing rule; the algebra lives entirely in the types of registered primitive operators,

$$\begin{aligned} \text{add}_d &: q\ d \rightarrow q\ d \rightarrow q\ d, \\ \text{mul}_{d_1 d_2} &: q\ d_1 \rightarrow q\ d_2 \rightarrow q\ (d_1 + d_2), \\ \text{div}_{d_1 d_2} &: q\ d_1 \rightarrow q\ d_2 \rightarrow q\ (d_1 - d_2), \end{aligned}$$

and an application is checked by ordinary function application. Erasing every dimension to the zero vector is sound and accepts `length + time`, so the untyped numeric baseline is the erasure of dimensional typing in the same sense that the representation baseline is the erasure of nominal typing. Dimensions might instead have been validation metadata; the argument against is that multiplication and division *produce* dimensions, so any checker recomputes the same inference, but that family was argued against rather than excluded.

Dimension and semantic identity are orthogonal. `Tilt` and `MotorAngle` both bound to $q\ \text{Angle}$ remain distinct types. A mapping realized by the dimensioned formula `λx. mk bright (rep x · gain)` with `gain : q (0 - Angle)` is typed; a dimension error inside the formula is caught by the same typing; and the formula cannot manufacture a `MotorAngle` despite the shared dimension. The association between a concept and its dimension lives in Θ , not in the identity and not in the type constructor. The earlier draft’s two-index `Sem[n, d]` becomes `sem s` together with $\Theta\ s = \text{some } (q\ d)$.

Units are surface. A literal `n u` elaborates to a dimensioned literal scaled by the unit’s factor; changing the unit changes the value, never the type, and mixed-unit addition works after elaboration. Expressing a quantity in a unit — its *coordinate* — and building a quantity from a coordinate are the same arithmetic against a unit constant: `inUnit(q, u)` is q divided by the unit’s scale and has dimension zero, `withUnit(x, u)` is x times the scale and has the unit’s dimension, and a conversion between two units is their composition; each is elaborated, none is a kernel construct, and a unit choice never reaches a type (`1 m` and `100 cm` are equal values of one type). The unit laws — round trips, derived conversion, dimension safety — are proved over an abstract scalar domain and instantiated by a symbolic group in which π is a generator, so that a degree is exactly $\pi/180$ radian; the executable kernel truncates to naturals and production approximates in floating point. A preferred display unit is presentation: it changes the number shown and no judgment of the design. Affine scales such as degrees Celsius are not linear (`0 °C` is `273.15 K`), yet their literals and coordinates elaborate exactly by adding an offset. Unit coordinates erase chart identity while preserving affine coordinate change: the conversion between two charts is an affine map, conversions compose and invert — compatible charts are isomorphic coordinate systems — and differences inherit the linear part of that transformation, so a difference of ten degrees Celsius is eighteen degrees Fahrenheit from any base point, while a conversion with a non-zero offset is not an additive homomorphism. The same abstraction covers sensor calibration and encoder offsets. These laws are proved over an abstract field and instantiated exactly at rationals; production floating point is held to a toleranced version of them. Whether a sum of two absolute temperatures should be permitted is a separate, optional validation question that the dimension does not decide and conversion does not need.

Part IV — The Data and Equation Language

The kernel of Part III computes with booleans, counts, quantities, nominal concepts and optional values, and abstracts with lambdas over them. That is enough to state the theorems about identity, refinement, time and outputs, and it is not enough to write the equations designers actually write: a brightness clamped to a range, a mode tested against a finite set of modes, every sensor below a threshold, a pair of readings, a calibration mapped over a collection. Phases 9b and 9c asked for the smallest typed data/function basis that supports those equations without turning BDL into a general functional language, and Phase 11 asked whether the natural surface forms for them can be added with no semantic change at all. This Part records the whole investigation: the candidates, the counterexamples, the kernel additions, the definitional library, the capability audit that reversed one of Phase 9b’s own decisions, and the surface forms.

The question and the hypothesis

The design direction was stated before the experiments: a very small core, parametric polymorphism, ordinary structured data, a rich definitional standard library, and intent-oriented abstractions at the UI. The hypothesis to test was a core of roughly `bool`, `nat/q d`, `sem s`, `A → B`, `A × B`, `opt A`, `list A`, plus rank-1 polymorphism — with the instruction not to assume any proposed feature belongs in the kernel.

The required cases were fixed in advance and every one is executed in `BDL/Experiments/EquationExamples.lean`: (A) clamp a brightness; (B) a mode in a finite set of modes; (C) every sensor below a temperature threshold; (D) any fault above a severity threshold; (E) a pair of temperature and humidity; (F) a calibration mapped over sensor values; (G) two sampled collections zipped; (H) an optional fallback; (I) dimension-preserving `min`; (J) semantic-type-preserving generic functions; (K) range membership; (L) a piecewise rule combining a range, a collection predicate and a boolean; and (M), added when production’s `enum LampMode { Off, Automatic, Manual(Brightness) }` was inspected, an enumeration with a payload.

What entered the kernel, and why each item could not be derived

Four additions were made to the kernel; each was tested against a derivation first.

Products. `Ty.prod a b`, `Value.pair`, and the operators `pair : a → b → a × b`, `fst`, `snd`. A pair is data exactly when both components are `(Ty.prod_data)`. The derivation that was tried and rejected is the Church encoding. It fails twice. First, a Church pair is an arrow, and arrows are not data: nothing of function type can be delayed or transported (`arrow_not_delayable`, **formally proved** by inversion of the `delay` and `sync` typing rules, which require `τ.Data`). Paired *state* — a delayed reading with its timestamp — therefore needs a data product. Second, a Church pair used as a first-class value needs rank-2 types: in the toy System F of `PolyAlternatives.lean`, the type of `fst` on Church pairs has rank 2 (`church_fst_rank`), and in the prenex fragment a pair instantiated at one result type serves only one projection (`church_pair_prenex_one_projection`). Products are value composition only. They are never a component interface, an output bundle or a system boundary — Phase 8b’s Counterexample 5 shows what a tuple-returning declaration does to the dependency graph.

The list recursor. `Expr.fold f z l`, with `fold f z [x1, ..., xn] = f x1 (... (f xn z))`, is a *term former*, not a registered operator. The kernel has no recursion, deliberately; a total language needs an eliminator for its inductive data, and this is the one construct in BDL that applies a function value in the course of evaluation. Registered operators still never apply closures. Its evaluation rule unrolls syntactically through the environment:

$$\frac{\Delta; I; t; \rho \vdash f \Downarrow v_f \quad z \Downarrow v_z \quad l \Downarrow []}{\text{fold } f \ z \ l \Downarrow v_z} \quad \frac{f \Downarrow v_f \quad z \Downarrow v_z \quad l \Downarrow x :: xs \quad [xs, v_z, v_f] \vdash \text{fold } \#2 \ \#1 \ \#0 \Downarrow r \quad [r, x, v_f] \vdash \#2 \ \#1 \ \#0 \Downarrow v}{\text{fold } f \ z \ l \Downarrow v}$$

The recursive premise evaluates the syntactic term `fold (var 2) (var 1) (var 0)` in an environment holding the three values; this keeps `Ev` an ordinary inductive relation with no mutual recursion, so every earlier proof by induction on `Ev` extends by one case. Totality is a separate lemma by induction on the list (`fold_total`, `mfold_total`), and the fundamental theorem’s `fold` case uses it. Every collection operation — `map`, `filter`, `any`, `all`, `contains`, `append`, `sum`, `zip`, and, through `toList`, the option eliminators — is a definition over it (below). The alternative of one primitive per operation was rejected because a primitive cannot apply a closure and each would need its own evaluation rule; the alternative of bounded unrolling was rejected because lists (the Phase-9a buffer) are unbounded.

Equality at every data type. $\text{eq } \tau \text{ (} h : \tau.\text{Data} \text{)}$: structural equality on data values — booleans, numbers, none/some, pairs and lists componentwise, semantic values by concept and representation — with the proof of $\tau.\text{Data}$ carried *in the syntax*. This is the kernel’s only capability evidence: an equality on a function type is unwritable rather than ill-typed, which keeps the `prim` typing rule unconditional. Before Phase 9b, equality existed only at quantities, and production encoded boolean equality as $(a \wedge b) \vee (\neg a \wedge \neg b)$. On first-order values structural equality is equality (`Value.beq_iff`, **formally proved** by a mutual induction over the nested value type).

Two first-order operators. `drop  $\tau$` (the dual of take) and `toList  $\tau : \text{opt } \tau \rightarrow \text{list } \tau$` . The second is the one that matters: without it an optional value has no eliminator that does not need a default — `getD` needs a fallback of the payload type, which a generic `mapOpt` cannot produce — and with it `fold` over a list of length at most one is the option eliminator (`optElimF`, `mapOptF`). `drop` lets `zip` be derived.

Every earlier theorem — determinism, totality in one and many domains, provenance, unfolding, the Phase-8 preservation results, the Phase-9a buffer — was re-established without change of statement. The logical relation gained the product clause; the fundamental theorem gained the `fold` case; the Wiring fragment gained a variant with variables for the recursor’s environment-passing sub-derivations (`Ev.noCloV`, `Ev.foldCons_move`).

Polymorphism: rank-1, by families, no kernel type variable

The five models compared, with the verdicts:

model	verdict	evidence
A — monomorphic STLC kernel	kept, unchanged	typing rules unchanged; <code>HasType.unique</code>
B — per-type duplication	what the kernel <i>sees</i> after elaboration	<code>instances_are_monomorphic</code> : three uses of <code>min</code> are three kernel terms
C — rank-1 parametric	adopted, as definitional families	every library entry is <code>Ty $\rightarrow$ Expr</code> (or <code>Dim $\rightarrow$ Expr</code>); <code>*_typed</code> proves its scheme at every instance
D — System F ($\wedge, [\tau], \vee$)	rejected	the toy <code>FTy</code> with rank measures what it would add; its prenex fragment <i>is</i> instantiation of families
E — higher rank	rejected	every candidate use has rank ≥ 2 (<code>applyBoth_rank</code> , <code>existential_rank</code>) and a rank-1 replacement (<code>applyBoth_replacement</code>)

Why C needs no kernel support: a use site always has *closed* argument types. Every declaration’s expected type is frozen and closed since Phase 1, and `infer` is bottom-up, so finding the instance of a scheme is one-way *matching* of the scheme’s pattern against closed types — decidable, returning the unique substitution on the pattern’s variables (`matchTy_sound`, `matchTy_complete`, **formally proved** in `Surface/Poly.lean`). There is no unification of two open types, no let-generalization inside expressions, and no principal-type search: those problems arise when a definition’s type is inferred from its body, and BDL definitions carry their signature (mappings do; a helper `fn` would). Dimension polymorphism (`sum : list (q d)  $\rightarrow$  q d`, `min at q d`) uses the same mechanism with dimension pattern variables; no kind system, no `Type + Dim` universe, because the dimension algebra already lives in the primitive table.

The scheme model: `PTy` is the kernel’s type formers over type variables and dimension patterns (`PDim.const d | PDim.dvar n`); `Subst` maps both kinds of variable; `Scheme = (pattern, caps)` where `caps : List (Nat  $\times$  Cap)` names, per variable, what it needs; `Scheme.instantiate 0  $\theta$   $\tau$` matches then checks the capabilities and is sound (`Scheme.instantiate_sound`). The two failure points have designer-level explanations — *no instance* (“expected a collection”) and *capability failed* (“cannot compare functions”; “Mode values can be compared for equality, but they have no default order”) — and a nominal mismatch is reported by the kernel’s unique typing as “Brightness and Opacity are different concepts”, never as a unification residue, because there is no unification.

Capabilities: the audit that reverted a decision

Phase 9b’s first form generalized both `eq` and `lt` to every data type through a structural order on values: booleans `false < true`, numbers, `none < some`, pairs and lists lexicographically, concepts by representation. It was formally consistent — `Red_prim` held, determinism held — and the closed capability vocabulary collapsed to `{Data}`. Phase 9c audited it and rejected it.

The audit asked, for each type, whether `==` and `<` have a domain-natural meaning for a behavior designer:

expression	meaning	verdict
<code>temperature1 == temperature2, <</code>	magnitude comparison of one quantity	Eq, Ord (q d; q Length < q Time still rejected)
<code>brightness1 < brightness2</code>	the concept is a magnitude the designer declared ordered	Ord <i>by declaration</i> , through the representation
<code>model == mode2</code>	same mode	Eq
<code>model < mode2</code>	none — any order would come from a code, a constructor tag or a SemanticId	rejected
<code>pair1 == pair2</code>	same reading	Eq
<code>pair1 < pair2</code>	lexicographic order is a mathematical convenience with no design meaning	rejected
<code>list1 == list2</code>	same collection in the same order	Eq
<code>list1 < list2</code>	none	rejected
<code>optional1 == optional2</code>	both absent, or both present and equal	Eq
<code>None < Some x</code>	a constructor-tag artifact	rejected

So `Data ≈ Eq` holds (extensionally on this type grammar: `Cap.eq_iff_data`), but `Eq ≈ Ord`. An implementation may need a total order on values for maps, canonical forms, serialization and tests; that is toolchain-internal and is not `<` in BDL. The kernel's structural order (`Value.bl`) was deleted and `lt` restored to `lt (d : Dim)` on quantities — the Phase-4 form — which made Phase 9b *smaller*. The surface vocabulary became `Cap = data | eq | ord (Poly.Cap)`: `data` is what delay/sync need and what the eq proof field checks; `eq` coincides with `data` today and is kept as a separate name because it answers a different question and is what diagnostics say; `ord` is a *surface* capability — a quantity, or a concept the designer declared ordered (`OrdDecl`) and represented by a quantity (`Ty.ordB`) — with no kernel counterpart, because an ordered concept compares as `lt d` on `rep`, a term the kernel already admits. Enumerations follow the same rule: equality is natural, declaration order is never silently behavioral order.

Ordering in the library is evidence-indexed: `Ordered τ` is `q d`, or `sem s d` for a concept declared ordered with representation `q d` (well-formed against Θ : `Ordered.WF`); `ltAt o a b` is `lt d a b` on a quantity and `lt d (rep a) (rep b)` on a concept, so `min`, `max`, `clamp`, `inRange`, `inInterval` return one of their arguments with identity intact. The comparator escape hatch `minBy/maxBy` needs no declaration and loses nothing: `minBy_recovers_min` (**formally proved**) shows the comparator $\lambda a\ b. a < b$ makes `minBy` compute exactly `min`. Negative examples on realistic concepts: `model == mode2`, `pair == pair`, `list == list`, `opt == opt` accepted; `mode < mode`, `pair < pair`, `list < list`, `None < Some`, `bool < bool` rejected at the surface (`lt_rejected`, `min_mode_rejected`) and unwritable in the kernel (`lt_only_on_quantities`).

Production adopted this as ADR-0025 and refined it as ADR-0026: two values of one concept compare as that concept (`==` always, `<` and the ordered equations only while the concept is declared ordered); two values of different concepts never compare, whatever their representations; a concept beside a *plain value of its representation* — `tilt < 10 deg` — is observed and compared as that representation, because the designer wrote a number. The remaining asymmetry is visible and intended: `min(o1, o2)` is refused while `min(o1, 0.5)` is not, and `Explain` shows the observation.

The definitional library

`Surface/Stdlib.lean` is the reference for production's bdl-equations. Each entry is a closed de Bruijn term indexed by types: `idF τ`, `constF τ σ`, `swapF a b`, `minF o`, `maxF o`, `clampF o`, `inRangeF o`, `inIntervalF o`, `minByF τ`, `maxByF τ`, `foldrF τ σ`, `anyF τ`, `allF τ`, `containsF τ h`, `mapF τ σ`, `filterF τ`, `appendF τ`, `sumF d`, `optElimF τ σ`, `mapOptF τ σ`, `getOrElseF τ`, `zipF a b`; `listLit`, `oneOfE` (a finite-set literal is `contains` over a list literal); records as right-nested pairs with positional projections (`recTy`, `recE`, `projE`). `zip` is derived through `fold`, `take 1`, `drop 1` and `reverse`, folding over the reversed first list with the remaining second list and the accumulated pairs in a pair — not pretty, and the point is that it is derivable.

Every entry is a **combinator**: variables, literals, lambdas, applications, registered operators, the recursor and, since 9c, `rep` — no reference, no state, no transport, no `mk`. For combinators, proved once: typing is independent of the design and the grant and reads Θ only through write-once representation bindings (`HasType.comb_irrelevant`); the value is the same in every design at every tick under every input (`lib_eval_context_free`, from the kernel lemma `Ev.pure`: a pure term in a pure environment is context-free); the term is clocked in every domain (`lib_clocked`);

nothing is constructed (`Comb.noConstruct`); and the four together as the inlining statement `lib_expansion`. This is what lets production inline an equation at each use without creating a declaration — a library entry as a declaration would be monomorphic and would enter the dependency graph (D-94).

The evaluation specifications connect each entry to the mathematical function: `any_spec` (`List.any`), `all_spec`, `contains_spec` (`List.any (beq x)`), `map_spec`, `filter_spec`, `min_spec`, `max_spec`, `clamp_spec`, `inRange_spec` (by the compared magnitudes `Ordered.key`), each **formally proved** through a general `fold_spec`: the recursor computes `List.foldr g` whenever the step closure implements `g` on the reachable accumulators. From these, finite quantification is a fold: `forall_in_list` ($\forall x \in xs, P \times \text{iff all } xs \text{ } P \text{ evaluates to true}$) and `exists_in_list`; a finite-set literal means membership (`oneOf_mem`, via `beq_iff`), and duplicates do not change the answer (`oneOf_dup_irrelevant`), so no uniqueness convention and no `Set` type. An interval is a pair with a convention and membership is a function of the pair. A predicate is an ordinary $\alpha \rightarrow \text{bool}$.

Nominality survives all of it: `generic_preserves_identity` — *any* family typed at $\alpha \rightarrow \alpha \rightarrow \alpha$, instantiated at concept `s`, rejects an argument of concept `s'  $\neq$  s`, the representations never consulted; `generic_preserves_dimension` for `q d vs q d'`; `pair_projections_keep_concepts`; `map_keeps_concepts`; `eq_across_concepts_rejected`. Executed: `min` at `Brightness` typed and `Opacity` rejected though both are `q 0` underneath (`exJ`), and the same for lengths and times (`exI`).

Enumerations, records, sets: what was encoded and what was deferred

Production's textual syntax declares enumerations with payloads and matches on them. Phase 9b encoded `LampMode` as a tag paired with an optional payload, with `match` as conditionals on the tag (`exM`, executed), and deferred a kernel `sum` type: nothing tested needs more than the encoding gives, and the cost of a kernel `sum` would be one more eliminator term former, exactly like `fold`. Production keeps user enums open (ISS-0005). Records are positional nested pairs; labels resolve to positions at elaboration; row polymorphism was rejected because no case needs a function generic over record shapes. Sets are list literals with `contains`; set algebra is list definitions when needed; nothing tested observes a canonical form.

The expressiveness ceiling

Total, first-order-data computation over booleans, quantities, nominal concepts, options, lists and pairs, with higher-order functions and one list recursor; generic definitions instantiated at closed types; no general recursion, no type abstraction in terms, no sums yet (encoded), no unbounded quantification. This is a design recommendation backed by the executed cases and the proved library, minimal among the tested candidates; it is not a minimality theorem. The seven questions the phase was asked have the answers: rank-1 by families constrained by `{Data, Eq, Ord}` is the weakest useful polymorphism; products belong in the data core; no `Set` type; finite $\forall/\exists$ reduce to folds, proved; no existentials (Phase 8a components hide by fresh instantiation); no user typeclasses; and the ceiling above.

The natural expression surface (Phase 11)

Designers should be able to write

```
all reading in readings:
  reading in 10 deg .. 45 deg
```

while the semantics stays exactly the verified Phase-9 machinery. Phase 11 tested that binder syntax and closed ranges are *conservative* surface abstractions and proved it for a six-form fragment (`Surface/Natural.lean`). The surface model `NatExpr` has an embedded closed core term, a named binder local, application, the four binders `all/any/map/filter` `x in xs: body` with the element type chosen by local inference, a range `x in lo .. hi` with its `Ordered` evidence, and a coalesce `x ?? d`. Elaboration `desugar` is one-way, under a binder stack:

```
all x in xs: p      ↪ app2 (allF τ) (lam τ p') xs'      (any, map, filter alike)
x in lo .. hi      ↪ app3 (inRangeF o) x' lo' hi'
x ?? d             ↪ app2 (getOrElseF τ) x' d'
```

A binder local is the kernel's lambda parameter: `desugar` resolves a name to its de Bruijn index — nearest binder first — and adds nothing. **Formally proved**: nearest-binder scoping, shadowing, an unbound name is an error rather than a free variable, ambient references are untouched (`desugar_local_nearest`, `desugar_shadow`, `desugar_unbound`, `desugar_core`); alpha-equivalence — renaming a binder and its occurrences to a name fresh in the expression and the stack leaves the elaboration unchanged (`desugar_rename`, `alpha`), which is what a Formula Composer

generating fresh locals needs; no `mk` is introduced (`desugar_constructs`); typing of each form with the local *forced* to the collection's element type by inversion (`binder_local_type`) and range bounds forced to the value's nominal type (`range_bounds_forced`); evaluation exactly the library's (`binder_all_eval`, ... from `all_spec`, `any_spec`, `map_spec`, `filter_spec`; `natural_forall`, `natural_exists`; `range_eval` = $lo \leq x \wedge x \leq hi$); and clocks of the elaborated term are the operands' (`binder_clock`, `range_clock`).

Executed: the call form and the natural form elaborate to literally the same term; `angle` in `10 deg .. 45 deg` after unit elaboration, with the boundary values checked; nested `all row in rows: any v in row: v > 0`; shadowing where the inner `x` ranges over `ys`; alpha renaming under shadowing; `Tilt` ranges with `Tilt` bounds accepted, `q Angle` bounds rejected, and accepted through `rep`; a `MotorAngle` predicate rejected on a `Tilt` local; the required negatives (`all x in 5: true`, `filter x in xs: 5`, `angle in 2 s .. 3 s`, an unordered concept in a range, a binder variable outside its body). Verdicts: binders, `BinderKind`, the range node and `??` are surface desugaring; an interval type, general comprehension (generators, `yield`, `where`) and a general quantifier are removed — nested binders cover every required case, and the forms are finite list equations.

At production revision `flce82c` the natural binder syntax was not implemented; at `3c6c8be` it is (P11, protocol 0.13, **production-tested**): `bdl-elab::formula::binder` lowers the three forms once to the equation library, the parser keeps them as their own nodes and the formatter keeps the spelling authored, binder locals are lexically scoped to the body, and the tests `natural_forms_lower_to_the_same_core_as_the_call_forms`, `binder_locals_are_elements_scoped_to_the_body` and `natural_form_mistakes_are_named_in_their_own_words` discharge the Phase-11 claims by differential elaboration; the Composer draws binders and ranges with local chips (`ComposeAction.binder`, `range`). The production recommendations recorded with the phase, which the implementation followed: `all`, `any`, `map`, `filter` and `in` as contextual keywords only in the binder head, so existing names keep parsing; the local visible in the body only; the natural form kept as authored and never reconstructed from `Core`; `..` binding tighter than `in` and looser than arithmetic, legal only as the right operand of `in`; local inference exactly `binder_local_type`; the Composer representing a binder as a node with a fresh local backed by `desugar_rename`; diagnostics in concept language (“5 is not a collection”, “Mode values have no order, so `in lo .. hi` does not apply”).

Part V — Reactive and Temporal Semantics

A Minimal Reactive Semantics

One primitive

The reactive kernel adds one expression form and no types:

$$\text{delay } \text{init } e.$$

Its meaning is given by a tick-indexed big-step evaluation relation $\text{Ev } \Delta \ I \ t \ \rho \ e \ v$: the value of e at tick t under a local environment ρ , where $I : \text{DeclId} \rightarrow \mathbb{N} \rightarrow \text{Value}$ supplies the inputs. Every declaration is a stream by interpretation. An unresolved declaration is an input and takes its value from I ; a realized declaration is evaluated from its body at the current tick; $\text{delay } \text{init } e$ evaluates e at tick t when read at tick $t + 1$, and init at tick 0:

$$\frac{\Delta^{\text{real}}(d) = \text{none}}{\text{Ev } \Delta \ I \ t \ \rho \ (\text{declRef } d) \ (I \ d \ t)}$$

$$\frac{\text{Ev } \Delta \ I \ 0 \ \rho \ \text{init } v}{\text{Ev } \Delta \ I \ 0 \ \rho \ (\text{delay } \text{init } e) \ v} \quad \frac{\text{Ev } \Delta \ I \ t \ \rho \ e \ v}{\text{Ev } \Delta \ I \ (t + 1) \ \rho \ (\text{delay } \text{init } e) \ v}.$$

There is no signal type in Ty. Under this semantics a signal type would be inhabited by exactly the terms of the underlying type and would reject nothing. The information a reactive type would carry in a multi-domain setting is *which clock*, and the next section places that information in a judgment rather than a type. There is likewise no event type. Within one domain an input delivers at most one value per tick by construction, so an occurrence is a stream of optional type, and the streams of type $\text{opt } \tau$ are exactly the streams of multiplicity at most one. What separates an occurrence from an optional value — two occurrences falling in one observation interval — can only be seen when a source ticks faster than its observer. That is a cross-domain question and is treated at the end of this part.

Two restrictions on delay were forced by the totality proof rather than chosen. The delayed type must be a **data** type, one with no function type inside, because a delayed closure would have to persist across ticks and the logical relation for closures is tick-indexed and cannot be transported. And delay may occur only at **top level**, under no binder, because a delay under a lambda would re-evaluate its operand at the previous tick in an environment created at the current tick. Temporal state therefore belongs to declarations, and mappings are pointwise. This is the arrangement of *pre* in Lustre, where it lives in nodes rather than in functions [6], and its practical consequence is that a reusable stateful component is instantiated into fresh declarations rather than abstracted over.

Causality

The dependency graph on declarations comes in three variants. Structural dependency, *DependsOn*, records every reference in a body. Instantaneous dependency, *InstDependsOn*, excludes references under the delayed operand of a delay (the initial value is read at tick 0 and counts as instantaneous). A design is **causal** when its instantaneous graph is acyclic, witnessed by a bounded rank:

$$\text{Causal } \Delta := \exists \text{rank}, R. (\forall d. \text{rank } d < R) \wedge \forall a \ b. \text{InstDependsOn } \Delta \ a \ b \rightarrow \text{rank } b < \text{rank } a.$$

On the delay-free fragment this coincides with structural acyclicity, so the earlier acyclicity condition is the timeless special case rather than a replaced requirement. A structural cycle every one of whose paths passes through a delayed operand — $A := \text{delay } 0 \ B$; $B := A$, or a self-delayed accumulator — is causal and runs at every tick. A cycle that is partly delayed is not causal. A strict cycle, one through neither a delay nor a lambda, has no value at any tick.

Evaluation is a partial function: one tick, one environment, one term, at most one value, with no hidden evaluation order, and this holds unconditionally. It is total on causal designs: if Δ is causal and globally well formed and the inputs are well typed, every declaration has a value at every tick, related to its type by a logical relation, by an

induction on tick, rank, and derivation. An executable interpreter is proved sound for the relation, and every trace reported in Parts III–VIII was obtained by running it.

One gap should be recorded. A cycle guarded by a lambda, $A := \lambda x. A \ x$, is rejected by Causal, yet `declRef A` does evaluate — to a closure; only applying it diverges. Causal is conservative for lambda-guarded cycles, and the negative theorem covers strict cycles only.

Derived operators

Every temporal operator of the surface language reduces to delay and the primitive operators. There is no independent kernel definition of `count` for the reduction to be proved equivalent to; what was done instead was to elaborate each operator, check its typing and causality, and run it on a concrete input trace. In each row the declaration refers to itself. Each is a self-delayed cycle, the class that structural acyclicity forbade and causality licenses.

Surface	Declaration shape
<code>previous x</code>	<code>delay init x</code>
<code>previous x without init</code>	<code>delay none (some x); the absence is pushed to consumers</code>
<code>hold init e</code>	<code>getD e (delay init self)</code>
<code>count e</code>	<code>ite (isSome e) (1 + delay 0 self) (delay 0 self)</code>
<code>since e</code>	<code>ite (isSome e) 0 (1 + delay 0 self)</code>
<code>once e</code>	<code>delay false self v isSome e</code>
<code>every n</code>	<code>modulo-<i>n</i> counter over delay</code>
<code>rise b</code>	<code>b $\wedge$ $\neg$ delay false b, as an optional Boolean</code>

State has no identity of its own. A cell is a delay in a declaration body, and nothing refers to it because consumers refer to the declaration. There is consequently no notion of two writers to one cell in this kernel.

State preserves semantic identity and dimension. The typing rule is $\text{delay} : \tau \rightarrow \tau \rightarrow \tau$ for data τ , so a delayed tilt is a tilt and a backward difference over a time step has dimension `Length – Time` with no derivative primitive. Provenance holds in the reactive setting too: if no signature announces a concept and no input carries it, no value at any tick carries it. Temporal state carries tags; it never creates them.

Initialization is semantic, not validation. Every delay carries an explicit initial value. Two toy relations without one show why: the first tick is either undefined or nondeterministic. Adding or removing a delay, or changing an initial value, is an edit.

Clock Domains

Identity, not rate

“Contact and orientation move with the interaction; temperature moves with the environment.” A designer can say this before any rate is known, and it is a statement about which quantities are updated together, not about how often. BDL records it as a nominal **clock domain**, `ClockId`, and treats rate as validation data that never enters the kernel.

The time model is one global base tick and a schedule $S : \text{ClockId} \rightarrow \mathbb{N} \rightarrow \text{Bool}$ saying at which global ticks each domain activates. A period n induces the schedule $t \bmod n = 0$; the schedule lives outside the design. Domain-local time is not a separate counter but the sequence of a domain’s activations.

Each non-agnostic declaration is assigned a domain by a **clock environment** $K : \text{DeclId} \rightarrow \text{Option ClockId}$; a declaration with no domain is a pure mapping that may serve any domain. The clock is interface data in every sense that matters — clients’ validity depends on it, it is frozen under refinement, and changing it is an edit — and it is stored as a projection beside the interface, exactly as a concept’s representation is stored in Θ rather than in the type. Whether to fold it into the interface record is churn rather than semantics.

Rate and identity are distinct. A clone of a domain with the identical schedule is a different domain, and a direct wire between them is rejected; a domain at the same rate but shifted in phase reads different values through a transport. Rate changes are validation-only. They change the induced schedule and hence the observed values, but no client’s well-formedness. This is where BDL departs from synchronous languages that recover clocks by inference [7]. The

domain is authored, because the information needed to infer it does not arrive until realization binding, which in this workflow is the point at which the designer is least able to make the decision.

One transport primitive

Cross-domain reading is the second and last temporal form:

$$\text{sync } c \text{ init } e,$$

the value of e , evaluated in domain c , at the last activation of c strictly before the current tick, and init if there has been none. The multi-domain evaluation relation $\text{MEv } S \Delta I c t \rho e v$ indexes evaluation by the domain in which it takes place, and its two transport rules are

$$\frac{\text{prevAct } S c' t = \text{none} \quad \text{MEv } S \Delta I c t \rho \text{init } v}{\text{MEv } S \Delta I c t \rho (\text{sync } c' \text{init } e) v}$$

$$\frac{\text{prevAct } S c' t = \text{some } t' \quad \text{MEv } S \Delta I c' t' \rho e v}{\text{MEv } S \Delta I c t \rho (\text{sync } c' \text{init } e) v}.$$

delay is sync at the expression's own domain: $\text{delay init } e \equiv \text{sync } c \text{init } e$ in domain c , as an equivalence of the two relations. The kernel therefore has one temporal primitive — read a domain at its previous activation — and the single-domain semantics of the previous section is its diagonal. Under the always-active schedule, MEv coincides with Ev in every domain, so the earlier results are the one-domain special case rather than a replaced machine. A delay in a slow domain reads three global ticks back where a delay in a fast one reads one, with the same syntax.

A **domain judgment** $\text{Clocked } K c e$ rejects every other cross-domain reference: a reference stays in its domain or is agnostic, a delay needs a domain, and $\text{sync } c'$ switches the domain of its operand. Typing is unchanged and is blind to domains; the direct wire between two domains at the same value type is well typed and rejected only by the domain judgment. Placing the domain in the type instead was tried and set aside. Every pure mapping would need clock polymorphism, and nothing the type rejects is missed by the judgment.

Strictly before

A transport sees only source activations strictly before the destination tick. That is a choice with an observable alternative, and the alternative was built. A transport that lets simultaneously active domains see each other's current values makes the scheduler order observable: two priorities between the domains give two outputs. The strictly-before rule has no such parameter, and multi-domain evaluation is deterministic with no order between simultaneously active domains appearing in the semantics. It also makes cross-domain causality free. A transport's operand is never instantaneous, so $\text{Causal } \Delta$ is unchanged and no cross-domain cycle can be instantaneous. Every crossing costs one destination-visible step; “synchronous sub-domains evaluated in one instant” are, in this model, the same domain.

Every sync carries an explicit initial value, used at a destination activation with no earlier source activation. Totality extends to the multi-domain case — a causal, globally well formed design with well-typed inputs has a value in every domain at every tick — so the first activation is deterministic with the stated initial values. Semantic identity and dimension pass through transport untouched, by the typing rule; a crossing from Tilt@fast to Tilt@slow authorizes neither $\text{Tilt} \rightarrow \text{MotorAngle}$ nor $q \text{ Length} \rightarrow q \text{ Time}$.

Cross-domain occurrences and lossless buffering (Phase 9a)

Phase 5 left one problem open, and it is worth recording in full because its resolution shaped the data language.

The problem. Within one domain an occurrence is a stream of optional type: an input delivers at most one value per tick, so $\text{opt } \tau$ streams are exactly the streams of multiplicity at most one. Across domains this fails. sync is a zero-order hold: a slow consumer of a fast event source sees the last value only. Phase 5's Counterexample C ($\text{opt_loses_multiplicity_under_sync}$, **executable**) has two fast events at ticks 1 and 2 and one event at tick 2 indistinguishable at the slow activation at tick 3, and a single event at tick 1 followed by a quiet fast tick *dropped* outright. The counterexample is against sync as an *event transport*, not against optional types.

The window model, on tick sets. What the destination should see is the source’s occurrences at the source activations since the destination’s own previous activation — the *window* $\text{windowTicks } S \text{ src dst } t = \text{srcTicks } S \text{ src } (\text{prevAct } S \text{ dst } t) \text{ } t$. Phase 5 proved (*buffer_from_log_and_cursor*) that this window equals the source’s accumulated log read at the current tick minus the log length read at the previous destination activation: two single-instant reads, a sync of a source-side accumulator and a delay of a cursor. Policies are functions of the window; *latest*, *count*, and *count with latest* each identify distinct windows (*policies_lose_information*), and only the list is injective. So multiplicity and order are observable, buffering is required to keep them, and buffering is a structured use of the existing state basis *plus list data* — which the kernel then lacked.

The failed abstractions, generalized. Phase 9a tested the models the brief named. Model A, *latest*; B, *count*; C, a fold such as a sum; D, a fixed tuple of the newest entries; E, the list. A–D are lossy, with witnesses (*latest_not_lossless*: [1,2] vs [2]; *count_not_lossless*; *sum_not_lossless*: [1,4] vs [2,3]; *modelD_not_lossless*). The general statement behind A and D is *bounded_summary_not_lossless* (**formally proved**): any summary depending only on the newest k entries, for any fixed k , identifies a k -entry window with a $(k+1)$ -entry window. A lossless summary is injective and therefore unbounded (*lossless_iff_injective*). The claim discipline is precise: not that the list is the only lossless representation, but that any lossless one must be injective on windows and hence unbounded in size, and that an ordinary list is the smallest general sequence representation tested.

The object-language buffer. With list τ (the `Ty.list` constructor, `Value.list`, and six registered operators `nil`, `cons`, `length`, `take`, `reverse`, `head`, with `length` yielding $q \ 0$), the Phase-5 construction is five ordinary declarations (`Surface/Buffer.lean`):

```
log      @src : cons src (delay nil log)      -- source-side accumulator
logD     @dst : sync src nil log              -- the log, transported
seen     @dst : length logD                  -- log length now
cursor   @dst : delay 0 seen                  -- log length at the previous activation
window   @dst : reverse (take (seen - cursor) logD) -- the new entries, oldest first
```

Nothing new is evaluated: *delay*, *sync* and registered operators. *logD* is the one cross-domain read, an explicit sync with the explicit initial value `nil`. **Theorem M** (*buffer_window_correspondence*, **formally proved**): for every schedule, every input, every destination domain and every tick,

$$\text{MEv } S \Delta I \text{ dst } t \text{ [] window } \Downarrow \text{list } (\text{map } (I \text{ src}) (\text{windowTicks } S \text{ src dst } t)),$$

whenever the six declarations are realized as above and *src* is an input. The strictly-before rule is untouched — the window at t contains source activations $< t$ only. The elaboration is well typed in any environment with the declared types (*buffer_elaboration_well_typed*) and well clocked with *src*, *log* in the source domain and the rest in the destination (*buffer_elaboration_well_clocked*). The list summary is lossless by injectivity (*buffer_lossless*); its entries are indexed by the window ticks in strictly increasing order (*window_to_list_preserves_order*), and for every predicate on values the number of entries satisfying it equals the number of window activations whose value does (*window_to_list_preserves_multiplicity*).

Executed (`Experiments/BufferAlternatives.lean`, on Phase 5’s multi-rate schedule with fast events at ticks 1 and 2): at the slow activation 3 the window is [none, some 1, some 2], at 6 it is [none, none, none], while *latest* sees some 2 at 3 (*buffer_trace*); the two histories Phase 5 could not tell apart under *latest* are told apart by the window (*buffer_distinguishes_what_latest_identifies*); the elaborated design passes the typing, clocking and causality checkers. The buffer is also a Phase-8a system with no new binding kind: a sensor component provides its event *and* its log, a consumer requires the log through an ordinary transported binding with initial value `nil` and computes the window (*transport_trace*), whereas binding the event itself through *sync* yields *latest* and loses the tick-1 event (*latest_transport_loses*). “Buffer or not” is a choice of *which provided port to bind*, visible in the interface.

Capacity is a validation obligation. The kernel model is the unbounded log. A deployment has finite memory and must show that no window exceeds its capacity. *CapacitySufficient* $S \text{ src dst cap } T$ states that every window up to horizon T fits; it is decidable for a finite horizon; *requiredCapacity* computes the least sufficient value with a proof (*requiredCapacity_sufficient*); and for periodic schedules the horizon is unnecessary — a window never holds more than one destination period of source activations (*periodic_window_bound*), so one period is a sufficient capacity at every horizon (*periodic_capacity_sufficient*). Overflow policies are explicit functions of the unbounded window — *dropOldest* *cap*, *dropNewest* *cap* — and under a sufficient capacity both are the identity (*sufficient_capacity_preserves*,

bounded_buffer_agrees); under an insufficient capacity they change the trace (negE, executed: the three-entry window at tick 3 becomes two entries under either policy, capacity 2 fails the check at horizon 6, the required capacity at horizon 30 is 3). Consequently the only overflow policy that preserves the kernel semantics is to *reject the deployment*; dropping is a semantic change and must be written by the designer as ordinary computation over the window if wanted.

What is still not a kernel type. An event stream is a data-typed declaration in a domain; an occurrence is its value at an activation; a lossless cross-domain view of it is the five declarations above; latest, count, coalesce, drop, sample are ordinary computations over window. None of this needs an Event type, a buffer primitive, a scheduler order, same-tick visibility, an implicit overflow rule or an effect system, and none was added. The negative examples of the phase also settle a subtler point: a clone of fast with the identical schedule is a different domain, and listing the value does not make the direct read legal (negF) — equal rate is not the same domain, as in Phase 5.

Production. Production follows this exactly, with two additions the formal model deliberately left to engineering (ADR-0024, ADR-0027; docs/spec/deployment-capacity.md). A collection in the generated core is alloc::vec::Vec<T> stored last element first, behind a collections feature of the runtime crate turned on exactly when the plan carries a list; a program without lists stays allocation-free and Copy. No capacity is imposed by the core and no core ever drops a value. *The design writes its bounds:* a remembered collection is bounded by the operators that bound it — take cap ... — and by nothing else, so the deployable window is the Phase-9a construction with take cap on the log and the count kept apart:

```
count @fast := 1 + delay 0 count
log    @fast := take cap (cons x (delay [] log))
logD   @slow := sync fast [] log
seen   @slow := sync fast 0 count
cursor @slow := delay 0 seen
window @slow := reverse (take (seen - cursor) logD)
```

With cap ≥ required(fast → slow) its window equals the unbounded construction's at every tick — the production reading of bounded_buffer_agrees, differentially tested across the reference evaluator, the executable-IR interpreter and the generated core (corpus bounded_buffer against buffer, 7 and 3 000 ticks); with cap smaller the oldest values of a window are the ones missing (corpus overflowing_buffer). *The toolchain computes the bound and the requirement:* a sound static bound per declaration and cell (bdl-exec-ir::bounds), the required window capacity per crossing under the deployment schedule (bdl-reactive::capacity, the production Capacity.lean), a CollectionsReport with readiness, byte estimates and per-crossing requirements, and deployment.* diagnostics; an unbounded state is reported on a host and **refuses the artefact on a bounded-memory target** (bdld compile --bounded-memory --period). The static bound is an engineering analysis with no theorem behind it — recorded as such in Part XI. A ring buffer specialized for transport buffers was considered and deferred: the window is not a construct the compiler can recognize without a surface form (ISS-0001), and a bounded refinement must be visible in the design to be equivalent — take cap is exactly that.

Part VI — Behavior Systems, Groups and Reuse

Phase 8 asked whether behavior can be a first-class *design object*: packaged behind an interface, instantiated with fresh identity, bound to other behaviors, nested, and flattened into the same kernel — without any kernel change, without Event/Signal kernel types, without a StateHandler kernel, without general effects, and without name-based identity. Phase 8a answers for reusable components; Phase 8b for the authoring structure designers form before packaging. The two are kept apart on purpose, and the distinction is the main result of this Part.

Behavior components (Phase 8a)

The requirements report. The phase began with a requirements report (BEHAVIOR_SYSTEM_REQUIREMENTS.md) rather than a formalization: six capability questions — can a behavior be reused, instantiated twice, bound to other behaviors, nested, opened for later completion, substituted — and, for each, whether the answer is REQUIRED of the kernel, OPTIONAL, or DERIVABLE from what exists. The outcome was that everything is derivable from Phase-1 realization plus renaming, and the formal development proves it.

Renaming. Ren bundles four renamings — declaration identities, semantic identities, clock identities, output identities — and the kernel’s judgments are equivariant under it: HasType.rename (typing, given agreement of the three environments on the image; no injectivity needed), Satisfies.rename, Clocked.rename (the domain judgment, for a clock environment that agrees on the declared identities), and an abstract condition on evidence, Evidence.Equivariant. This is the machinery of Phase 8; nothing else is new.

Interfaces and components. A Port is a template declaration by local identity with the public part of its interface and its parameter clock. A BehaviorInterface has required ports (unresolved declarations a composer binds), provided ports (declarations offered to others), elaboration-time parameters (unresolved data-typed declarations bound to closed constants at instantiation), and clock parameters (the domains the template is written against). A BehaviorComponent is an interface, a template Design over local identities below a width, and a partition of its concepts and sinks into private (freshened per instance) and shared. Realizes $ev \vdash C$ is a predicate over the *existing* judgments: the template is a well-formed design; every required port is an unresolved declaration of the stated interface; every provided port is declared with it; parameters are unresolved, data-typed and clock-free.

Fresh instantiation. $fresh\ W\ k\ n = W \cdot (k+1) + n$ maps local identity n of instance k into the global space, with decode its inverse; the identities of two distinct instances never coincide (inst_decl_disjoint, Theorem A). The encoding is a device, not a semantics: any injective allocator would do, and production uses the same allocator its own ids come from. An instance is a component with a clock-parameter assignment; Ren.inst renames the template’s private identities into the instance’s range and its clock parameters to the assignment.

Bindings and flattening. A Binding realizes a destination port of one instance from a source — a port of another instance, or a closed constant — with an optional *transport*: none for a direct reference in the same or an agnostic domain, some init for sync from the source’s domain with an explicit initial value. A BehaviorSystem is a width, a list of instances, a list of bindings, the shared concept environment and the external sinks. flatten is the union of the renamed instances followed by the bindings applied as Phase-1 realization steps: the destination port is realized as declRef src or sync c init (declRef src). The result is an ordinary Design. Every existing pass consumes it unchanged.

Theorems B–I (Behavior/Preservation.lean, **formally proved**): under ComposeWF — every instance realizes its interface, every binding is well formed (BindingWF: types agree, a direct binding’s source is in the destination’s domain or agnostic, a transported binding’s source has a domain), external sinks are driven by at most one instance — the flattening is globally well formed (flatten_WF), causal under an acyclic inter-instance graph (flatten_causal), well clocked (flatten_wellClocked), single-driver (flatten_singleDriver); open ports stay open (open_port_stays_open). Evidence must be Monotone, Equivariant and PortSound (a discharged commitment survives when a port copy is realized by a reference to a declaration of the same interface); a concrete compositional evidence model would discharge these once and is still open.

Semantics (Theorem J, Behavior/Semantics.lean): for wiring designs with closure-free inputs and *direct* bindings, the value of a declaration in an instance evaluated alone with a consistent modular input equals its value in the flattened system (eval_flat_to_inst, eval_inst_to_flat, modular_iff_flat). The restriction is exact and recorded: transported

bindings under MEV need a domain-indexed input for the transported port, and higher-order bodies are not covered — the same obstacle in both directions.

Substitutability. `IfaceRefines A B`: B may replace A when every port A provides, B provides with the same type and clock, and every port B requires, A required. `substitute_composeWF`: replacing an instance’s component by a refining one preserves composition well-formedness. `toComponent` packages a whole system as a component of a chosen interface (the decidable side condition that the chosen interface is realized is not proved).

Counterexamples (`Experiments/BehaviorAlternatives.lean`): a name-based identity collides on double instantiation; a shared clock captured inside a template cannot be re-bound; a binding across domains without transport is rejected by the domain judgment; a binding of a required port to a wrong type is rejected; two instances driving one external sink violate single-driver. The lamp system — a source instance and two dimmer instances at different domains — is executed through the flattening.

Production (ADR-0021, ADR-0022; `bd1-system`; `docs/architecture/behavior-systems.md`): components, stored port contracts (“a component interface is a stored promise”: the contract is persisted and checked against the body, so an edit to the body that breaks the promise is reported at the contract), instances with fresh identity from the project’s allocator, direct and transported bindings, flattening with an *origin map* back to instances and ports so diagnostics land on the instance, composition validation, packaging, versions and substitution. There is one BDL: no system type checker, evaluator, clock judgment or code generator exists. Every project is a behavior system — a design with no components is the degenerate one — and the flat design is derived and never persisted. What production supports beyond the theorem fragment — transported bindings under the multi-domain evaluator, higher-order bodies — is *production-tested*, not proved (Part XI).

Multi-clock limitations, exactly. Theorem J is proved for EV (one domain) with direct bindings. The clock parameter list must cover the clocks of declarations and sinks; a sync clock appearing only inside a body is renamed to a fresh domain, harmless for EV and Clocked but something an elaborator should collect. `InstAcyclic`, the inter-instance graph condition for causality, is too coarse for extraction (below); a port-level graph would subsume both results.

Behavior groups (Phase 8b)

Three things that must not be confused. A designer who selects several mapping blocks and chooses *Group as Behavior* has made an authoring decision, not a semantic one. A designer who later chooses *Package as Component* has made a semantic decision: from that point the unit has an interface, can be instantiated with fresh identity, and can be bound.

	mapping / declaration	BehaviorGroup	BehaviorComponent
identity	<code>DeclId</code>	<code>GroupId</code> and a member list	template identities, fresh per instance
public interface	its own <code>DeclInterface</code>	none — only projections	required / provided ports, clock parameters
instantiation	—	none	fresh identities
flattening	—	none needed	flatten
semantics	kernel	none : authoring metadata	elaborates to declarations

The hierarchy mapping → group → component → system has exactly one semantic step, packaging, and that step is an elaboration into the Phase-8a system model.

Transparency. A `BehaviorGroup` is a `GroupId` and a list of member `DeclIds`; a `GroupedDesign` is a `Design` with a list of groups beside it; `eraseGroups` forgets the list. Every group operation — `group`, `ungroup`, `addMember`, `removeMember`, `move`, `merge`, `split` — acts on the list and leaves the design untouched, so every kernel judgment of the erased design is the *same proposition* before and after (Theorems A–G, each proved by `rfl` or `Iff.rfl`). That is not a weakness of the theorems; it is the point. Dependency edges are untouched; nesting is a relation on the flat group list with no recursive structure. In the Phase-1 vocabulary this is a fourth class below “validation-only”: nothing is rechecked because nothing changed.

Boundaries as projections. Over a finite enumeration of the design’s declarations: `crossIn` lists the non-members some member depends on (the required ports), `crossOut` the members some non-member depends on (the provided ports), `openMembers` the unresolved members, `drivenMembers` the members driving a sink, `privateMembers` the rest (Theorems H–L: `mem_crossIn`, `mem_crossOut`, `internal_not_crossIn`, `internal_only_not_crossOut`). The aggregate input socket a collapsed group shows is `crossIn ++ openMembers`; the output socket is `crossOut`. None is a declaration. Theorem H, `socket_no_fanout`, is what makes the socket safe: $a \in \text{crossIn}$ says *some* member depends on a and nothing about the others. Counterexample 6 shows what goes wrong if the socket is realized as a declaration every member reads (a member acquires an instantaneous dependency it never had); Counterexample 5 shows the tuple-returning-declaration encoding (the consumer of one mapping comes to depend on the inputs of all of them). The kernel has no tuples for boundaries, and this is a reason not to add them for this purpose — the products of Phase 9b are values, not sockets.

Inference rules and their counterexamples. Required = `crossIn`; provided = `crossOut`; private = members neither provided nor driving a sink; clock parameters = every clock the design uses (a group owns no domain, so nothing is captured — Counterexample 3 shows the reconnecting binding failing its clock condition when a clock is captured); physical sinks are not ports — a member that drives a sink keeps its drive edge inside the component and the sink stays external (Counterexample 4: converting a sink into a provided port yields a behavior with no physical effect). Two naive rules are rejected: every reference of a member as required exposes internal producers and changes behavior (Counterexample 1); only the members’ own open declarations hides external dependencies and leaves the template ill-typed (Counterexample 2).

Extraction. Extract builds two *restrictions* of the design: the component keeps the members and holds an unresolved copy of each crossing-in declaration as a required port; the residual keeps the non-members and holds an unresolved copy of each crossing-out member. It forms a Phase-8a system — instance 0 the residual, instance 1 the component, clock parameters mapped identically — with one direct binding per crossing declaration. Each binding is a Phase-1 realization step: the port copy is realized as a reference to the home copy. No body is translated, copied across the boundary, or rewritten. **Formally proved:** both templates realize their inferred interfaces (`restrict_realizes`, needing Evidence.InterfaceLocal: a discharged commitment depends only on the interfaces of the referenced declarations); the system is a well-formed composition (`system_composeWF`) and its flattening a well-formed design (`flat_WF`, Theorems M–Q); causality needed its own argument — a group with both inputs and outputs is never `InstAcyclic`, yet the flattened instantaneous graph is the original graph with every crossing edge subdivided through a port copy, and doubling the original rank witnesses it (`flat_causal`: rank $2 \cdot r$ on home copies, $2 \cdot r + 1$ on port copies); open members stay open; private members are not provided ports, source no binding, and are never referenced by the residual (`private_unobservable`); each crossing-out member is its own provided port (`provided_iff`) — several independent mappings yield several independent ports; and behavior is preserved (Theorem R, `orig_iff_flat`): for a wiring design with closure-free inputs, an original declaration and its home copy in the flattened extraction evaluate to the same value at every tick.

Identity. Before packaging nothing is renamed. After packaging the templates keep the original identities, and the flattened system carries Phase-8a fresh identities: an original n lives at $W + n$ on the residual side and $2W + n$ on the component side, with a port copy on the other side where the boundary is crossed; a later second instance receives $3W + n$. The group’s identity never becomes a component identity.

Production (ADR-0019; `.bdl/authoring.json`; `ui/layout.json`). A group persists as an identity and its member list by identity, in the authoring sidecar — no types, clocks, formulas, outputs or sockets; anything derived is recomputed and not authoritative if stored. Collapsed/expanded state, position, size, colour, member ordering and socket placement are layout and affect no check. Group operations never trigger semantic invalidation: the workspace does not re-run typing, causality, clock, output or hardware checks and does not clear cached verdicts, because these are the same propositions (`group_is_identity_on_design`). *Package as Component* computes interface candidates from the member set and lets the designer widen them — promote a private member, add an open required port — but not narrow below the inferred required set or turn a sink into a semantic port; the flattened result is guaranteed well formed under the stated hypotheses, and the causality check passes even though the instance graph shows edges both ways, so packaging is not gated on `InstAcyclic`. What cannot be inferred and is asked: whether an unresolved member is the component’s input or an open internal; whether a private member should nevertheless be exported;

whether a member's physical sink is private to the component; display name and documentation. Packaging a group *inside* a component body is open (ISS-0007).

Hardware interaction

Components partition sinks into private and external. Private sinks are freshened per instance, so two instances of a component with a private status LED are two LEDs and two requirements; external sinks are shared and may be driven by at most one instance in a system (`ExternalSingleDriver`). Requirements generation (Part VIII) runs over the flattened design's sinks, so the number of instances is exactly what the board must accommodate — a fact the hardware validator sees only through the flat design, which is the intended separation.

Part VII — Quantities, Units, Charts and the Formula Composer’s Formal Basis

Dimensions entered the kernel in Phase 3 (Part III): a quantity $q\ d$ carries an exponent vector, the dimension algebra lives in the types of the primitive operators ($\text{mul} : q\ d_1 \rightarrow q\ d_2 \rightarrow q\ (d_1 + d_2)$), and dimensional typing rejects $\text{length} + \text{time}$ where the erased numeric baseline accepts it. Units were surface from the start (D-32): a literal $n\ u$ elaborates to a scaled dimensioned literal; changing the unit changes the value, never the type. Phase 10 asked what more the same quantity in different units needs — extracting a coordinate, constructing from a coordinate, letting an editor infer dimensions for incomplete expressions, keeping units out of type identity — and Phase 10b revisited the one conclusion of Phase 10 that turned out to be too strong. This Part records both, the production realization, and the design decision about who owns a unit in the authoring surface.

Three notions kept apart

notion	formal object	layer
physical quantity	a kernel value of type $q\ d$: a canonical magnitude and a dimension	kernel
unit coordinate	$\text{inUnit}\ q\ u = q / \text{scale}(u)$, a dimensionless number	surface elaboration
display / authoring unit	$\text{Presentation.preferred} : \text{SemanticId} \rightarrow \text{Option Unit}$	presentation

Collapsing any two of these is the mistake the phase was designed to avoid: a quantity with a unit inside it splits one quantity into many types; a coordinate that remembers its unit is a runtime tag nobody needs; a display preference that enters elaboration changes semantics when a designer changes what they like to see.

Linear units: no kernel construct

Six models were compared. Literal-only elaboration (A) is insufficient alone — a coordinate is needed for display, for normalization (tilt in degrees $\div 90$), and for export. $\text{inUnit}(q, u) : \text{Scalar}$ by elaboration (B) and $\text{withUnit}(x, u) : Q[d]$ by elaboration (C) were adopted: inUnit is $\text{div}\ q\ (\text{lit}\ d\ \text{scale}(u))$, typed $q\ (d - d) = q\ 0$ (inUnitE_typed) and rejecting a quantity of another dimension by inversion (inUnitE_safe); withUnit is $\text{mul}\ x\ (\text{lit}\ d\ \text{scale}(u))$, typed $q\ (0 + d)$ (withUnitE_typed) and never a concept ($\text{withUnitE_is_quantity}$); a literal is withUnit . First-class runtime units (D) were rejected — no tested case delays, syncs, stores or compares a unit, and a UI dropdown is not a runtime value. Units in quantity types (E) were rejected — $1\ m$ and $100\ cm$ would differ in type while being one quantity (exA: same type, equal values). A kernel conversion primitive (F) was rejected — $\text{convert}\ x\ u\ v = \text{inUnit}\ (\text{withUnit}\ x\ u)\ v = x \cdot \text{scale}(u) / \text{scale}(v)$ (convert_eq , ev_convertE), transitive (convert_trans), identity on the same unit (convert_self), mismatch rejected never coerced (inUnit_mismatch). Unit operations construct nothing ($\text{unitOps_no_construction}$), and on a concept they go through rep — so $\text{inUnit}(\text{TiltValue}, \text{deg})$ obeys the representation discipline, and rebuilding a concept from a coordinate requires mk under the concept’s grant. Tilt and MotorAngle are both readable in degrees and stay distinct under every unit round trip (nominal_distinct); Brightness and Opacity likewise. Unit compatibility is not identity.

The numeric domain, in the open

The formal development says exactly where exactness holds. Unit laws are proved over an abstract scalar domain $\text{Scalars}\ K$ — a commutative monoid with a division that cancels multiplication by a non-zero scale — and instantiated by Sym , the free abelian group on the generators $2, 3, 5, 127, \pi$. Every registered scale is an element, exactly: $\text{deg} = \pi/180\ \text{rad} = \pi \cdot 2^{-2} \cdot 3^{-2} \cdot 5^{-1}$, $\text{inch} = 127/5000\ m$. Round trips (inUnit_withUnit , withUnit_inUnit), conversion and mismatch hold with π symbolic: $90\ \text{deg}$ in radians is $\pi/2$ (exD), not $1.5707\dots$. The executable kernel’s magnitudes are naturals, so its registry uses integer scales relative to canonical sub-units — $0.1\ mm, ms, \text{arc-second}, g, K/180$ — under which $mm, cm, m, km, \text{inch}, \text{deg}, \text{turn}, ms, s, \text{min}, g, kg$ and K are exact; round trip 1 is exact for every positive scale ($\text{inUnit_withUnit_nat}$) and round trip 2 exact when the scale divides the magnitude ($\text{withUnit_inUnit_nat}$), the strongest law integer division admits; radians are *not* in that registry because their scale is not an integer in any degree-compatible basis. Production uses IEEE doubles with radians canonical (ADR-0011; bdl-eLab::units): neither round trip holds exactly there, and the document says so rather than hiding it (Part XI).

A unit itself is (id, dim, scale) in a registry; its symbol is not part of it — two units with equal identity are the same unit whatever they are spelled — and unitsFor reg d is sound and complete relative to the registry (unitsFor_sound, unitsFor_complete).

Presentation, and the realistic formula

Presentation is a separate object; a design with a presentation is a pair, and every kernel judgment of the pair — typing, evaluation, dependencies, clocks — is literally the judgment of the design (presentation_irrelevant_*, by construction, rfl); what changes is the displayed number (presentation_changes_display). Ordering compares canonical magnitudes through ltAt, which mentions no unit; comparing *displayed* coordinates would not be safe, since integer display can identify distinct magnitudes (display_may_identify_distinct).

The recurring formula $\text{Tilt} \rightarrow \text{Brightness: tilt} / (90 \text{ deg})$ and $\text{inUnit}(\text{tilt}, \text{deg}) / 90$ are both dimensionless and evaluate equal for every tilt (normalizations_agree, exF). The representation-aware form is the second — it names the unit the designer thinks in and keeps the divisor a plain number — and neither changes the concept: both consume rep tilt, and the result is wrapped as Brightness by the block’s own signature (ADR-0013).

Design guidance, not a theorem, for persistence: the *source literal unit* (90 deg) is semantic source — its scale enters elaboration — and lives in the formula text; the *concept preferred display unit* (Tilt shown as deg) is authoring metadata beside the display name and never enters elaboration; a *simulation-input display unit* is session state.

Unit ownership in authoring

The production Formula Composer made this a rule (ADR-0028): in $\text{tilt} / 90 \text{ deg}$, tilt is a semantic reference and 90 deg is a designer-authored quantity literal. **A literal owns an editable unit; a semantic reference does not; a display boundary may carry a presentation unit.** The reason is semantic ownership. A literal’s unit is part of what the designer *wrote* — its scale enters elaboration, so switching it with the quantity kept ($180 \text{ deg} \rightarrow 3.141592653589793 \text{ rad}$) is a change of spelling that preserves meaning, and editing the coordinate is a change of meaning; both are the designer’s to make on the literal. A reference’s kind is its declaration’s: tilt is a Tilt, represented as an angle, and no unit is part of that; offering a unit pop-up on a reference would invite a rewrite of the representation binding of a concept from inside one formula, which is exactly the representation escape hatch the grant discipline forbids. The display of a reference’s *value* — in simulation, in the inspector — may use the concept’s preferred unit, which is presentation. This decision is traceable to unitOps_no_construction, withUnitE_is_quantity and presentation_irrelevant_*, not to taste: the Composer draws a literal as two fields (coordinate and unit pop-up) and a reference as a chip with its concept’s socket glyph, because those are the two semantic situations.

Affine units: the failure of scale-only units, and the revision

Temperature scales are affine: $\text{canonical} = \text{scale} \cdot x + \text{offset}$. Phase 10 showed, executably, that the linear model cannot represent °C or °F — $0 \text{ °C} = 273.15 \text{ K}$, which no scale produces from 0 (celsius_not_linear) — and that an affine *literal* and an affine *coordinate* nevertheless elaborate exactly with no kernel change: $n \text{ °C} \mapsto \text{lit Temp } (n \cdot s + \text{off})$, $\text{inUnit °C } q \mapsto (q - \text{off})/s$, in the K/180 basis where °C and °F have integer scales and offsets (affLitE_typed, affInUnitE_typed, affine_roundtrip_ev; $0 \text{ °C} = 32 \text{ °F}$, $100 \text{ °C} = 212 \text{ °F}$ in scales_agree). What breaks is arithmetic on *absolute* values: $20 \text{ °C} - 10 \text{ °C}$ is a difference of 10 K (delta_is_linear), while $10 \text{ °C} + 10 \text{ °C}$ is well typed at q Temp and reads 293 °C (sum_of_points_is_not_a_point, sum_well_typed). The kernel’s dimension cannot separate a point on the scale from a difference.

Phase 10’s conclusion, as first recorded: the missing information is a *sort* — point or difference — on top of the dimension (AffSort, with affAdd/affSub as the affine-space rules: point + point refused, point - point = delta), and a Formula Composer cannot offer only difference units for a delta slot from the dimension alone (delta_candidates_need_sort). Affine conversion was called “safe now”; affine *arithmetic safety* was said to need the sort and was deferred, with the sort treated as potentially necessary for future production.

Phase 10b tested the smaller hypothesis that for *conversion* chart-specific information is intentionally erased at coordinatization and only the affine transformation structure between coordinate systems must be preserved. It holds, and the conclusion was revised. This revision is recorded as research evidence, not smoothed over.

Charts: coordinate erasure and conversion functoriality (Phase 10b)

Over one physical dimension, a chart is (scale, offset), valid when $\text{scale} \neq 0$, with

$$\text{reconstruct}_u(x) = s_u x + o_u, \quad \text{coord}_u(q) = (q - o_u)/s_u.$$

The scalar domain is an abstract field (Field K: commutative, with inverses of non-zero elements, negatives and fractions) instantiated by `q`, exact rationals built in the development as a quotient of `Int` fractions with every law proved from `Int`’s ring identities — because core Lean’s `Rat` proves its algebra with `Classical.choice`, which this development has kept out of every proof. Concrete equalities are decided by cross-multiplication.

Formally proved, over any field:

law	theorem
chart left inverse <code>coord u (reconstruct u x) = x</code>	<code>chart_left_inverse</code>
chart right inverse <code>reconstruct u (coord u q) = q</code>	<code>chart_right_inverse</code>
conversion is affine: <code>C(u,v)(x) = (s_u/s_v)·x + (o_u - o_v)/s_v</code>	<code>convert_is_affine</code> , <code>convertMap</code>
identity <code>C(u,u) = id</code>	<code>convert_identity</code>
composition <code>C(v,w) ∘ C(u,v) = C(u,w)</code> — from the chart laws alone, no algebra	<code>convert_compose</code> , <code>convertMap_compose</code>
inverse <code>C(v,u) ∘ C(u,v) = id</code> , both ways	<code>convert_inverse</code>
difference map <code>f(y) - f(x) = a·(y - x)</code>	<code>difference_map</code>
offset cancels <code>f(x + δ) - f(x) = a·δ</code>	<code>difference_offset_cancels</code> , <code>difference_converts_linearly</code>
linear-part functoriality <code>L(u,u) = 1</code> , <code>L(v,w)·L(u,v) = L(u,w)</code>	<code>linear_part_identity</code> , <code>linear_part_compose</code>
not additive when the offset is non-zero	<code>not_additive_of_offset</code>
erasure keeps conversion structure <code>coord v q = C(u,v)(coord u q)</code>	<code>unit_erasure_preserves_conversion_structure</code>
display switch preserves the quantity <code>reconstruct v (C(u,v)(coord u q)) = q</code>	<code>display_switch_preserves_quantity</code>
coordinate edit changes the quantity	<code>coordinate_edit_changes_quantity</code>

Compatible charts form a groupoid of affine isomorphisms (theorem-level; no category-theory framework was built). Unit coordinates erase chart identity while preserving affine coordinate change; differences inherit the linear part of that transformation. A conversion with a non-zero offset is *not* an additive homomorphism, and this document never calls Celsius-to-Fahrenheit one.

Celsius and Fahrenheit, exactly. Kelvin canonical, `celsius = (1, 27315/100)`, `fahrenheit = (5/9, 45967/180)`: `C(°C, °F)(x) = 9/5·x + 32` and `C(°F, °C)(x) = 5/9·(x - 32)` for every `x` (`CtoF_closed`, `FtoC_closed`); `0 °C = 32 °F`, `100 °C = 212 °F`, `-40 °C = -40 °F`, the `°C→°F→°C` round trip, `°C→K→°F` equal to `°C→°F`, `Δ10 °C = Δ18 °F` from two base points, `Δ°C = 5/9·Δ°F`, and the non-additivity counterexample `f(0+0) = 32 ≠ 64` (`executed`, `Experiments/AffineExamples.lean`).

Not a Celsius special case. A 10-bit ADC over 5 V, millivolts, and a calibrated reading with a 0.5 V zero instantiate the same theorems — composition through millivolts, inverse back to raw, a difference independent of base (`exH`); so does an encoder with home offset, `angle = 45/512·count + 30°` (`exI`). Unit conversion and sensor calibration are one affine-map abstraction.

The scalar coordinate is chartless. `32` is a Fahrenheit coordinate of `0 °C` and a Celsius coordinate of `32 °C` (`coordinate_needs_chart`); the destination chart supplied to `reconstruct/convert` interprets it; no runtime tag is needed, and no theorem requires a unit to be data, delayed, synced, stored or compared — Phase 10’s rejection of runtime units re-audited and confirmed.

AffSort, revised. Conversion never takes a sort and the sort checker never takes a chart (`sort_orthogonal_to_conversion`, `conversion_orthogonal_to_sort`, by construction). Two claims coexist: (A) unit conversion is complete and correct without point/delta; (B) a domain-specific checker may still use point/delta to reject `point + point`, as *optional physical-arithmetic validation*, orthogonal to A. The design decision D-106 was rewritten from “affine conversion works, but point/delta is missing information” to “affine conversion is complete as coordinate-change semantics; point/delta is additional validation information for restricting physical arithmetic.” `Ty.q d` is unchanged; no conversion theorem needed a sort in the type.

The production chart model

Production (`crates/bdl-elab/src/units.rs`, ADR-0028 and its amendment) realizes the registry as `UnitDef { id, symbol, dim, chart }` with

```

pub enum Chart {
  Linear { factor: f64 }, // canonical = coordinate × factor
  Affine { scale: f64, offset: f64 }, // canonical = coordinate × scale + offset, scale ≠ 0
}

```

and coord, reconstruct, linear_part on the chart; `convert(x, from, to) = coord_to(reconstruct_from(x))` is the one conversion operation and hides the chart's shape. The Phase-10b laws are property-tested on f64 — identity, composition, inverse, display switch preserving the quantity, the difference map linear — within a few ulps, including the Celsius/Fahrenheit charts; the formal laws are exact. The affine charts exist as tested infrastructure and are **not** in the registry offered to formulas or the Composer's pickers (ISS-0004): conversion and display of an absolute temperature are safe, arithmetic on absolute temperatures is a separate validation concern, and the product does not offer 20 °C in a formula until the point/difference validation exists or a decision is taken not to need it. The symbol is presentation; identity is the id; inch is spelled out because in is the membership keyword.

The production contract for floating point, as recommended by the formal phase: for registered charts u , v , w and sampled x , $|C(u, u)(x) - x| \leq \epsilon |x|$, $|C(v, w)(C(u, v)(x)) - C(u, w)(x)| \leq \epsilon |x|$, $|C(v, u)(C(u, v)(x)) - x| \leq \epsilon |x|$, $|C(u, v)(x + \delta) - C(u, v)(x) - L(u, v)\delta| \leq \epsilon |\delta|$, and $|reconstruct_v(C(u, v)(coord_u(q))) - q| \leq \epsilon |q|$, with ϵ a few ulps scaled by the largest coefficient, plus exact-rational oracles — never a claim of exact f64 identity or composition.

The Formula Composer's formal basis

A structured composer needs, from the formal side, exactly three things: typed holes, expected-dimension propagation, and candidate-unit inference. `Surface/Composer.lean` gives them and no more.

`PEExpr` is holes, known operands by dimension (a literal, a reference or a known equation result), and `add/sub/mul/div`; it is never executed. `check` is bottom-up on the known parts. `solve` pushes an expected dimension down by the local rules of each operator in the `Dim` group — `add/sub`: both sides get the result; `mul`: the other side gets $r - d$; `div`: numerator $r + d$, denominator $d - r$ — deterministically and in one pass; a product of two holes is reported *unsolved*, never searched. **Formally proved** for the whole fragment: `solve_sound` (filling the holes as solved makes the expression check at the expected dimension) and `solve_complete` (any filling that checks agrees with the solution — a one-hole equation in an abelian group has one solution); `candidates_sound`, `candidates_complete` and `slot_sound` (the units offered for a hole are exactly the registered units of its solved dimension); `refCandidates_sound` for declaration references. Executed: $? / (1\ s) : \text{Speed} \Rightarrow ? : \text{Length}$ with candidates mm, cm, m, km, inch (exG); $\text{Force} \times ? : \text{Torque} \Rightarrow ? : \text{Length}$ (exH); the explanation $\text{Speed} = [\text{Length}] / [\text{Time}]$ as slots (`speed_slot`). No unification, no symbolic algebra beyond the group laws.

The Composer does not need to make a unit part of an expression's type: an affine quantity literal needs the physical dimension, the chosen chart and the scalar coordinate, and changing the displayed chart is `coord_u(q) → coord_v(q)` with q preserved. What it *would* need beyond the dimension to offer only difference units for a delta slot is the point/difference sort — which is why that sort is recorded as optional validation information the editor could carry, not as anything in the kernel.

Part VIII — Physical Outputs, Hardware Validation and Deployment Capacity

Physical Outputs Without Arbitration

The model

A declaration computes a value; it does not move hardware. Physical effect happens only through an explicit **drive edge** from a declaration to a **physical sink**:

- OutputId — the nominal identity of a sink, a logical actuator channel;
- $\Omega : \text{OutputId} \rightarrow \text{Option OutputSpec}$, a sink’s accepted type and clock, declared by the deployment;
- $\beta : \text{DeclId} \rightarrow \text{Option OutputId}$ — the drive edges, a write-once per-declaration projection of the same shape as K ;
- $\text{DriveWF } \Omega \ K \ \Delta \ \beta$ — each edge is well formed when the driver’s expected type *equals* the sink’s accepted type and the driver’s clock is the sink’s clock;
- $\text{SingleDriver } \beta$ — at most one driver per sink: if $\beta \ d_1$ and $\beta \ d_2$ are both some o then $d_1 = d_2$;
- $\text{CompleteOutputs } \beta \ req$ — every required sink is driven.

Nothing was added to types, typing, the domain judgment, the evaluation relation, or the grant. The binding neither coerces nor converts nor synchronizes. A declaration typed `Tilt`, or bare `q Angle`, cannot drive a `MotorAngle` sink. A sink that accepts a representation type needs an explicit `rep`-typed declaration in front of it, so the distinction between semantic target and hardware representation stays visible. A slow driver reading a fast value must sync it upstream, since a fast driver cannot drive a slow sink and the edge never synchronizes.

Sink identity is nominal for the same reason concept identity is. Keying the binding by type collides when two servos accept the same type; keying by concept conflates a concept with a device, since one concept may feed several; keying by declaration makes two declarations that both mean the steering motor into two sinks, and the multiple-driver counterexample cannot even be stated. “Desired steering angle” is a value; “the steering motor” is a resource; the kernel keeps them in different sorts.

One final driver

The principle is *many contributors, one explicit final driver*. Take two declarations driving one sink, each globally well typed, well clocked, causal, and individually well formed. Only `SingleDriver` fails, and it fails globally rather than at either edge. What has gone wrong is semantic: with two drivers the physical output is not a function of the tick, and there is a tick at which the sink receives two values. With one driver, the physical output is a partial function of the tick, and with `SingleDriver` it is unique wherever it exists:

$$\begin{aligned} \text{PhysicalOutput } S \ \Delta \ I \ \Omega \ \beta \ o \ t \ v &:= \exists d \ spec. \\ \beta \ d &= \text{some } o \wedge \Omega \ o = \text{some } spec \\ \wedge \text{MEv } S \ \Delta \ I \ spec.\text{clock } t \ [] \ (\text{declRef } d) \ v. \end{aligned}$$

Contributors are dependencies, not drivers. `base + corr -> final -> motor` passes every check; priority is an ordinary conditional in the single driver; `blend`, `maximum`, and `clamp` are ordinary declarations of the target type. Why arbitration must be explicit is best shown rather than argued: first-wins, last-wins, and maximum over the same value graph give three different physical outputs. A hidden policy is a design decision made on the designer’s behalf.

Binding an unbound declaration to an undriven sink is a refinement and preserves `SingleDriver`. Binding to an already-driven sink is invalid. Retargeting a sink’s accepted type, renaming a sink, or detaching an edge invalidates an unchanged design. Partial designs may leave sinks undriven; executable designs may not.

What was removed

The earlier draft of BDL had an effect row on the behavior judgment, action requests as values, and per-context policies that allowed, suppressed, transformed, and arbitrated requests, resolved in a dedicated phase of each tick. Each was tried in toy form against the single-driver model, and the outcome is narrow. Direct effect rows — the set

of sinks a declaration drives — are exactly the drive edges, and single-driver is exactly their pairwise disjointness. Propagated rows, which also include the sinks of everything a declaration reads, flag a valid design in which a display reads the driver. Action values move the conflict into the collector that consumes them, which must then be a policy, which is the single driver by another name. None of this bears on richer effect systems; it says that these formulations add no rejection the single-driver rule lacks. Per-context action policies no longer exist as a mechanism, and the StateHandler cases that involved them — event-latched activation with exit-wins, state-local output choice, nested choice with an output — are ordinary declarations with one driver and were run as such.

Why $A \rightarrow ()$ is not a sink (Phase 12)

Once zero-input relationships have the canonical type $() \rightarrow A$ (Part III), the dual form suggests itself: could a physical consumer be a relationship $A \rightarrow ()$, a function that takes a value and returns nothing? The kernel has no unit type, so the form cannot be written; Phase 12 records why it should not become writable (**formally proved**, denotationally). In a pure total language every function into the one-point type is the same function — `unit_codomain_collapse` : $\forall f g : A \rightarrow 1, f = g$, by function extensionality alone — so two “consumers” $\text{sink}_1 \text{ sink}_2 : A \rightarrow ()$ are indistinguishable (`consumers_indistinguishable`): nothing in a value of that type says *which* physical output receives A, or that anything receives it at all. The kernel’s evaluation relation has no effect component (`eval_independent_of_drives`); a derivation relates a tick, an environment, a term and a value, and the physical consequence of a value lives outside it. Naming a receiver needs an effect or output semantics, and BDL already has exactly one: the sink identity `OutputId`, the drive edge with `DriveWF` (the driver’s type equals the accepted type, in the sink’s domain), `SingleDriver` and `CompleteOutputs`. The receiver is named by the edge, the value delivered is the driver’s, of type A, and the driver is a unit-domain declaration whenever the sink accepts a concept (`driver_is_unit_domain`). This is a design result, not a preference: physical consumption stays on the drive boundary, and $A \rightarrow ()$ is removed from consideration.

The separation this makes explicit is worth stating once, because Part IX depends on it. *Behavior semantics* is environment-provided inputs ($(I(d, t))$), pure internal computation ((Ev/MEv)), and output obligations (`DriveWF`, `CompleteOutputs`, `PhysicalOutput`). *Realization* is sensor reads, ADCs and buses on the input side, and GPIO, PWM and device I/O on the output side — the platform adapter’s business, named nowhere in the kernel. A source is not a read; a drive edge is not a write. Both are boundaries at which the environment provides and receives values, and the kernel’s theorems are about what happens between them.

Deployment provision of Sources (Phase 13, PRP-0001 audited)

A Source $s : () \rightarrow C$ is environment provision at the concept’s type (Part III): the kernel input gives a `Temperature`, and nothing in the design or the deployment says how a value of `Temperature` comes to exist. On a product it does not; an ADC yields counts, a GPIO a level, an I²C sensor a register image. Production’s proposal PRP-0001 (876005c, status *draft*) asks for a construction that turns the abstract Source into a raw reading $r : () \rightarrow R$ plus a realization $s := \text{tr}(r)$ taken from a *device profile*, with a theorem that the design cannot tell the difference — so that the transducer, today the platform adapter’s unchecked host code, becomes an ordinary term the kernel types and the compiler compiles, while the design file keeps mapping `TempSensor : () \rightarrow RoomTemp`. Phase 13 tested the proposal as a hypothesis; the construction exists, and four of its seven claims had to be corrected (**formally proved** unless marked; `Surface/Provision.lean`).

The construction. A *channel* is a representation type `rep`, a term `tr`, its transfer function `transfer` on values, and the coherence computes (the term computes the function on every raw-typed value); the term is **pure** — no `declRef`, `delay` or `sync`. A *device profile* is a raw type (sem-free data) and its channels; it mentions no concept. A *provision* is one fresh raw declaration `r` with its clock and an assignment of channels to target Sources — several targets may share one raw reading (an IMU image feeding pitch, roll and acceleration), and one target is the singleton case. Then

$$\text{provision}(\Delta, P) = \Delta [r \mapsto \langle \text{raw}, [] \rangle \text{ unresolved}] [s \mapsto \text{mk}_c(\text{tr}(\text{declRef } r)) \text{ for each target } s : \text{sem } c],$$

with `tr (declRef r)` alone at a representation-typed Source; the raw declaration’s kernel type is `raw` and its canonical type $() \rightarrow \text{raw}$ (Part III). Fitting is decidable: at `sem c` the concept’s representation is the channel’s; at a representation type the types coincide. Nothing enters the kernel: `provision` is a function on environments built from `DesignDecl`, `declRef`, `app` and `mk`.

Why purity, and why a transfer function. Typing the channel term in the *empty* design already forbids reading a declaration (`Channel.WF_refFree`), but not memory: $(\lambda k. \lambda n. k) (\text{delay } 0 \ 1)$ is typed at $q_0 \rightarrow q_0$ and maps the raw value 7 to 0 at tick 0 and to 1 at tick 1 (exD, **executable example**). A transducer must be a function of the raw reading, so the profile condition is purity — equivalently, typed in the empty design and delay-free (`pure_iff_delayFree_of_wf`). The channel carries transfer beside `tr` because the *induced* abstract input must be a function: extracting it from per-tick existence of a value would be a choice principle, which this development does not use. The grant argument the proposal relied on is a theorem of the existing rules: a declaration typed `sem c` is realized under `Grant.of (sem c) = {c}` (`realization_checked_under_own_grant`, `grant_of_sem`), and a channel term typed under `Grant.none` constructs nothing (`channel_constructs_nothing`); the same $\lambda x. \text{mk RoomTemp } x$ is refused under the empty grant and accepted under the Source’s own (exC).

Structure. Provision is an environment refinement (`provision_envRefines`): each target keeps its identity and interface and goes from unresolved to realized, `r` is new, nothing else moves. The provisioned design is globally well formed (`provision_wf`) from the abstract design’s well-formedness, monotone evidence, the provision’s preconditions — and evidence for each target’s *commitments* on its new realization, a hypothesis the proposal did not state: a Source’s commitments are obligations on the profile. Causality is preserved with the rank shifted by one and `r` at the bottom; purity keeps the channel term edge-free (`provision_causal`). The domain judgment is preserved with $K \ r = K \ s$ (`provision_wellClocked`); no device clock is introduced, and a device with its own rate is a later sync.

Transparency. For every schedule, domain, tick, term that does not mention `r`, and local environment whose closures avoid `r`,

$$\text{MEv } S \ \Delta \ I \ \text{ct} \ \rho \ \text{ev} \Leftrightarrow \text{MEv } S \ (\text{provision } \Delta \ P) \ I' \ \text{ct} \ \rho \ \text{ev}, \quad I = \text{induced}(\Delta, P, I'),$$

where the induced input gives each target the wrapped transfer of the raw reading and leaves every other identity as `I'` gives it (`provision_transparent`). The hypotheses the proposal lacked are stated: the raw input is typed at `r` and closure-free, and the abstract design mentions no `r` — true of every globally well-typed design (`NoMention.of_globalWF`). The observation boundary has two equivalent forms: syntactic (`r \notin e.refs`) and by typing (a term typed in the abstract design cannot name `r`, `provision_transparent_typed`). The proof is one simulation lemma over `MEv` with the closure invariant “no closure body mentions `r`”, instantiated in both directions and once more for input congruence; the provisioned target’s value comes from the pure term’s canonical evaluation transported to any design, input, domain and tick (`Transduces.mev`, `MEv.of_ev_pure`) — at top level, where a realization is evaluated, which is what avoids a closure-equivalence theorem. Physical outputs are unchanged (`provision_physicalOutput`).

Trace abstraction, exactness, strictness. Every behaviour of the provisioned design under a raw input is a behaviour of the abstract design under the induced input (`provision_abstracts`): deployment *restricts* the abstract environment; it does not give the abstract design its meaning. The proposal’s converse — “when the transducer is surjective the trace sets are equal” — is wrong as stated: pointwise surjectivity is an existence per tick, and with a shared raw reading it is insufficient even in principle (`id` and `succ` from one reading are each onto, and the abstract pair (5, 9) has no witness, `no_joint_witness`). Equality needs a *joint section*, a raw trace every channel transfers to what the abstract input gives its target (`provision_exact`); for one channel a pointwise right inverse on typed values is one (`JointSection.one`). The strict case is executed: a saturating ADC never yields 451 K, the abstract design observes `TempSensor = 451 K`, and no provisioned deployment does (exE).

Re-application and commutation. The proposal called provision “idempotent per Source”. After provision a target is realized, is no longer a Source, and the operation’s precondition fails because `r` is no longer fresh (`provision_not_reapplicable`; the Source role moves to `r`, `provision_source_role`). The totalized function does satisfy $P(P(\Delta)) = P(\Delta)$ (`provision_idem_total`) — only because a second pass overwrites every target with the same body; a second pass with a different term is not a refinement (`provision_reprovision_not_refinement`). Independent provisions commute *exactly*, as environment equality (`provision_comm`), and the channel assignment is a set (`provision_perm`).

Executed (`Experiments/ProvisionExamples.lean`): the identity GPIO channel on a `bool` Source; the thermistor $T = 2n + 250 \text{ K}$ on `TempSensor`, with the consumer `tooHot` computing the same truth values from counts as from the induced temperature and the provisioned design proved refining, causal and well clocked; rejected profiles; the impure typed term; the saturating ADC; one IMU image provisioning pitch and roll with level reading both.

What changed in the proposal (design recommendation): the purity condition; the transfer function beside the term; the commitment hypothesis; the transparency hypotheses; the joint-section exactness; “not re-applicable” for “idempotent”; shared raw as the primitive with the singleton as its case; the terminology — *abstract Source*, *provisioned Source*, *raw declaration*, never “monomorphised”, which Part IV’s rank-1 polymorphism owns; and the removal of the dependency on designer-facing °C/°F (ISS-0004): the thermistor is a linear chart on counts and the language keeps kelvin. Output provision remains the duality note the proposal made; nothing here made it free. Verdict: provision is a deployment/surface construction over existing kernel terms, not a kernel construct. The proposal stays a draft, revised, for human review; nothing in production implements it.

Target-Specific Hardware Validation

Everything to this point is board-independent. A design that is typed, causal, clock-consistent, and output-complete may still not fit the microcontroller it is to run on, and that question is answered by a validation layer that never touches the kernel.

Resources, capabilities, requirements

A target is a finite table. A **resource** — a pin — has an identity, a list of **capabilities** from a shared vocabulary (digital in/out, PWM, analog in, interrupt, the I2C, SPI, and UART lines), and, per capability, the **unit** backing it, such as the timer behind a PWM pin or the peripheral behind a bus line. A **hardware description** is a list of resources with a per-capability sharing policy: bus lines are shareable, everything else is exclusive.

A design’s needs are **requirements**. Each has an identity, one capability, an optional fixed resource for a manual pin choice, and an optional membership in a group with a unit relation, *same* or *distinct*:

$$\text{Requirement} = \langle id, cap, \\ fixed : \text{Option ResourceId}, \\ group : \text{Option } (\mathbb{N} \times \text{UnitRel}) \rangle.$$

Requirements are generated from the physical sinks of the previous section by a **device kind**: an H-bridge channel needs a PWM line and a digital output; an I2C sensor needs SDA and SCL on the same unit; a quadrature encoder needs two interrupt lines. The pipeline is

$$\text{OutputId} \rightarrow \text{DeviceKind} \rightarrow \text{Requirements} \rightarrow \text{solve} \rightarrow \text{Assignment},$$

and the sink never enumerates pins, so swapping the board is re-solving the same requirements with the design untouched. Requirement identity is independent of sink and declaration identity: one sink generates several requirements, and two identical PWM needs must be two distinct variables.

An **assignment** is a list of requirement–resource pairs. It is **partially valid** when every entry is supported — the resource has the capability and any fixed choice is respected — and every pair is **compatible**: two entries on the same resource must have the same capability and it must be shareable, and two entries in the same group must have equal units under *same* and different units under *distinct*. It is **valid for** a requirement list when partially valid and covering exactly that list; the target is **satisfiable** when such an assignment exists.

A sound and complete solver

Every constraint is unary or binary, so validity is prefix-closed, and an exhaustive depth-first search that prunes on unary support and pairwise compatibility with the current prefix is complete as well as sound. Both are proved: if the solver returns an assignment it is valid for the requirements, and if a valid assignment exists the solver returns one. Feasibility of a finite instance is therefore decidable, and the solver runs inside the proof checker by decide. The instance is small enough that a general constraint solver [8] is unnecessary; the claim is about this scope, not about embedded allocation in general.

Extension of a target — adding capabilities, units, or sharing to existing resources — preserves every valid assignment. Removing a resource, adding or strengthening a requirement, or fixing a pin may not.

Case study: an Arduino Nano

The Arduino Nano’s digital and analog pins are encoded as a table: PWM on D3, D5, D6, D9, D10, and D11 backed by timers 2, 0, 0, 1, 1, and 2; external interrupts on D2 and D3; I2C on A4 and A5. A design with four H-bridge motor channels and one I2C inertial sensor is satisfiable, and the solver’s output is the assignment a tool would present:

```
M1 -> D3 / D0      M2 -> D5 / D1
M3 -> D6 / D2      M4 -> D9 / D4
IMU -> A4 / A5
```

Two counterexamples fix the boundary between kernel and validation. Seven independent PWM actuators pass every kernel condition — globally well formed, well clocked, causal, every drive edge well formed, single-driver, output-complete — and are unsatisfiable on the Nano, which has six PWM pins. The same requirements are satisfiable on a larger mock board with six more PWM pins on three more timers. Two interrupt lines and six PWM lines meet every capability *count* exactly, two and six, and are unsatisfiable, because D3 is both the only second interrupt pin and one of the six PWM pins. Capability counting is not feasibility. Further examples establish that two PWM requirements pinned to the same pin are rejected, that two I2C sensors on A4/A5 are accepted since allocation is not all-different, that a manual pin choice can turn a satisfiable design unsatisfiable while a consistent one is honoured, that four PWM lines required on independent timers are unsatisfiable on the Nano’s three timers though six PWM pins exist, and that TX and RX pinned to one UART unit stay together.

The explanation facility reports the first dead end under greedy placement. For the seven-actuator design it names the seventh actuator and, for each PWM pin, the actuator blocking it. This is *a* conflict under one placement order, not a minimal unsatisfiable core, and it is meaningful only when the solver has already returned no assignment.

Two kinds of evidence

Hardware feasibility is evidence about the pair (design, target), and it is not monotone in the sense the preservation theorems require. Six actuators are satisfiable; adding a seventh — a monotone extension of the design by a declaration and its sink — is not. The distinction drawn earlier between evidence that survives refinement and evidence that is rechecked after every change is here concrete. Commitments discharged compositionally survive EnvRefines; deployability on a target is re-solved; and the two are never merged. This is why the workspace reports them as different states.

What the layer does not model should be stated as plainly. Voltage, current, thermal budgets, memory, processor load, deadlines, bus bandwidth, torque, travel, and PWM frequency values are outside it. Some of these need summation constraints that are not binary, and while the architecture leaves room for a compatibility predicate over sets, nothing here establishes it. Units are one integer per capability per resource, which models which timer or which UART but not timer modes.

Deployment capacity

The third kind of validation evidence, added in Phase 9a, is deployment capacity: whether every bounded collection in the design has a bound sufficient for the target’s memory and the schedule. Like feasibility and unlike a commitment, it is not monotone — adding a fast source to a slow consumer can exceed a bound that was sufficient — and it is re-established after every change. Part V gives the formal result (`sufficient_capacity_preserves`, `periodic_capacity_sufficient`; **formally proved**) and production’s realization (`bdl-exec-ir::bounds`, `refusal at deploy`; ADR-0027). The workspace state *hardware-feasible* of Part II is, in production, the conjunction of allocation and capacity.

Two remarks across Parts III–VIII

Why the signature is a design object

The most consequential decision in BDL is treating a declared relationship as visible product intent. In programming, a signature is often documentation and a static contract around code. In BDL it can precede any code-like definition and remain useful on its own: $?f : \text{Tilt} \rightarrow \text{Brightness}$ says that the designer has committed to a causal design relationship and to its semantic boundary, and does not say how the mapping is computed. Clients are typed against the type view and depend on the interface only through it and through monotone evidence, so the partial commitment is not a weaker form of a complete one. It is the form on which everything downstream already rests.

Nominal identity in three places

BDL makes the same choice three times. On the axis of quantity, `sem` makes semantic identity nominal: two concepts of equal representation are distinct, and moving between them is a declared relationship. On the axis of time, `ClockId` makes temporal identity nominal: two domains of equal rate are distinct, and moving between them is a `sync` with an initial value. On the axis of effect, `OutputId` makes sink identity nominal: two sinks of equal accepted type are distinct, and driving one is an explicit edge. In each case the identity is independent of representation, of rate, and of type respectively; in each case the crossing is a visible artifact rather than a compiler action; and in each case the alternative — identity by representation, by rate, or by type — collided or was ambiguous for a mechanized reason. The three are not all placed alike. Semantic identity is in the type, while clock and sink identity are projections beside the interface checked by separate global judgments, because putting the clock in the type forces polymorphism on every pure mapping. Symmetry was not a design goal; it is what remained.

Part IX — Production Compiler and Runtime Architecture

KCN-judu/BDL is the engineering implementation of the language whose kernel was derived in BDL_FV. It builds what the formal development deliberately did not: the elaborator, the tooling and the execution path. It follows the formally developed semantics and is not itself formally verified. One property the whole system is built to keep obvious: *BDL semantics flows downward; implementation mechanisms never flow upward and redefine the language*. This Part describes the architecture at revision f1ce82c in enough detail that a tool paper could be extracted from it.

Four trust layers

layer	owns	never does
Flutter Studio	presentation, interaction, layout, ephemeral render state	compute type validity, semantic identity, dimensions, causality, clocks, output ownership, hardware feasibility, simulation
Rust compiler (bdtd and crates)	the canonical project model, every semantic judgment, diagnostics, simulation, allocation, code generation, the placement of entities that have no position yet	render, decide where a placed node goes
generated Rust core	deterministic executable behavior: domain step functions, state, output values	touch hardware, know about tasks or executors
platform adapter	physical I/O, clock activation sources, telemetry transport	interpret BDL semantics

Studio may render an edit optimistically, but the truth comes back from the compiler as a *projection*; Studio never holds a second copy of the language (ADR-0001). bdtd is a process boundary (ADR-0002), speaking protobuf over framed stdio (ADR-0007), with one canonical revision stream per opened project.

Crates and semantic ownership

The dependency direction is strict and acyclic: `model` → `ir` → {`syntax` → `elab`, `check` → `equations` → `elab`, `check` → `reactive` → `output`} → `compiler` → `ide-db` → `ide` → {`lsp`, `daemon`}; `protocol` sits between `compiler` and `daemon`; `text` and `layout` hang off `system` and are joined by `daemon`, `lsp` and the CLI; `hardware` depends on `model` only — it never sees Δ — and `compiler` joins the two; `lower` → `codegen` hang off `exec-ir`; `runtime-core` depends on nothing and is what generated code links against. A crate exists only where a real boundary exists.

crate	owns
bdl-model	stable ids · surface model · revisioned edits (apply_edit, pure) · persistence · quantity vocabulary
bdl-ir	the Design IR and the Reactive Core IR: the kernel's Ty, Expr, Prim, environments, transcribed
bdl-diagnostics	Diagnostic, Span, stable codes, deterministic order
bdl-syntax	Logos lexer · event parser (recursive descent + Pratt) · Rowan lossless CST · typed AST · lowering
bdl-equations	the equation library: rank-1 schemes with type and dimension variables and {Data, Eq, Ord}, first-order matching, one closed Core builder per equation
bdl-elab	concepts $\rightarrow \Theta$ · signatures $\rightarrow$ interfaces · formulas $\rightarrow$ Core with equations inlined at their instance · the unit registry and charts
bdl-check	Core typing · Grant · realization vs interface · pretty-printing; the authority for every type judgment
bdl-reactive	dependency graph · causality · Clocked · the reference evaluator · simulation · window capacity
bdl-output	DriveWF · SingleDriver · CompleteOutputs · output_values
bdl-hardware	capabilities · resources · hardware · device $\rightarrow$ requirements · boards · solve/diagnose
bdl-exec-ir	the executable IR: slots, first-order expressions, evaluation plan; interpreter; static list bounds
bdl-lower	reactive lowering: Design IR $\rightarrow$ Exec IR (clock, state, input and output slots; inlining; order)
bdl-codegen-rust	Exec IR $\rightarrow$ owned Rust AST $\rightarrow$ printed crate + host bridge + bdl-manifest.json
bdl-compiler	analyze(snapshot), analyze_deployment(snapshot, target), compile(snapshot, options), the collections report
bdl-system	components · instances · bindings · freshening · flatten $\rightarrow$ ProjectSnapshot + origins · packaging
bdl-text	persistence: source discovery · identity sidecar and reconciliation · load_workspace · item-level write-back · legacy migration
bdl-layout	deterministic, incremental placement of entities without a position; never semantics
bdl-library	concept libraries: data-driven templates instantiating ordinary concepts; search
bdl-ide-db	IDE ground state: IdeHost · overlays · EntityRef/EntityRole · projections · immutable stamped AnalysisSnapshot · cancellation
bdl-ide	semantic IDE queries over a snapshot: diagnostics, hover/explain, completion, references, rename, actions, invalidation preview, symbols, tokens, the Formula Composer queries
bdl-lsp	an LSP adapter only (ADR-0017)
bdl-protocol	protobuf schema · framing · conversions
bdl-daemon	bdld: session, coordinator, transport, analysis push, layout on open and commit; bdld check compile simulate
bdl-runtime-core	no_std vocabulary of every generated core: ActiveDomains, ClockSlot, RuntimeError, checked numerics; feature collections: list operators and the recursor over alloc::Vec
bdl-runtime-host	std harness: DynValue, JSON run request/trace over stdio, cargo driver

Planned and designed but not implemented: bdl-component (supplied Rust component contracts; “supplied Rust cannot drive outputs”, ADR-0005) and bdl-runtime-embassy (the first platform adapter).

The compiler as a pipeline of explicit passes

load/parse $\rightarrow$ identity resolution $\rightarrow$ signature resolution $\rightarrow$ surface elaboration
 $\rightarrow$ type checking $\rightarrow$ semantic-construction (grant) checking $\rightarrow$ dimension checking

→ dependency analysis → causality → clock domains → physical outputs
→ hardware requirement generation → hardware allocation → reactive lowering
→ Rust code generation

Each pass has an explicit input and output type and is pure where practical (docs/architecture/compiler-pipeline.md). Diagnostics are first-class outputs: an incomplete project is the normal case, never a fail-fast, which is the engineering face of the signature-first position. `apply_edit` classifies every operation as a *refinement* (dependents' established facts remain valid) or an *edit* (dependents must be re-validated) and reports an explicit Invalidation set — Interface | Realization | Semantic | Reactive | Clock | Output | Deployment — with the originating declarations (ADR-0009); incremental analysis subscribes to these categories, so the refinement-versus-edit distinction of Part III is an engineering asset, not folklore.

Numerics are IEEE f64 (ADR-0011), a recorded deviation from the kernel's Nat: division by an exact zero and any non-finite result fail the tick with a structured error, equality is exact, and the production numerics are not what the formal theorems are about (Part XI).

Formula elaboration (ADR-0013, extended by ADR-0025): a designer-facing formula language `— + - * / < <= > >= == != && || !, if, match` over Bool/Option/numbers/counts, `let`, calls, list and pair literals, rules `x => ...` as equation arguments, membership `x in [...]`, the slot `?` — parsed by a hand-written Pratt parser into a surface AST that is never reused as Core. Input names are the concepts' display names, re-resolved on every analysis; inputs appear as their representations (`rep (var i)`) and the whole formula is wrapped in `mk B` under the declaration's own grant — the elaborator never emits `mk` of any other concept, and the checker refuses one anyway. Units elaborate through the registry; equations are matched and inlined at the use; `bd1-check` re-derives the Core term's type and is the authority.

Executable IR and lowering

`bd1-exec-ir` is a first-order IR with dense slots: clock slots, state cells addressed by `StateCellId { decl, path }` (the declaration and the expression path of the `delay/sync` inside its realization, so identity survives unrelated edits), input slots for unresolved declarations, output plans with their driver. Lambdas and applications are inlined as `Let` bindings; the recursor becomes `Fold { elem, acc, step, init, list }` with one closure per recursor applied by the runtime — never a closure *value*; `rep/mk` become `Unwrap/Wrap` on a per-concept newtype. Lowering fixes an evaluation order from the plan; the reference evaluator's own tests show its result is independent of the host's processing order and the generated program's traces equal the reference's.

The generated core

```
pub const DESIGN: &str; pub const HAS_DOMAINS: bool; pub const CLOCK_COUNT: u16;
pub const CLOCK_k: ClockSlot;
pub struct SemN(pub Repr); // one per concept carried
pub struct Cells { pub cell_k: Option<T>, ... } // temporal state, None until first written
pub struct State { pub cells: Cells }
pub struct Inputs { pub decl_n: Option<T>, ... } // unresolved declarations
pub struct Values { pub decl_n: Option<T>, ... } // every declaration, None when not due
pub struct Outputs { pub output_n: Option<T>, ... }
pub struct Tick { pub values: Values, pub outputs: Outputs }
pub fn init() -> State;
pub fn step(state: &mut State, active: ActiveDomains, inputs: &Inputs) -> Result<Tick, RuntimeError>;
```

A program without lists is Copy, statically sized, laid out by the compiler: no graph, no map, no allocation, no traversal at runtime. Types: `q d` → f64 (the dimension is static and in the manifest), `bool`, `nat` → u64, `sem s` → `SemN`, `opt τ` → `Option<T>`, `list τ` → `Vec<T>` (last element first), `τ × σ` → `(T, S)`; a function type has no runtime representation. Symbols derive from stable ids (`decl_17`, `Sem3`, `cell_0`), never from display names. The core is `#![no_std]#![forbid(unsafe_code)]` and mentions no HAL, pin, peripheral or board; `cargo check --lib` of every corpus crate is a test.

Embedded execution model

The formal semantics is a tick-indexed relation; production gives it a deterministic step function per domain and states what the generated core must preserve (docs/spec/runtime-semantics.md):

- **Declarations are not tasks** (ADR-0004). Never one async task per declaration. Per clock domain, one deterministic `step(prev_state, inputs) → (next_state, outputs)`: hardware timer or interrupt → activate domain

$c \rightarrow \text{step} \rightarrow \text{publish } c\text{'s snapshot} \rightarrow \text{commit the physical outputs owned by } c$. Meaning never depends on executor task order.

- **Two phases.** *Read*: every declaration due this tick is evaluated (lazily, memoized) with temporal forms yielding their cell's committed value, or the initial value if the cell was never written. *Write*: every temporal site whose writing domain is active — the owner's domain for `delay`, `src` for `sync` — evaluates its operand in the same read mode into the *next* state. Nothing is updated in place; reads never see writes of the same tick.
- **Previous/next state.** `step` reads only `prev`, writes only `next`, and assigns at the end — not at all on error. Every `delay` carries its explicit initial value; a cell is `Option<T>`, `None` until first written; there is no implicit zero.
- **The sync snapshot rule.** `sync src init e` reads the last committed snapshot of `src` from an activation strictly before the current tick, `init` if there was none, and never invokes `step_src` recursively. When two domains are ready at the same instant, each observes only the other's previously committed activation — the scheduler's order is unobservable, exactly as in the formal model (`scheduling_order_observable` is the counterexample for the alternative). In generated code a `sync` cell's writer is the *source* domain's slot.
- **Output commit timing.** Outputs are built after the write phase from the driving declarations' values (`output_values`), and committed by the domain that owns the sink.
- **Allocator requirements.** A core that carries a list needs a global allocator (ADR-0024); the manifest records `requires_allocator`, per-cell bounds, `state_bytes_max` and `tick_bytes_max`; the first platform adapter is to declare an arena sized from the manifest and state the most it supplies for each list input.
- **Collection bounds.** Static, sound per declaration and cell (`bd1-exec-ir::bounds`); the window capacity model per crossing; refusal of an unbounded state on a bounded-memory target (ADR-0027, Part V).
- **Cost discipline.** Moves, not clones: a use analysis per expression tree moves a local referenced once in its scope, clones any other reference, and always clones a local captured by a fold's closure, so a fold step that conses onto its accumulator moves it — `map`, `filter`, `append`, `sum`, `any`, `all`, contains clone nothing per element (clone-counting tests; measured $\times 15$ for 2 000 $\rightarrow$ 32 000 elements). A total conditional — neither branch can fail — is emitted as a Rust `if`, observationally the strict `ite`; a branch that can fail keeps `prim::ite` and its strictness. Borrowed reads for `length`, `head`, `take`, `==`, `<`. Commit-only writes: a tick clones no cell it does not read. What remains: `zip` clones its accumulator pair per element (ISS-0013); no fusion of `map` $\rightarrow$ `filter` chains, because each is linear and measured and a fused emission was not justified by the numbers.
- **The realization boundary.** A source (an unresolved unit-domain declaration) is an *input slot* of `step`, filled by the adapter from a sensor, a bus or a simulation trace; a driven output is an *output field* of the step's result, committed by the adapter to GPIO or PWM. Neither is a function call inside the core: a zero-input relationship compiles to a zero-argument accessor of the committed value (its unit argument erased, ADR-0029), and a sink is a field, not an `A -> ()` callback (Part VIII). The core therefore has no device vocabulary at all.
- **Targets.** The core is target-independent. macOS and Windows are first-class hosts for the toolchain; no platform adapter exists at `3c6c8be` — Embassy is roadmap priority 1; an RP2040 board file is not yet present.

Current cost figures are recorded, not optimized: a lamp core is ~95 lines and ~7 KB of source with a zero-byte State; the collections and buffer corpus cases allocate and their debug-build timings are two orders of magnitude above the allocation-free cases — a baseline for later evaluation, nothing more.

Differential testing

For the same design, schedule and input trace, the reference evaluator and the generated program must agree value by value, tick by tick (`crates/bdl-compiler/tests/backend/*.rs`). The corpus: lamp and lamp-with-output, pure arithmetic with dimensions and a count, collections (`any`, `sum`, `map`, `filter`, `zip`, `contains`, `clamp`, `getOrDefault/head`, structural equality, a list in a state cell), buffer (the Phase-9a window as five declarations), `bounded_buffer` and `overflowing_buffer`, semantic `rep/mk` with a Boolean concept, `booleans/comparisons/strict if/options`, `delay`, a cycle broken by `delay`, `sync` across two domains in both directions with the slow domain on period 2, a domain-agnostic declaration shared by two domains, division by zero, non-finite result, missing input, a design with no domain — 22 cases, each generated, cargo checked as a `no_std` library, built with its bridge, run and compared, with golden files pinning the generated bytes and a determinism check that generating twice gives identical output; a 3 000-tick run; property-based generation of designs (250 per run in process, a seeded batch of 8 through the full toolchain); and the storage-order audit of the reversed `Vec`. Comparison is exact (f64 bit-for-bit in effect; the generated code performs the same IEEE operations in the same order — no reassociation, fusion or CSE — and the JSON transport is exact); errors

must fail at the same tick with a failure the reference could report, computed by starting the reference’s traversal at every due declaration.

Project persistence and the daemon

A project is one directory (ADR-0023, superseding ADR-0020’s third project *kind*): `bd1.toml` (name, schema, compiler version — no kind), `src/**/*.*bd1` (the authored design and system — the semantic source), `.bd1/identities.json` (source key $\rightarrow$ stable id, allocators, the flat-id freshening table — tool-owned), `.bd1/authoring.json` (behavior groups by identity), `ui/layout.json` (positions, viewports, group boxes — presentation). “Source of truth” is a per-fact statement: the semantic model never depends on geometry, the graph never stores a semantic fact, the text never stores a position; erasing both sidecars and the layout changes no semantic fact; erasing the sources loses the design. A legacy JSON project is migrated in place the first time it is opened, identities and layout kept.

`bd1d`’s coordinator is the single imperative loop: it owns the `Session`, applies edits serially through the pure `apply_edit`, and hands immutable snapshots to analyses. Every semantic edit is sent against the revision Studio holds and refused if the project has moved on; every response and event carries its revision; revisions are strictly monotone (undo produces a new revision), so staleness is a $<$ comparison. Overlays — a definition draft per mapping, a text document per file — never create a revision; `AnalysisSnapshot` composes committed + overlays, runs `analyze`, and is stamped (revision, generation); a query holding an old snapshot keeps seeing the world it started in.

The elaboration passes the kernel implies

The surface editor and kernel are connected by an elaboration function from surface designs to kernel environments with diagnostics. These passes were stated in the conference manuscript before any elaborator existed; the production pipeline above realizes them — resolution and formula elaboration in `bd1-elab`, temporal lowering and domain assignment in `bd1-lower` and `bd1-reactive`, output binding in `bd1-output`, hardware validation in `bd1-hardware`, normalization and erasure in `bd1-exec-ir` and `bd1-codegen-rust`. The list is kept because it is the specification those crates were written to, and because it says what each pass may and may not consult.

Name and signature resolution. Resolve semantic properties to identities, Mapping signatures to declaration interfaces, contexts, device kinds, units, and imported components. At this stage an unresolved `Tilt` $\rightarrow$ `Brightness` mapping already has a stable identity and an expected type, and every reference to it is typed.

Formula elaboration. For each Mapping with a definition, check the definition against the declared codomain under the grant of the declaration’s own signature. A scalar formula attached to a `Brightness` output elaborates to `mkBrightness(...)` because the surrounding signature announces the concept; the constructor is inserted by the elaborator and is legal only there. Units elaborate to scaled dimensioned literals. Curve, example, and component definitions elaborate to the same realization form.

Temporal lowering. Surface temporal phrases lower to the declaration shapes of the derived-operator table. A context lowers to an activation declaration, an entry declaration, gated and reset local state, and a conditional in each output’s driver. A reusable stateful component is instantiated into fresh declarations at each use, because temporal state cannot occur under a binder.

Domain assignment. Domains are declared, so this pass records the clock environment and checks the domain judgment. A reference across domains without a transport is reported at the reference, in the designer’s names for the two domains, with the two legitimate resolutions — transport the value with a stated initial value, or move the reader to the source’s domain — stated in terms of their behavioral consequence.

Output binding. Record the drive edges and check type and clock equality, single-driver, and, for executable designs, completeness. Where several contexts would drive one output, the elaborator emits one driver whose body selects among them; the selection is visible in the elaborated design.

Hardware validation. Generate requirements from device kinds, run the solver against the selected target, and attach the assignment or the explanation to the bindings.

Normalization and erasure. After checking is complete, semantic identities, dimensions, and domains carry no computational content and may be erased; erasure is proved sound for identities and for dimensions. Nominal wrappers introduced by elaboration cancel, $\text{rep}(\text{mk}_s\ e) \rightsquigarrow e$, and the flattened program is checked under the

universal grant because every construction was authorized at its declaration. Unfolding all realizations into a closed term preserves typing on the delay-free fragment and agrees with tick-by-tick evaluation on first-order designs; the higher-order case holds up to closure equivalence and was not formalized. Erasure is applied selectively at boundaries the checker cannot see through — supplied blocks, device bindings, the public interface of generated code — where wrappers are retained so that the host compiler continues to check what BDL cannot.

The kernel never depends on normalization to decide type equality; it is not dependently typed, and normalization is an analysis and code-generation instrument. Floating-point arithmetic is not associative, so any symbolic normalization over the reals must record the resulting numerical deviation as an obligation rather than silently altering the property being checked.

Part X — Studio: The User Interface as a Semantic Projection

Studio is not a drawing tool with a compiler attached; it is a set of projections of the compiler’s verdicts, arranged so that semantic distinctions become interaction distinctions and nothing else does. This Part describes the intended interaction model as designed and, at f1ce82c, largely built (docs/architecture/studio-ui.md; the implementation status is in Part XI). Whether the model is *usable* is an empirical question no study has yet answered (Part XIII).

Three information levels

Every semantic fact is designed for exactly one primary level and may echo at the next only as detail behind the first (ADR-0018):

level	answers	may use	may not use
1 Canvas	what does the product do; what depends on what; what is still open; where does behavior become physical	object silhouette, socket shape, socket hue, links, grouping, containment, line style, the object’s own state; one state word where unavoidable	type labels, ids, compiler words, badges, counts
2 Inspector	what does the selected object mean; what can I change; what will the change affect	designer vocabulary: <i>Meaning</i> , <i>Value</i> , <i>Unit</i> , <i>Reads</i> , <i>Produces</i> , <i>Relationship</i> , <i>Used by</i> , <i>affects</i> , <i>checked again</i> ; diagnostics in product language attached to the field they concern	SemanticId, DeclId, Ty, Grant, realization, Clocked, SingleDriver, solver, protocol, revision, enum names
3 Explain	why was this accepted or refused; what did the surface form elaborate into; which rule applies	all of the above, kernel notation, Core IR, diagnostic codes, revision	—

Explain is one collapsed disclosure at the end of the inspector, never open by default; nothing in levels 1–2 depends on it. This is the progressive-disclosure claim of Part I made operational: the kernel is reachable, and it is never in the way.

The canvas

Socket hue = semantic identity. In a node editor a socket’s colour is its data type; in BDL the type that matters is the nominal concept, so each concept gets a stable hue derived from its `SemanticId` — deterministic, never from the name, with lightness chosen per hue so every identity clears 3:1 contrast in both appearances. A link is accepted only between sockets of the same concept: while a link is dragged every compatible socket gains a faint halo, the one under the pointer a strong halo, and an incompatible socket shows the forbidden cursor. The canvas shows the nominal typing rule without a diagnostic. Tilt and `MotorAngle`, both angles, have two hues and never connect.

Socket shape = value form. Quantity ○, on-off ◇, count □, collection  (a stack), grouped value  (a split square), optional value ◯ (a ring with a hole); a concept whose value form is not yet chosen is a hollow ring. The same glyph, drawn by the same code, appears in the library, in chips, toggles and pop-ups. No type words are written beside a socket anywhere.

Unresolved state. A declared mapping — realization none — is drawn with a dashed outline, an empty definition region and the one word *declared*; dashed survives selection; never red. A definition the compiler cannot accept gets a red mark at the definition line, where the problem lives, and no word in the header. A mapping waiting on a concept whose value form is not chosen is a solid node whose read socket is hollow, with the inspector saying *checked once Temperature’s value is decided*. These are the three states of Part III’s signature-first model, drawn.

What the canvas never means. Edges are dependency, not execution order; drawing order does not set output priority; node position is layout only (ADR-0003) — an entity without a position is placed by the daemon’s layout service on open and on commit, deterministically, without moving anything placed, and Studio arranges nothing at render time. No node type exists per arithmetic operator: formulas live in the inspector. Dragging a node over a link does not auto-insert it, because links are typed by concept and silent insertion would be a semantic edit.

Header colour = category: concept, mapping, context, output, transport — muted; identity hues are the only saturated marks on the canvas. Interaction follows Blender’s node editor conventions; a concept in use cannot be deleted without the banner naming its dependents.

Groups and components on one canvas

A behavior system has three zoom levels on one canvas: the system of instances, an instance’s body, and the flat design. A group is a box that can be collapsed to its aggregate sockets (crossIn ++ openMembers in, crossOut out — Part VI), moved, merged and split with no revision and no re-check; *Package as Component* opens a sheet that shows the inferred required, provided and private members and lets the designer widen but not narrow them. An instance node shows its required and provided ports with the same socket glyphs; a component’s body is edited in its own scope with its own drafts and completion. The packaging sheet teaches by showing: the boundary the designer sees on the collapsed group is exactly the interface the component receives.

Design, Code and Split: views of one project

The same open project is shown as a graphical behavior model, as its source files, or both side by side (ADR-0023). Switching is a view change — no conversion, import, export or project type. Graph → text: a canvas operation is a semantic edit on the model, and the textual projection is the minimal item-level splice of the sources, keeping comments and formatting outside the touched item; computed for the Code view on request and written on save. Text → graph: a Code-view edit is the whole text of one file against a revision; if it builds, declarations are bound to their identities by reconciliation (retained, allocated, dropped, renamed, ambiguous, duplicate) and the project moves to a new revision; if not, the committed project stays and the draft is held with the loader’s faults — malformed text never erases the graph. An entity authored first in Code and one authored first in Design are indistinguishable once synchronised. The Code pane shows an out-of-step banner with the daemon’s reasons and syncs selection in Split.

Ownership boundaries, stated once and relied on everywhere: source semantics in `src/**/*.bdl`; identity in `.bdl/identities.json`; authoring metadata (groups) in `.bdl/authoring.json`; layout in `ui/layout.json`; viewport and session state in the running Studio only. Studio state is three things kept apart — semantic projection (owned by the compiler, only rendered), editor state (selection, open inspector, pending requests), rendering state (drag position, hover, zoom, never persisted or sent).

The definition editor and the Formula Composer

The definition editor has two projections of one draft, chosen with a **Formula | Text** control (ADR-0028). The Text view is a field over the compiler’s verdict, with completion, hover and fixes from the IDE service. The Formula view draws the compiler’s `FormulaProjection` as the expression it is — `clamp( [Tilt] ÷ [90][deg ▼], [0], [1] )` — never as a graph inside the node.

Architecture. The draft text of a mapping’s definition is an overlay in the IDE host (`MappingDefinitionDraft`), never a revision; `bdl-ide::formula` builds the projection from the *effective* definition text on every request and never stores it:

```
FormulaProjection    formula_projection(&snapshot, mapping)
                    = trace_formula_in (bdl-elab: the surface tree and the type of every
                      sub-expression, on success and on failure)
                    → FormulaNode tree: tree-path ids, byte ranges, kinds, actual types,
                      expected types by local dimension inference (solve),
                      diagnostics placed on the innermost node, the slots in source order

SlotInfo             formula_slot(&snapshot, mapping, node)
                    = the projection's expectation at `node` + units_for(dim)
                      + reference candidates by type + equations by scheme match and capability

ComposeResult        compose(&snapshot, mapping, source, ComposeOp)
                    = the text a structured action makes: byte-range edits of the draft,
                      parenthesised by precedence, a unit switched with the value kept
```

`FormulaTrace` is the elaborator’s typing trace, kept on success and on failure, so a formula that does not check still has a projection for the parts that do. A `FormulaNode` carries its *actual* type (what the sub-expression is) and its *expected* type (what its position demands, by `solve`), and node identity is the tree path — ephemeral, stable within one draft generation, not an entity, never in `.bdl/identities.json`. `bdld` carries the projection with every

DefinitionDraftAnalysis (one round trip per keystroke, the same generation) and answers GetFormulaProjection (the committed definition, no overlay), GetFormulaSlot and ComposeFormula (protocol 0.12).

Every structured action is a text edit. compose answers a ComposeOp — fill a slot, wrap in an operator, call an equation with the component as first argument, set a literal’s unit, set a literal’s coordinate, remove — with byte-range edits of the draft and the whole new source; Studio puts it into the draft through the ordinary DefinitionDraftChanged, and the ordinary analysis follows. One commit path, one conflict model, one set of diagnostics. Studio stores the mode, the selection and the open pop-up — never a parsed tree, an inferred dimension or a unit rule; building the tree in Dart was rejected because precedence, parenthesization and unit conversion would move into Studio (ADR-0001).

Encodings. A *slot* is a dashed hollow chip (dashed = not decided, as the canvas’s *declared* node); a *reference* is a chip with its concept’s socket glyph; a *literal* is two fields, the coordinate and a unit pop-up listing the compiler’s units for the literal’s own dimension, where the pop-up switches the unit and keeps the quantity and the coordinate field makes a new quantity in the same unit — only a literal has a unit pop-up (Part VII); *operators* are their glyphs, a *call* its name and parentheses, an *unsupported form* (if, match, a block, a rule, a collection or grouped literal, delay/sync) its text in monospace, selectable and edited as text. A *finding* is a red underline on the component *and* its row under the field — one diagnostic, two projections. Beneath the field, for the selected component: the compiler’s sentence — *Expected: an angle, because an angle ÷ an angle = a dimensionless quantity* — with the kernel’s notation behind Explain; for a slot, a number entry whose unit pop-up holds the units of the expected dimension (none for a dimensionless slot; none, with a sentence, when the position is not determined), then references by type and equations folded; for a component, + − × ÷, Compare, Function (the equations whose result fits, wrapping the component as first argument) and Remove.

Stale projection policy. A projection is current only when it is of exactly the text on screen and that text parsed; otherwise the field is dimmed with a notice — *Waiting for the compiler to read the formula...* while the verdict for this text is on its way, *The text cannot be read as a formula.* when it never will be, and then no tree is shown because none is invented — no component answers a click, no slot panel opens, no key acts, the reducer refuses a structured action (or a second one in flight) and discards an answer for text that has moved on; **Edit as text** is the way out. Generation safety: every answer carries the draft generation it was computed for and is discarded if the draft has moved. A formula with a slot may be saved and is *invalid* until filled; commit, revert, conflict and detach are the Text view’s, unchanged.

A walkthrough. The designer selects `dimByTilt : Tilt → Brightness` and chooses `Function → clamp`; the draft becomes `clamp(?, ?, ?)`, three dashed slots. Selecting the first, the slot panel says *Expected: a dimensionless quantity* — because `clamp`’s scheme unifies all three arguments with the result and the result is `Brightness`’s representation — and lists references of that kind; the designer chooses `tilt`, and the compiler answers *This is an angle; a dimensionless quantity is expected*, red on the chip. Wrapping it in `÷` yields `clamp(tilt ÷ ?, ?, ?)`; the new slot’s panel now says *Expected: an angle, because an angle ÷ an angle = a dimensionless quantity* and offers `rad`, `deg`, `turn`; the designer types 90 and picks `deg`. Filling 0 and 1 completes `clamp(tilt / 90 deg, 0, 1)`, which checks, and the node’s red mark goes. Switching the `deg` pop-up to `rad` rewrites the literal to `1.5707963267948966 rad` and nothing else. Every step was a byte-range edit of one text; the Text view shows the same characters throughout.

Partial expressions: ?

The slot deserves its own record because it is the clearest example in BDL of authoring semantics distinct from runtime semantics. `?` is a `SlotExpr` of the textual grammar: it parses wherever a value may stand and formats as itself; it is never a value, an operator or a unit (`1 ? 2` is a syntax error). It exists in the *surface* because an incomplete structured formula must be ordinary draft text — the same state as any unfinished draft, savable, visible in the Text view, subject to the same conflict model — rather than a second representation held in an overlay of partial expressions (the rejected alternative, which would have needed its own persistence and its own protocol of operations). Elaboration refuses it with `formula.slot.empty` — *This slot is empty; it expects an angle* when the context says what it expects — and it never reaches Core, the checker, the evaluator or generated code; a kernel hole term was rejected because the kernel has no notion of an unfinished expression and needs none. Expected information flows *into* the slot from the formal Composer’s solve: the operator above it and the known sibling determine its dimension; a product of two slots is *insufficient information* — the formal model reports it unsolved rather than searching, and Studio shows the position as not determined with a sentence. That two-hole case is not a limitation to be engineered away: local,

deterministic inference is what makes the expectation explainable in one sentence, and a global solver would trade that for guesses.

Simulate, Deploy, Library, Monitor

Simulate runs the reference evaluator over structured inputs per tick (including list and pair values, and concept values in the design's terms — `Brightness(0.5)`), showing every declaration's value and every output; the simulator does not get a separate interpretation. *Deploy* is the target-relative read model (ADR-0015): board selection reruns deployment analysis only, and the workspace reports validity and deployability in different places; the readiness matrix (bounded-memory refusal, unbounded state, window capacity) is shown here. *Library* instantiates concept templates as ordinary concepts with fresh ids (create-then-rename). *Monitor* — telemetry from a deployed core — is designed and not built, as is the platform adapter it needs.

What Studio does not decide

Studio computes no type validity, semantic identity, dimension, causality, clock, output ownership, hardware feasibility or simulation; it holds no parsed formula tree and no unit rule; it arranges nothing at render time. Every one of those is a projection from the daemon, keyed by `EntityRef` and role, never by text position or node id. That is the discipline that lets the same facts appear as socket hues, inspector sentences and Explain notation without three implementations of the language.

Part XI — Formal-to-Production Correspondence and Deviations

The formal development is a specification (ADR-0010): production transcribes it and discharges it by tests; a theorem never proves the Rust or Dart code. This Part states, per area, what production follows and how strongly, and then lists every known deviation with its reason, its risk, the evidence that bounds it, and whether closing it is planned.

Correspondence by area

area	formal object	production realization	strength
declarations, interfaces, refinement	DeclEnv, DeclInterface, EnvRefines, local_refinement_preserves_global_refinement	bdl-model snapshot and apply_edit with refinement/proof classification and Invalidation sets	informed by FV; production-tested (edit classification tests)
typing through the type view	HasType, infer_sound/complete	bdl-check over bdl-ir (transcribed Ty/Expr/Prim)	transcribed; production-tested
nominal concepts, representation, grant	sem, 0, rep/mk, Grant.of, constructs_granted	elaborator inserts rep on inputs and one mk under the declaration's grant; checker refuses others	transcribed; production-tested
dimensions	Prim.ty dimension algebra	bdl-model::Dim (eight bases) and primitive typing	transcribed; product-language diagnostics tested
reactive semantics	Ev, Ev.det, reactive_total, Causal	bdl-reactive reference evaluator, two-phase tick, StateCellId	transcribed; the evaluator is the executable definition
clock domains	MEv, Clocked, strictly-before, single_domain_embedding	Clocked pass; sync snapshot rule; step_in_order test	transcribed; production-tested
outputs	DriveWF, SingleDriver, CompleteOutputs	bdl-output	transcribed; production-tested
hardware validation	solve_sound, solve_complete, diagnose	bdl-hardware, board files, target-relative deployment analysis	transcribed; production-tested
behavior systems	Theorems A–I, J (restricted)	bdl-system: freshening from the project allocator, bindings, flatten with origins	informed by FV; production supports transported bindings and higher-order bodies beyond the theorem fragment, production-tested
groups	Theorems A–R	.bdl/authoring.json, no invalidation on group edits, packaging sheet	informed by FV; production-tested
list data, buffer, capacity	Ty.list, Theorem M, Capacity.lean	Vec last-element-first, take cap bounds, bdl-reactive::capacity, bdl-exec-ir::bounds, refusal	informed by FV; production-tested (corpus buffer, bounded_buffer, overflowing_buffer); the static bound is an engineering analysis
products, fold, eq, lt	Phase 9b/9c kernel	bdl-ir gains exactly these; Fold in exec IR with one runtime closure	transcribed; production-tested
equation library, schemes	Stdlib, Poly, Generic	bdl-equations: schemes with {Data, Eq, Ord}, matching, one Core builder per instance, inlined at the use	informed by FV (ADR-0025); production-tested
order by declaration	OrdDecl, Ty.ordB, Ordered	Concept::ordered, ordered concept; ADR-0026 mixed-case rule	informed by FV; the mixed case is a production decision beyond the formal model
units and charts	Units.lean, Charts.lean	UnitDef { id, symbol, dim, chart }, Chart::{Linear, Affine}, convert	informed by FV; property-tested on f64 within ulps
Composer inference	Composer.lean solve, candidates	bdl-idel::formula projection, slot, compose	informed by FV; production-tested (formula_composer.rs)
natural binder syntax	Natural.lean	bdl-elab::formula::binder (P11, protocol 0.13)	transcribed; production-tested (natural_forms_lower_to_the_same_core_as_the_call_f
Source provision by device profiles	Provision.lean provision_envRefines, provision_wf, provision_causal, provision_wellClocked, provision_transparent, provision_abstracts,	not implemented (PRP-0001, draft)	formal guidance; the proposal revised from the audit
unit-domain canonical type	UnitDomainCanonicalType, UnitDomainCanonicalType.refl, UnitDomainCanonicalType.agree, UnitDomainCanonicalType.source_value	bdl_ir::Ty::Unit, Ty::of_signature, Ty::kernel_of_signature, Ty::canonical_mapping_ty, Signature::is_unit_domain (ADR-0029, protocol 0.14)	formally proved at the interface; transcribed at the kernel encoding; production-tested (...the_encodings_are_inverse,..._argument_is_erased)

Deviations

deviation	why	risk	evidence that bounds it	closing the gap
kernel <code>Nat</code> magnitudes vs production <code>f64</code>	proofs need decidable, saturating naturals; products measure reals	truncation in the kernel's executed examples is not production's rounding; <code>NaN/∞</code> and division by zero have no kernel counterpart	production fails the tick on exact-zero division and non-finite results; exact comparison; the formal theorems are about structure, not rounding	not planned to close; <code>f32</code> on device is a further recorded deviation (ISS-0006)
exact symbolic unit scales (<code>Sym</code> , π as a generator; exact <code>Q</code> charts) vs approximate floats	$\pi/180$ is not rational; exactness is what the formal laws are about	round trips and compositions hold only within ulps in production	toleranced property tests of identity, composition, inverse, display switch and difference law; exact-rational oracles for registered charts	not planned; the contract is stated and tested
kernel <code>Nat</code> registry has no radian; production is radian-canonical	no integer scale for radians against degrees	none semantic; the registries differ in which units are exact	documented per registry	none needed
theorem-level semantics for direct bindings on wiring designs vs production's transported bindings and higher-order bodies in systems	Theorem J's obstacle: a domain-indexed input for a transported port	production behavior beyond the fragment is tested, not proved	flattening plus the reference evaluator; corpus systems	future: Theorem J under MEV with a domain-indexed modular input
formal buffer model (unbounded, capacity as a validation predicate) vs production static list bounds and bounded-memory refusal	the kernel has no capacity; production must decide deployability	the static bound analysis has no theorem behind it	differential tests of <code>bounded_buffer</code> vs <code>buffer</code> at 7 and 3 000 ticks; <code>overflowing_buffer</code>	future: a proved bound analysis, or a surface window form (ISS-0001)
formal <code>Ev/MEV</code> vs generated Rust	the core lowers representation (slots, structs, <code>f64</code> , inlined lambdas)	a lowering bug would be a semantic bug	22-case differential corpus, golden files, determinism, property-based designs, error-tick agreement	future: a generated-code refinement proof (Part XIII)
abstract evidence conditions (<code>Monotone</code> , <code>Equivariant</code> , <code>PortSound</code> , <code>InterfaceLocal</code>) vs production's concrete evidence	the formal model is parametric in evidence	a production evidence model could violate a condition unnoticed	commitments are not yet a production feature beyond typing; nothing to violate today	future: a concrete compositional evidence model
mixed comparison rule (ADR-0026)	the formal order policy is between concept values only; production had to decide the concept-beside-plain-value case	<code>min(o1, 0.5)</code> accepted while <code>min(o1, o2)</code> refused	a tested matrix; Explain shows the observation	recorded as intended
affine charts hidden from formulas	conversion is safe; point/difference validation does not exist	a designer cannot write 20°C	ISS-0004 open	future: the point/difference validation, or a decision not to need it
<code>bd_l_ir::Ty::Unit</code> exists; the formal kernel <code>Ty</code> has no unit	production wants one type language for the interface and the kernel encoding; the formal model keeps the canonical types in a separate <code>CTy</code> above the kernel	none semantically: <code>Ty::Unit</code> never types a Core term, a representation or a runtime value, which is exactly <code>elim unit = none</code>	<code>elim_canonical</code> ; the production tests that the encodings are inverse and the argument is erased	not planned: the formal <code>CTy</code> is the record that <code>Unit</code> is an interface type
simulation inputs are unresolved <i>unit-domain</i> declarations; the kernel <code>Input</code> provides for every unresolved declaration	an environment provides values, not functions	none: on a unit-domain source the two agree (<code>SimulationInput.value</code>)	<code>source_value</code> , <code>resolved_not_source</code>	not planned; recorded as surface policy

Implementation status snapshot (production 876005c, 2026-09-20)

Implemented and exercised by named tests: the language core through outputs and completeness; formula language v0 with the slot; the unit registry with linear charts offered and affine charts as infrastructure; the data core; the equation library; the compiler and simulator with product-language diagnostics and Explain; the Rust backend with differential, golden and property tests; collections at deployment; hardware and deployment analysis with Nano and a larger board file; behavior systems, groups, packaging, versions and substitution; the unified project format with migration; the layout service; text $\leftrightarrow$ graph synchronisation; the IDE service and LSP with a VS Code extension; the protocol and daemon at 0.14 (0.13 added the binder and range nodes, 0.14 the unit kind); the natural forms (P11); one canonical type per relationship with the empty product as its unit domain (ADR-0029), with $() \rightarrow A$ the preferred spelling and the shorthand deprecated with a quick fix and a lossless migration; complete-project persistence (ADR-0030); a first internationalization layer with locale as presentation only (ADR-0031); the Source as a derived presentation role on the canvas, inspector, hover and Explain, consuming Phase 12 (ADR-0032, ISS-0014 resolved, ISS-0016 opened for a device binding for a Source); the generalized Standard Library with Sources as items (protocol 0.17); canvas reference edges (ADR-0034, protocol 0.18); PRP-0001, the Source-provision proposal Part VIII audits (draft, not implemented). Partial: Studio (the Formula view's remaining pieces are listed in production's status page). Planned, designed and not implemented: any platform adapter, build orchestration, flash, telemetry, supplied Rust components. Not implemented by decision: user enums (open, ISS-0005), temporal modifiers and contexts in the surface (ISS-0010), a surface form for occurrence windows (ISS-0001), record syntax, forall/exists sugar, Set/interval/sum types; an $A \rightarrow ()$ consumer form $()$ is refused in output position, in agreement with Phase 12).

Part XII — Rejected Alternatives and the Minimality Ledger

The kernel was obtained by a method, and the method is part of the record. Each phase of the formal development took a family of candidate constructs — from the earlier draft of the language, from the production roadmap, or from the standard repertoire of typed functional languages — formalized the smallest plausible version and its alternatives in Lean 4 without external libraries, and asked the same questions of each: What does it reject that the others accept? What does it accept that it should not? Is it a special case of another? Which operations on a design are refinements under it? A construct entered the kernel only when every tested alternative failed for a stated reason, recorded as a theorem or an executable counterexample. Verdicts use one vocabulary: KEEP IN KERNEL, KEEP IN SURFACE-DESUGAR, KEEP IN STANDARD LIBRARY, MOVE TO VALIDATION, MOVE TO AUTHORING/UI, DEFER, REMOVE. The full construct-by-construct table is `MINIMALITY.md` in the formal repository; the decisions are `DESIGN_DECISIONS.md` (D-1 ... D-114). This Part records the major rejections with candidate, reason considered, counterexample or theorem, and verdict.

Kernel constructs retained

construct	why it could not be derived	evidence
<code>DeclId</code> and frozen expected types	references resolve by identity; retyping invalidates clients	probes 1–6, <code>local_refinement_preserves_global_wf</code>
nominal <code>sem s</code> with <code>θ</code> , <code>rep</code> , <code>mk</code> under a grant	the numeric baseline accepts the invalid wire; a metadata checker is η-evaded; concept-as-declaration has category errors	Phase 2/3 Models A–C, <code>constructs_granted</code> , <code>temporal_state_preserves_semantic_identity</code>
<code>q d</code> with the algebra in <code>Prim.ty</code>	erasure accepts <code>length + time</code>	<code>dimension_mismatch_rejected</code> , <code>counterexampleB_baseline_accepts_length_plus_time</code>
<code>delay</code> , <code>sync</code> (one temporal read)	totality forces data-typed, top-level state; <code>delay</code> is <code>sync</code> at the own domain	<code>reactive_total</code> , <code>delay_is_sync_own</code> , <code>scheduling_order_observable</code>
<code>ClockId</code> , <code>Clocked</code>	a direct wire is ambiguous without it; equal rate is not the same domain	<code>equal_rate_not_same_domain</code>
<code>OutputId</code> , drive edge, <code>SingleDriver</code> , <code>CompleteOutputs</code>	two drivers make an output non-functional; hidden policy is observable	<code>multiple_direct_drivers_rejected</code> , <code>hidden_arbitration_observable</code>
<code>list τ</code> and its operators	a lossless window is unbounded sequence data	<code>bounded_summary_not_lossless</code> , Theorem M
<code>prod</code> , <code>pair</code> , <code>fst</code> , <code>snd</code>	function encodings are not data and cannot be state	<code>arrow_not_delayable</code> , <code>church_fst_rank</code>
<code>fold</code>	one eliminator derives every collection operation; operators never apply closures	<code>fold_total</code> , the <code>*_spec</code> family
<code>eq</code> at every data type (proof field)	boolean/concept/pair/list equality was unwritable	<code>Value.beq_iff</code>
<code>lt</code> on quantities	ordering is a property of magnitudes	<code>lt_rejected</code> , <code>lt_only_on_quantities</code>

Rejected constructs

candidate	reason it was considered	counterexample or theorem	verdict
Signal τ in Ty	the earlier draft's reactive types	inhabited by exactly the terms of τ ; rejects nothing	REMOVE
Event τ (single domain)	occurrences as a type	opt τ streams are the multiplicity- ≤ 1 streams	REMOVE
Event τ (across domains), a buffer primitive	multiplicity across rates	the window is five declarations over delay/sync/lists; Theorem M	REMOVE (surface desugar)
effect rows	output effects in types	duplicate the drive edge or false-positive	REMOVE
action requests / action values	outputs as values	relocate the conflict without resolving it	REMOVE
runtime output arbitration	several drivers	hidden_arbitration_observable: the hidden policy is observable	REMOVE (single driver)
clock-indexed types <code>Signal[c, τ]</code>	domain in the type	clocked_type_forces_polymorphism on every pure mapping	REMOVE
same-tick cross-domain visibility	"synchronous sub-domains"	scheduling_order_observable: the scheduler order becomes observable	REMOVE (strictly-before)
scheduler / staging rule	ordering simultaneous domains	unnecessary under strictly-before	REMOVE
explicit machine state / Step	a state machine object	Phase 4/8: state is gated self-delayed declarations; components hold state by instantiation	REMOVE
StateHandler kernel	contexts as a construct	elaborates to activation, entry, gated and reset state, conditional drivers (tested behavior only)	KEEP IN SURFACE-DESUGAR; nesting/handler-scoped clocks untested
state identity as a kernel notion	naming state	StateCellId by declaration and expression path is an engineering device; the kernel has none	REMOVE
user-defined typeclasses / dictionaries	constrained generics	closed {Data, Eq, Ord} covers the library; comparators recover the rest (minBy_recovers_min)	REMOVE
structural ordering on all data	Phase 9b's first form	mode < mode, None < Some, lexicographic pairs have no design meaning	REMOVE (9c)
Set type	finite sets	oneOf_mem, oneOf_dup_irrelevant: membership is a fold	REMOVE
Interval type	ranges	a pair with a convention; inRange_spec; no case stores or compares a range	REMOVE
record type / row polymorphism	records	nested pairs with positional projection; no case needs shape-generic functions	REMOVE
kernel polymorphism (type variables, $\forall$, Λ , $\{\tau\}$)	generics	prenex fragment = family instantiation; matchTy_sound/_complete	REMOVE (surface families)
higher-rank types	polymorphic function arguments	every candidate rank ≥ 2 with a rank-1 replacement	REMOVE
existential types	hiding	Phase-8a instantiation hides; encoding is rank 2	REMOVE
general recursion	loops	total language; fold is the one eliminator	REMOVE
general quantification in expressions	$\forall x : \text{Real}$	finite $\forall/\exists$ are folds (forall_in_list); unbounded forms belong to a commitment layer	REMOVE (surface finite forms)
low-level hardware interface (polymorphic tuple, bounded polymorphism)	hardware interface declaration	65-layer hardware interface (hardware is a layer of hardware); the hardware is a layer of hardware (zero input memory)	REMOVE (surface hardware interface in the design)

Decisions that changed

Not rewritten to look inevitable:

original position	evidence that broke it	replacement
Signal and Event as kernel primitives (draft)	a signal type rejects nothing; occurrences are optional streams; cross-domain multiplicity is a window over lists	ordinary typed declarations; the window as five declarations
runtime output arbitration (draft)	the hidden policy is observable	one explicit driver per sink; combination is ordinary computation
eq/lt at every data type through a structural order (Phase 9b)	mode < mode, None < Some, lexicographic pairs have no design meaning (Phase 9c audit)	eq on data; lt on quantities; order by declaration at the surface
affine units need a point/difference sort — “missing information” (Phase 10)	chart laws, groupoid laws and the difference law hold without a sort; conversion never takes one (Phase 10b)	conversion complete as coordinate change; point/delta is optional validation
a third project kind, text, beside JSON kinds with conversion (ADR-0020)	two persistence forms for one meaning are two authorities that drift	one project; Design, Code and Split are views (ADR-0023)
an authoring overlay of partial expressions (considered for the Composer)	a second representation of the formula to keep in step with the text	the slot ? in the text; every action a byte-range edit (ADR-0028)
a relationship without inputs as “a value declaration of type B”, tested by <code>inputs.is_empty()</code> at every layer (production before ADR-0029)	no record said what the type of such a relationship <i>is</i> ; six layers each carried their own special case	one canonical type <code>domain(inputs) -> B</code> with <code>()</code> for no inputs, one predicate <code>is_unit_domain</code> , the kernel encoding by unit elimination proved exact (Phase 12)
a fixed-capacity list type with an overflow error (considered)	a runtime failure the formal model does not have; a type depending on a deployment fact	Vec under a validated bound; refusal on bounded targets (ADR-0024/0027)

Part XIII — Known Limitations and the Open Research Agenda

Formal limits, stated once

Causality is conservative for lambda-guarded cycles: `Causal` rejects them and the negative theorem does not cover them. The agreement between unfolding and tick-by-tick evaluation is proved for first-order wiring designs; higher-order closure equivalence is not. Theorem J (modular semantics) is proved for direct bindings on wiring designs with closure-free inputs; transported bindings under `MEv` and higher-order bodies are not covered, in either direction, for the same reason — a domain-indexed input for a transported port. The `StateHandler` reduction covers the tested behavior and not contexts with their own clocks or independently clocked nesting. `InstAcyclic` is too coarse for extraction; a port-level graph would subsume `flatten_causal` and `flat_causal`. Four conditions rest on the abstract evidence relation (`Monotone`, `Equivariant`, `PortSound`, `InterfaceLocal`). Hardware validation covers discrete pin and peripheral allocation with unary and binary constraints, and no numeric electrical, thermal or timing property; the explanation facility reports a first dead end, not a minimal core. Interface-level references — commitments that mention other declarations — are not modelled, so the dependency graph is over realizations only. Whether a realization may delegate its grant to a higher-order argument is untested. The buffer correspondence assumes an input source. The equation library’s evaluation lemmas assume the predicate value implements a Boolean function; the executed cases discharge it. The exact unit model is related to production floating point by a stated contract, not a theorem. Sums are encoded, not added. Two-hole operands are outside `solve` by design. No parser is modelled for the natural surface; its type annotations are the output of local inference, described, not proved. No theorem covers the Rust or Dart code.

Open research problems

Each is stated as a problem with what exists and what would resolve it; none is hidden in a “future work” sentence.

1. **A general edit/invalidation relation.** Refinement is a preorder with proved client stability; an arbitrary edit is outside it and forces a recheck of dependents. Production classifies edits into seven invalidation categories (ADR-0009). What is missing is a formal relation that says, for each edit kind, exactly which established facts survive — the theory behind incremental re-analysis. Existing: `DeclRefines`, `EnvRefines_update`, the invalidation set in `apply_edit`. Would resolve: a proved per-category preservation theorem.
2. **Commitments that depend on interface references.** A commitment today is a property id discharged by evidence over the realization; a commitment mentioning *another declaration* has no model, so the dependency graph is over realizations only and interface-level cycles are invisible (ISS-0003). Would resolve: an interface-level dependency relation and its interaction with refinement.
3. **Lambda-guarded causality precision.** `Causal` counts a reference under a lambda as instantaneous. A finer analysis — references that are only evaluated when the closure is applied, and where it is applied — would accept more designs. Would resolve: a causality judgment with application sites and its totality theorem.
4. **Higher-order closure equivalence.** `unfolds_preserves_eval` holds for wiring designs. Would resolve: an equivalence of closures across unfolding, or a proof that higher-order realizations can be first-order-normalized before unfolding.
5. **Full multi-clock component semantics.** Theorem J under `MEv` with transported bindings needs a domain-indexed consistent modular input and a proof of its existence. Production supports the case; the theorem does not.
6. **Full StateHandler elaboration.** Contexts with handler-scoped clocks and independently clocked nesting; the production surface has neither temporal modifiers nor contexts yet (ISS-0010). Would resolve: an elaboration with a preservation theorem, or a counterexample forcing a kernel construct.
7. **Affine physical arithmetic validation.** The point/difference sort as an operation-level validation, its rules (`affAdd`, `affSub`) integrated with typing at the surface, and the Composer offering only difference units for a delta slot. Would resolve: the validation and a decision to offer °C in formulas (ISS-0004).
8. **Hardware: minimal unsatisfiable cores.** `diagnose` reports a first dead end. Would resolve: a minimal core with a proof of minimality, or a decision that the first dead end is the better explanation.

9. **Numeric electrical, thermal and timing constraints.** Out of scope for the solver (DI-22); a design can be allocated and still exceed a current budget. Would resolve: a numeric constraint layer beside the finite solver, with its own soundness.
10. **A generated-code refinement proof.** The core is held to the reference evaluator by differential tests. Would resolve: a proof that lowering plus code generation refines Ev/MEv — or a verified evaluator — closing the largest deviation in Part XI.
11. **Enums and sums.** Encoded as tag × optional payload; production keeps user enums open (ISS-0005). Would resolve: a case that needs match exhaustiveness beyond the encoding, and then one eliminator term former like `fold`; or a decision that the encoding is the language.
12. **Natural surface evaluation.** Phase 11's forms are proved conservative and, at 3c6c8be, implemented; whether designers read all reading in readings: better than `all(readings, reading => ...)` is an empirical question.
13. **A designer user study.** No usability, learnability, productivity or cognitive-load claim has been tested (below).
14. **A mathematical specification backend.** The kernel is a specification; a backend rendering a design as a mathematical document (equations, dimensions, timing) for engineering hand-off does not exist.
15. **A formal verification backend.** Commitments beyond typing — monotonicity, ranges, invariants over ticks — have an evidence slot and no discharge mechanism; the exact chart model and the choice-free rational field are the first pieces of a verification story about quantities.
16. **A surface form for occurrence windows** (ISS-0001), so a bounded refinement can be recognized and a ring representation offered without a hidden transformation.
17. **Several candidate definitions with one active** (ISS-0002): whether this is a surface convenience over a write-once kernel realization.
18. **Source provision, next steps** (PRP-0001, ISS-0016): stateful transducers and a stream-level transparency theorem; a device clock with a deployment sync; how a profile's declared range discharges a Source's commitments; the output dual; whether computes is checked or trusted at the catalog; out-of-type raw readings as validation.
19. **Projection deltas and a persisted edit history** (ISS-0009); **a structural diagnostic entity for outputs** (ISS-0008); **packaging inside a component body** (ISS-0007); **a linear zip in the core** (ISS-0013).

Empirical questions, explicitly

The interaction model of Part II and Part X is fully described and no part of it has been evaluated with users. The following are hypotheses, to be tested, and nothing in this document should be read as evidence for them:

- that an unresolved typed relationship is a natural stopping point for designers, and that its cost is close to zero;
- that socket hue for identity and socket shape for value form are read correctly without type labels;
- that the three information levels place each fact where a designer looks for it, and that Explain is opened rarely and usefully;
- that the Formula Composer's slot expectations and candidate lists reduce the semantic-translation cost the workshop exposed;
- that product-language diagnostics ("This adds an angle and a time"; "Mode values have no default order") are understood and acted on;
- that Design, Code and Split are experienced as views of one thing;
- that groups, packaging and instances match how designers organize behavior;
- learnability across the vocabulary; productivity against a Node-RED- or Arduino-style baseline; cognitive load in the sense of the cognitive-dimensions framework.

The planned studies from the conference manuscript stand, and are restated below with their measures: a comparative study of designers realizing a given brief in BDL and in a dataflow tool, measuring correct first realizations, time to a checked design, and errors caught before hardware; and a field study in an industrial-design course, observing where designers stop, what they declare before defining, and what they ask Explain. Neither has been run.

Expected cognitive advantages, as hypotheses

Lower viscosity. Changing a transfer function edits one Mapping definition rather than a procedural chain of read/compute/store/write nodes.

Reduced hidden dependencies. Semantic signatures expose what a relationship consumes and produces, while the kernel rejects incompatible connections before device code exists.

Reduced premature commitment. An unresolved declaration allows a designer to commit to a relationship without committing to its implementation, sensor, or exact parameters.

This last item requires a qualification that the rest do not. Signature-first authoring is not claimed to be the spontaneous habit of every designer. The author, working on a hardware project, adopted it when the complexity of a subsystem exceeded what could be held in view at once, and did not adopt it on simpler tasks, where a signature and its definition were written in a single motion. The hypothesis is therefore narrower than a claim about how designers think. It is that conventional tools provide no legal position for an undefined relationship, so the strategy cannot be adopted even when it would help, and that BDL supplies that position at no cost. Whether inexperienced designers take up the strategy once it is available, and whether doing so improves their designs, is an empirical question and is included in the comparative study.

Better role expressiveness. Contexts, Mappings, and temporal modifiers correspond to product-design concepts rather than generic program-control constructs.

Improved error locality. A mismatch is attached to a product relationship or binding rather than surfacing later as an embedded runtime fault, and a hardware infeasibility is attached to the binding that causes it.

Comparative study

A first controlled study should compare BDL against at least two baselines: a statechart-based prototyping environment and a node-based or Arduino-style implementation workflow. Participants should be industrial-design students and practitioners with limited professional software-engineering experience.

Tasks should include specifying a sensor-to-actuator mapping; adding temporal qualification such as “for 300 ms”; adding an orthogonal safety override that competes with an interaction context for one output; replacing a sensor with a different sample rate; relating a slowly updated quantity to a fast interaction, which forces a cross-domain decision; choosing a board that cannot accommodate the design; and modifying a mapping late in the task.

Primary outcomes should not be limited to task time or a usability scale. More important measures are semantic errors in the final behavior; the number of implementation-only concepts participants must manipulate; time to detect an impossible or conflicting behavior; fidelity between verbal design intent and the elaborated model; the number and diversity of behavior alternatives explored; the quality of handoff to an engineer who did not observe the authoring session; subjective confidence calibrated against actual correctness; and whether, and at what level of task complexity, participants declare a relationship before defining it when the tool permits both.

The last measure tests the hypothesis stated above rather than assuming it. Because signature-first authoring may be a strategy that appears only above a complexity threshold, the tasks should vary in scale, and the default state of a newly created Mapping Block is itself a manipulable factor: a block that opens onto an empty formula editor and one that opens onto a signature with an explicitly legal undefined body invite different first actions. A small comparison of these two defaults is considerably cheaper than the full study.

The interaction model of Part II adds hypotheses of its own. Whether the single-driver diagnostic leads participants to an explicit combination block they can later read, whether the cross-domain question is answered correctly for a safety condition, and whether participants distinguish a valid design from a deployable one when the workspace reports them separately, are each measurable in the tasks above.

Field study

A controlled study cannot establish whether the representation fits real design practice. A second phase should embed the tool in a semester-long product-design studio or an industry project. The study should observe where unresolved declarations persist, which semantic types designers invent, where they request escape hatches, how often the single-driver condition is met by a combination rule the designer finds natural, and how often engineers

reinterpret or replace BDL artifacts during implementation. This field evidence is necessary before claiming that the language is native to industrial design rather than merely pleasant to its authors.

Risks of the design

Semantic-type proliferation. If every semantic distinction creates a visible type, the editor may become bureaucratic. The system needs reusable type libraries and sensible defaults, and the study should record which types designers invent.

Formula anxiety. Not every designer wants to write equations. Formula authoring must coexist with curves, examples, and direct manipulation.

Hidden elaboration. A context that elaborates to an activation declaration, an entry declaration, gated state, and a conditional driver is, in the elaborated design, several objects the designer did not draw. Hiding this can make runtime behavior mysterious. The explanation view of the interaction model is the proposed answer; whether designers use it, and whether it explains what they came to ask, is untested.

False confidence. A formally typed diagram can look verified even when no physical property has been checked, and a typed, causal, clock-consistent, output-complete design can be unplaceable on the chosen board. The work-space states of the interaction model exist to keep these apart, and the risk is sharpest for obligations discharged by declaration rather than by analysis.

Complexity migration into tooling. Much of what was removed from the kernel — event policies, output selection, context semantics — reappears as elaboration. The kernel is smaller and better understood; the elaborator is larger, and as of f1ce82c it exists (Part IX) and is held to the kernel by differential testing rather than by proof (Part XI). The claim that the surface is “only syntax” over the kernel is a claim about tested cases, and the untested cases are the ones most likely to demand a kernel extension.

Usability hypotheses unestablished. Every statement in this document about what designers find natural is a hypothesis, including every sentence of the interaction model. The one anecdote reported is a single author’s practice on a single project.

Related Work

This chapter places BDL against the literatures it draws from and the tools it is compared to. Each subsection ends with what distinguishes BDL, stated as a difference in *artifact* or *author*, not as a claim of superiority.

Industrial design tools and physical prototyping

Phidgets reduced the implementation cost of physical interaction by presenting hardware components through a uniform software abstraction [1]. d.tools integrated physical prototyping, statechart-based behavior, testing, and analysis for designers [2]. Exemplar addressed the same authoring cost from the opposite direction, letting designers demonstrate sensor behavior and having the system infer the recognizer [9]; in BDL that technique is one of several ways to supply a definition, attached to a signature that already fixes the relationship’s semantic boundary. The Arduino ecosystem [10] is the de facto medium of the workshop reported in Part I, and its `loop()/digitalRead/analogWrite` vocabulary is the operational structure whose missing “place for a fact” motivated the language. The difference from these systems is not that they could not represent behavior. It is that their dominant representation still asks designers to formulate substantial portions of behavior in an operational structure; BDL investigates whether the typed semantic relationship can be the primary artifact, with operational machinery elaborated underneath.

Visual programming and end-user programming

Node-RED [11] is the closest deployed representative of the flow-graph medium that Part II analyses: nodes are computations wired by message passing, and the graph is a dependency view onto an event loop. Scratch and its block-language descendants [12] showed that syntax can be removed as an obstacle without removing the program-counter model, and Part II’s argument is precisely that the program counter, not the syntax, is the cost for designers. End-user software engineering [13] documents the tension between low-threshold authoring and the errors that follow from the absence of static structure; BDL’s answer is to make the static structure — semantic identity, dimension, domain, single driver — the medium, and to make it legible through the three information levels rather than through diagnostics after the fact. Cognitive dimensions [4] supplies the vocabulary — viscosity, hidden dependencies, premature commitment — in which Part XIII’s empirical questions are posed.

Synchronous languages and functional reactive programming

The kernel’s temporal basis is that of the synchronous dataflow tradition. Lustre’s `pre` with an initial value, in a declaration-per-stream setting, is the delay primitive here [6]; the rule that a value computed at an instant is visible at the next instant, applied across domains, is the strictly-before rule; and causality as a static property follows the same line [14]. Esterel established the synchronous hypothesis under which these semantics are deterministic [15], and SCADE industrialized the tradition with a qualified code generator [16] — the destination that Part XIII’s “generated-code refinement proof” would move BDL toward. Where these languages recover clocks by a clock calculus [7], BDL requires domain identity to be declared, for the reason given in Part V. Zélus extends the lineage to hybrid systems [17], which is the direction in which the continuous-dynamics limitation would have to be addressed.

Functional reactive programming established behaviors and events as compositional abstractions for time-varying computation [18], and FrTime gave a dynamic dataflow embedding with formal semantics [19]. BDL’s reactive core is much more restricted, and the restriction is a result rather than a starting point: under a tick semantics with one domain, a signal type rejects nothing and an event type is an optional-valued stream, so neither appears in the kernel; across domains, the event buffer is five declarations over `sync` and lists (Part V).

Model-based design, statecharts and systems engineering

LabVIEW established the graphical dataflow instrument-control paradigm; Simulink with Stateflow combines dataflow blocks with hierarchical state machines; Modelica models physical systems through acausal equations with units and dimensions [20], [21], [22]. These systems are considerably more capable than BDL. The distinction is in the primary artifact and the intended author: in each of them the artifact is an executable model whose blocks denote computation and the author holds an engineering model; BDL’s artifact is a set of typed relationships that need not yet compute anything, and the author holds a product model. Statecharts [5] are the representation that BDL’s *contexts* replace for the designer: a context is a named situation with an activation condition, elaborated to gated and reset state and a conditional driver, rather than a transition table the designer maintains. SysML and model-based

systems engineering address precise system structure, requirements, and verification at a broader level [23]; BDL is intended as a front end for early design that could later export into such representations, not as a replacement.

Typed holes, live and structured editing, projectional editing

Hazelnut demonstrated that incomplete structured terms can remain statically meaningful under bidirectional typing [24], and Hazel extended this to live evaluation around holes [25]. BDL takes the idea that incompleteness is a first-class static state and relocates it: the kernel object is a named declaration whose realization is optional; holes are positional in the Hazelnut tradition and named here at the declaration level, while the Formula Composer’s slot ? (Part VII, Part X) is a positional hole *inside* a definition, elaborated as an expression with an unresolved type and never shipped. The stability result concerns clients of a declaration under refinement rather than the typing of the incomplete term itself. Projectional editors [26] edit an AST directly and render it; the Composer is deliberately not one — the text remains the source of truth and the Composer is a projection over it (ADR-0028), which is why byte-range edits, ephemeral node identity and a stale-projection policy exist at all. Cognitive dimensions [4] frames the trade-off these editors make and the one BDL makes.

Dimensional typing and units

Dimensional typing follows the units-of-measure line begun by Kennedy [27]. The contribution here is the placement — the algebra in primitive operator types with no dimension-specific rule, and the separation of dimension from nominal identity — together with the mechanized observation that the numeric baseline is the erasure. Part VII goes further than the units-of-measure literature usually does in two respects: units are *coordinates on a dimension* with a symbolic exact scale group, and affine units are charts whose conversions form a groupoid of affine maps with a proved point/difference decomposition; Modelica’s `displayUnit` [22] is the closest deployed analogue of Part VII’s presentation layer.

Effects

An earlier draft of BDL borrowed the separation between operation and interpretation from algebraic effects [28] and anticipated scoped effects for context-sensitive interpretation [29]. The kernel has no effect system. The negative result is stated narrowly in Part VIII and does not bear on effect systems in general; it bears on the formulations tried for this design problem, where a single explicit driver per output expressed everything the request-and-policy model expressed and made visible what it hid.

Resource allocation

The hardware validation layer is a finite constraint satisfaction problem with unary and binary constraints [8], and its solver is a plain backtracking search whose soundness and completeness are proved. No claim is made relative to the embedded co-design literature; the layer’s contribution is architectural — the design is never an input to the solver, and feasibility is kept as a separate, non-monotone kind of evidence — rather than algorithmic.

Formal verification, verified compilation and differential testing

The kernel is mechanized in Lean 4 [3] without external libraries, and every result in this document rests on propositional extensionality and quotient soundness alone. The relationship between the kernel and the production compiler is *specification*, not *extraction*: production reimplements the reference evaluator in Rust and holds itself to it by differential testing [30] over a corpus (Part IX). This is the weakest link in the trust chain (Part XI) and the point where a verified-compiler approach in the CompCert tradition [31] would apply; Part XIII lists a refinement proof from lowering to Ev/MEv as an open problem. The choice-free rational field of Part VII was built because importing a general-purpose library would have brought classical choice into the axiom base; that discipline is a methodological choice of this project and not a claim about the libraries.

Embedded DSLs and toolchains

Production BDL’s shape — a daemon with a project model, a compiler producing a `no_std` core, and a thin platform adapter — is a conventional embedded-DSL toolchain, and no novelty is claimed for it. What is specific is the placement of the semantics: the generated core is held to a reference evaluator that is itself held to a mechanized kernel, and the capacity, bounds and allocation questions are answered at validation rather than by a runtime allocator (Part V, Part IX).

Part XIV — Evidence Ledger, Decision Index and Extraction Notes

Evidence ledger

The index from which future papers can extract evidence. Theorem names are in KCN-judu/BDL_FV; production locations in KCN-judu/BDL at f1ce82c.

claim	formal theorem / counterexample	production implementation	production tests	decision	known limitation
refining one declaration preserves every client	local_refinement_preserves_wf	bdl::global_typing, model::apply_edit refinement classification	edit classification tests	D-11–D-16; ADR-0009	arbitrary edits are outside the order
typing consults only the type view	HasType.mono_env, refFree_env_irrelevant	bdl-check	pipeline tests	D-4	—
semantic identity is nominal; construction needs a grant	constructs_granted, no_semantic_value_without_order_checker temporal_state_preserves_semantic_identity	elaborator rep/mk inst_id_mapper checker	semantic corpus case	D-19–D-28	—
dimensions reject length + time	dimension_mismatch_rejected_erasure_counterexample	reduced typing	diagnostics tests	D-31	—
evaluation is deterministic and total on causal designs	Ev.det, reactive_total, Ev.not_of_strictCyclic	reference evaluator	delayed_cycle, property tests	D-36–D-42	lambda-guarded cycles conservative
one temporal primitive; delay is sync own	delay_is_sync_own, single_domain_embedding	StateCellId cells; sync snapshot rule	sync, agnostic corpus	D-44–D-48	—
scheduler order is unobservable	M.Ev.det, scheduling_order_observable (alternative)	step_in_order	order-independence tests	D-45	—
outputs have one explicit driver	multiple_direct_driver_excluded, single_driver_output_deterministic	bdl-output	lamp_output	D-50–D-54	—
hardware feasibility is decidable, sound and complete for the finite fragment	solve_sound, solve_complete, satisfiable_iff_solve	bdl-hardware::solve	Nano cases A–H	D-55–D-63	numeric constraints out of scope
behavior systems flatten into a well-formed design	flatten_WF, flatten_causal, flatten_wellClocked, flatten_singleDriver	bdl-system::flatten	system tests, behavior-systems-correspondence.md	D-64–D-73; ADR-0021/0022	Theorem J restricted
groups are transparent; extraction preserves behavior	Theorems A–G (rfI), orig_if_flat	.bdl/ authoring.json, packaging	group tests	D-74–D-82; ADR-0019	single-domain wiring fragment
the cross-domain window is expressible with lists	Theorem M buffer_window_correspondence bounded_summary_not_lossless	five-declaration idiom myec core	buffer, bounded_buffer, overflowing_buffer	D-83–D-87; ADR-0024/0027	input source assumed
capacity is validation; only refusal preserves semantics	sufficient_capacity_proves periodic_capacity_sufficiency negE	reserves, bdl-driver::capacity, bdl-exec-ir::bounds, refusal	3 000-tick differential	D-86; ADR-0027	static bound unproved
products and one recursor suffice for the tested equations	fold_total, arrow_not_delayable, church_fst_rank	bdl-ir prod, Fold	collections corpus	D-88–D-91; ADR-0025	no minimality theorem
rank-1 polymorphism by matching, no kernel change	matchTy_sound, matchTy_complete, instances_are_monomorphic	bdl-equations	equation tests	D-92; ADR-0025	—
the library adds no privilege	lib_expansion, Comb.noConstruct, lib_clocked, lib_eval_context_free	inlining at elaboration	—	D-94	—
nominality survives genericization	generic_preserves_identicality	checker on inlined	exI, exJ executed;	D-92; ADR-0026	—
specificity survives genericization	generic_preserves_distinctness map_keeps_concepts	diagnostic error messages !Bdbidicator sent as KKKK)topof signature/4... carriage_type arguments suppressed, Signature::is_unit_domain main shorthand_and_the_explicit_unit_domain are rna-declaration @domain,	(mixed-comparison) to the matrix elements are inverted ...	D-95–D-99; ADR-0029 amendment	ADR-0029 was published before the service should exist so it had not been revised by production

Design decision index

decision	section of this document	Lean file / theorem	production ADR	implementation location
declarations by identity; refinement vs edit	Part III	Core/Decl.lean, Env.lean	ADR-0008, ADR-0009	bdl-model
typing through the type view	Part III	Core/Typing.lean	—	bdl-check
nominal concepts, representation, grant	Part III	Core/Decl.lean, Typing.lean, Phase 3 experiments	ADR-0013	bdl-elab, bdl-check
dimensions in primitive types	Part III/VII	Core/Base.lean	ADR-0011	bdl-model::Dim
delay/sync, strictly-before	Part V	Core/Reactive.lean, Clock.lean	ADR-0004	bdl-reactive, bdl-lower
single driver	Part VIII	Core/Output.lean	ADR-0005	bdl-output
hardware outside typing	Part VIII	Validation/Hardware.lean	ADR-0006, ADR-0015	bdl-hardware, bdl-compiler
Lean is a specification	Part XI	—	ADR-0010	docs/project/formal-correspondence.md
components flatten; contracts stored	Part VI	Behavior/*.lean	ADR-0021, ADR-0022	bdl-system
groups are authoring metadata	Part VI	Behavior/Group.lean, Extract*.lean	ADR-0019	.bdl/authoring.json, Studio
one project, three views	Part IX/X	—	ADR-0023 (supersedes ADR-0020)	bdl-text, bdl-layout, Studio
lists, buffer, capacity	Part V	Core/ListData.lean, Surface/Buffer.lean, Validation/Capacity.lean	ADR-0024, ADR-0027	runtime collections, bdl-exec-ir::bounds
equations as definitional families; order by declaration	Part IV	Surface/Poly.lean, Stdlib.lean, Generic.lean	ADR-0025, ADR-0026	bdl-equations
the Composer as a projection; slots in the text	Part VII/X	Surface/Composer.lean	ADR-0028	bdl-ide::formula, Studio
units and charts	Part VII	Surface/Units.lean, Charts.lean, Rational.lean	ADR-0028 amendment	bdl-elab::units
natural binders	Part IV	Surface/Natural.lean	ADR-0028 second amendment	bdl-elab::formula::binder, Studio Composer
unit-domain canonical type; source role; A -> () rejected	Part III, Part VIII	Surface/UnitDomain.lean	ADR-0029	bdl_ir::ty, bdl-model::Signature::is_unit_domain, bdl-ide::explain
Source provision as a construction over designs	Part VIII	Surface/Provision.lean	PRP-0001 (draft)	—
LSP is an adapter	Part IX	—	ADR-0017	bdl-lsp
three information levels	Part X	—	ADR-0018	Studio
generated Rust implements the reference evaluator	Part IX	—	ADR-0016	bdl-codegen-rust, differential tests

Possible paper slices (internal)

- **PL**: the minimal kernel and its derivation by counterexample; typing through the type view; nominal concepts with grants; rank-1 by families with matching; one recursor; the capability audit. Evidence: Parts III, IV, XII.
- **Reactive / synchronous**: one temporal primitive, strictly-before, totality on causality; the object-language window and capacity as validation. Evidence: Part V.

- **HCI / UIST:** the Formula Composer as a projection with slots in the text; socket hue and shape; three information levels; Design/Code/Split; groups as cognitive structure with proved transparency. Evidence: Parts X, VI, XIII (needs the user study).
- **Embedded systems:** deterministic step per domain, the two-phase tick and sync snapshot, allocation-free cores, bounded collections with refusal, differential testing. Evidence: Parts IX, V.
- **Formal methods:** chart semantics over an abstract field with a choice-free rational instance; exact symbolic scales; the AffSort revision as a case study in testing a smaller hypothesis. Evidence: Part VII.
- **Design research:** behavior as design material; the representation problem; signature-first as a legal stopping state; the semantic-translation cost. Evidence: Parts I, II, XIII.

Closing

Modern industrial products increasingly combine physical form with sensing, computation, and control, yet designers still lack a behavior medium with the immediacy that CAD provides for geometry. BDL proposes that the missing medium should not be a friendlier version of procedural programming. It should be a language in which typed product relationships are first-class design artifacts, and in which an unresolved relationship is a legal state of the design rather than a defect in a program.

What that looks like in use is a workspace in which a designer names what the product is about, draws the relationships between those things, and leaves each undefined until there is something to say; refines them locally, by formula or curve or example, without the diagram changing shape; states time as a qualifier rather than as a timer; gives situations a name and a boundary rather than a transition table; is asked for one final target where two behaviors reach for one output, and for one stated way of seeing across a boundary where two quantities move at different speeds; and chooses a board last, receiving either a pin allocation or a conflict, with the design itself untouched either way.

The kernel that supports this is small, and it is small for reasons that were checked. A declaration has a frozen type, growable public commitments, and a write-once body; clients are typed against the type view, and refinement preserves what they established while edits reopen it. Semantic concepts are nominal, represented through a write-once binding, and constructed only where a signature announces them; dimensions are carried by the types of primitive operators. One temporal primitive reads a clock domain at its previous activation, and single-domain delay is its diagonal; evaluation is deterministic and total exactly on causal designs; every designer-facing temporal operator is a shape over it. Clock domains are nominal and checked by a judgment rather than a type; crossings are explicit, initialized, and strictly earlier. Physical outputs are nominal sinks with one driver each, and every combination of behaviors is ordinary computation upstream of the drive edge. A separate validation layer decides whether the design fits a board, and its evidence is kept apart from the evidence that survives refinement.

Each of these is minimal among the designs that were tested, and this document has tried to say, for each, what was proved, what was rejected by counterexample, and what was preferred. The elaborator, the editor and the firmware path have since been built and are described in Parts IX–X; the studies have not, and they are where the claims about designers would be tested. The research question is not whether designers can be taught a simpler programming language. It is whether product behavior can become a *design material* whose structure is intuitive at the surface and rigorous underneath, and the kernel presented here is the part of that question that can now be stated precisely.

References

- [1] S. Greenberg and C. Fitchett, “Phidgets: Easy Development of Physical Interfaces through Physical Widgets,” in *Proceedings of the 14th Annual ACM Symposium on User Interface Software and Technology*, ACM, 2001, pp. 209–218. doi: 10.1145/502348.502388.
- [2] B. Hartmann *et al.*, “Reflective Physical Prototyping through Integrated Design, Test, and Analysis,” in *Proceedings of the 19th Annual ACM Symposium on User Interface Software and Technology*, ACM, 2006, pp. 299–308. doi: 10.1145/1166253.1166300.
- [3] L. de Moura and S. Ullrich, “The Lean 4 Theorem Prover and Programming Language,” in *Automated Deduction – CADE 28*, in Lecture Notes in Computer Science, vol. 12699. Springer, 2021, pp. 625–635. doi: 10.1007/978-3-030-79876-5_37.
- [4] T. R. G. Green and M. Petre, “Usability Analysis of Visual Programming Environments: A 'Cognitive Dimensions' Framework,” *Journal of Visual Languages & Computing*, vol. 7, no. 2, pp. 131–174, 1996, doi: 10.1006/jvlc.1996.0009.
- [5] D. Harel, “Statecharts: A Visual Formalism for Complex Systems,” *Science of Computer Programming*, vol. 8, no. 3, pp. 231–274, 1987, doi: 10.1016/0167-6423(87)90035-9.
- [6] N. Halbwachs, P. Caspi, P. Raymond, and D. Pilaud, “The Synchronous Data Flow Programming Language LUSTRE,” *Proceedings of the IEEE*, vol. 79, no. 9, pp. 1305–1320, 1991, doi: 10.1109/5.97300.
- [7] J.-L. Colaço and M. Pouzet, “Clocks as First Class Abstract Types,” in *Proceedings of the Third ACM International Conference on Embedded Software*, ACM, 2003.
- [8] R. Dechter, *Constraint Processing*. Morgan Kaufmann, 2003.
- [9] B. Hartmann, L. Abdulla, M. Mittal, and S. R. Klemmer, “Authoring Sensor-Based Interactions by Demonstration with Direct Manipulation and Pattern Recognition,” in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, ACM, 2007, pp. 145–154. doi: 10.1145/1240624.1240646.
- [10] Arduino, “Arduino Language Reference.” 2024.
- [11] OpenJS Foundation, “Node-RED: Low-code programming for event-driven applications.” 2024.
- [12] M. Resnick *et al.*, “Scratch: Programming for All,” *Communications of the ACM*, vol. 52, no. 11, pp. 60–67, 2009.
- [13] A. J. Ko *et al.*, “The State of the Art in End-User Software Engineering,” *ACM Computing Surveys*, vol. 43, no. 3, p. 21:1–21:44, 2011.
- [14] J.-L. Colaço, B. Pagano, and M. Pouzet, “A Conservative Extension of Synchronous Data-flow with State Machines,” in *Proceedings of the 5th ACM International Conference on Embedded Software*, ACM, 2005. doi: 10.1145/1086228.1086261.
- [15] G. Berry and G. Gonthier, “The Esterel Synchronous Programming Language: Design, Semantics, Implementation,” *Science of Computer Programming*, vol. 19, no. 2, pp. 87–152, 1992, doi: 10.1016/0167-6423(92)90005-V.
- [16] G. Berry, “SCADE: Synchronous Design and Validation of Embedded Control Software,” in *Next Generation Design and Verification Methodologies for Distributed Embedded Control Systems*, Springer, 2007, pp. 19–33.
- [17] T. Bourke and M. Pouzet, “Zélus: A Synchronous Language with ODEs,” in *Proceedings of the 16th International Conference on Hybrid Systems: Computation and Control*, ACM, 2013, pp. 113–118. doi: 10.1145/2461328.2461348.
- [18] C. Elliott and P. Hudak, “Functional Reactive Animation,” in *Proceedings of the Second ACM SIGPLAN International Conference on Functional Programming*, ACM, 1997, pp. 263–273. doi: 10.1145/258948.258973.
- [19] G. H. Cooper and S. Krishnamurthi, “Embedding Dynamic Dataflow in a Call-by-Value Language,” in *Programming Languages and Systems: 15th European Symposium on Programming*, Springer, 2006, pp. 294–308.

- [20] National Instruments, “LabVIEW.” 2024.
- [21] The MathWorks, “Simulink and Stateflow.” 2024.
- [22] Modelica Association, “Modelica Language Specification, Version 3.6.” 2023.
- [23] Object Management Group, “OMG Systems Modeling Language (SysML) Version 2.0.” [Online]. Available: <https://www.omg.org/spec/SysML/2.0>
- [24] C. Omar, I. Voysey, M. Hilton, J. Aldrich, and M. A. Hammer, “Hazelnut: A Bidirectionally Typed Structure Editor Calculus,” in *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*, ACM, 2017, pp. 86–99. doi: 10.1145/3009837.3009900.
- [25] C. Omar, I. Voysey, R. Chugh, and M. A. Hammer, “Live Functional Programming with Typed Holes,” *Proceedings of the ACM on Programming Languages*, vol. 3, no. POPL, p. 14:1–14:32, 2019, doi: 10.1145/3290327.
- [26] M. Voelter, J. Siegmund, T. Berger, and B. Kolb, “Towards User-Friendly Projectional Editors,” in *Software Language Engineering (SLE)*, in LNCS, vol. 8706. 2014, pp. 41–61.
- [27] A. Kennedy, “Relational Parametricity and Units of Measure,” in *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, ACM, 1997, pp. 442–455. doi: 10.1145/263699.263761.
- [28] G. D. Plotkin and M. Pretnar, “Handling Algebraic Effects,” *Logical Methods in Computer Science*, vol. 9, no. 4, 2013, doi: 10.2168/LMCS-9(4:23)2013.
- [29] Z. Yang, M. Paviotti, N. Wu, B. van den Berg, and T. Schrijvers, “Structured Handling of Scoped Effects,” *Proceedings of the ACM on Programming Languages*, vol. 6, no. ICFP, 2022, doi: 10.1145/3547654.
- [30] W. M. McKeeman, “Differential Testing for Software,” *Digital Technical Journal*, vol. 10, no. 1, pp. 100–107, 1998.
- [31] X. Leroy, “Formal Verification of a Realistic Compiler,” *Communications of the ACM*, vol. 52, no. 7, pp. 107–115, 2009.